

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC TÂY ĐÔ
KHOA KỸ THUẬT - CÔNG NGHỆ**



Trí tuệ - Sáng tạo - Năng động - Đổi mới

**KHÓA LUẬN TỐT NGHIỆP
HỆ THỐNG RAG HỖ TRỢ TRA CỨU
PHÁP LUẬT GIAO THÔNG VIỆT NAM**

**CHUYÊN NGÀNH
CÔNG NGHỆ THÔNG TIN**

**SINH VIÊN THỰC HIỆN
QUÁCH TRƯỜNG PHÚC - 227060168**

Cần Thơ, tháng 09 năm 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC TÂY ĐÔ
KHOA KỸ THUẬT - CÔNG NGHỆ**



Trí tuệ - Sáng tạo - Năng động - Đổi mới

**KHÓA LUẬN TỐT NGHIỆP
HỆ THỐNG RAG HỖ TRỢ TRA CỨU
PHÁP LUẬT GIAO THÔNG VIỆT NAM**

**CHUYÊN NGÀNH
CÔNG NGHỆ THÔNG TIN**

**SINH VIÊN THỰC HIỆN
QUÁCH TRƯỜNG PHÚC - 227060168**

**GIẢNG VIÊN HƯỚNG DẪN
THS. NGUYỄN CHÍ CƯỜNG**

Cần Thơ, tháng 09 năm 2026

LỜI CẢM ƠN

Khóa luận được thực hiện trong quá trình học tập tại Trường Đại học Tây Đô, Khoa Kỹ thuật - Công nghệ và chuyên ngành Công nghệ thông tin. Tác giả trân trọng cảm ơn Ban Giám hiệu Nhà trường, quý Thầy, Cô của Khoa và Bộ môn Công nghệ thông tin đã cung cấp kiến thức, môi trường học tập và điều kiện cần thiết cho đề tài.

Tác giả cảm ơn ThS. Nguyễn Chí Cường đã hướng dẫn và góp ý trong quá trình thực hiện khóa luận. Các buổi trao đổi giúp tác giả xác định phạm vi đề tài, cách kiểm tra trích dẫn và phương pháp đánh giá hệ thống bằng kết quả chạy thực tế.

Tác giả cảm ơn gia đình, bạn bè và những người đã hỗ trợ, động viên trong quá trình học tập và hoàn thiện đề tài.

Sinh viên thực hiện

Quách Trường Phúc

LỜI CAM ĐOAN

Tác giả cam đoan khóa luận này là kết quả nghiên cứu và thực hiện của chính tác giả, không sao chép dưới bất kỳ hình thức nào và không thuê hoặc nhờ người khác thực hiện thay.

Các nội dung tham khảo từ công trình, tài liệu kỹ thuật và nguồn dữ liệu bên ngoài đều được ghi nhận trong phần Tài liệu tham khảo. Các số liệu thực nghiệm, chỉ số đánh giá và kết quả hệ thống chỉ được trình bày như kết quả đạt được khi có bằng chứng đo lường tương ứng. Những hạng mục chưa hoàn thành tại thời điểm viết được ghi rõ trạng thái, không được mô tả như kết quả đã xác nhận.

Cần Thơ, ngày, tháng, năm 2026

Sinh viên thực hiện

(Ký tên và ghi rõ họ và tên)

Quách Trường Phúc

TÓM TẮT

Khóa luận xây dựng hệ thống RAG hỗ trợ tra cứu pháp luật giao thông Việt Nam. Hệ thống tìm các đoạn văn bản pháp luật liên quan trước khi mô hình ngôn ngữ tạo câu trả lời. Người dùng đặt câu hỏi bằng tiếng Việt và có thể mở lại văn bản, Điều, Khoản hoặc Điểm được dùng làm căn cứ.

Hệ thống quản lý dữ liệu pháp luật bằng tệp manifest và các văn bản Markdown cục bộ. Manifest hiện khai báo 17 văn bản, gồm Nghị định và Thông tư giao thông. Bộ xử lý Markdown nhận diện Điều, Khoản và Điểm theo các quy tắc đã định nghĩa, tạo đoạn dữ liệu có metadata rồi ghi ra tệp JSONL. Mỗi đoạn có mã nhận dạng, thông tin văn bản, vị trí pháp lý và mã băm SHA-256 để kiểm tra nguồn dữ liệu.

Khâu tìm kiếm dùng Qdrant ở chế độ kết hợp. Dense embedding tìm theo ý nghĩa, sparse BM25 tìm theo từ khóa và số hiệu. Khi câu hỏi nêu rõ Điều, Khoản, Điểm hoặc số hiệu văn bản, hệ thống đối chiếu metadata trước. Với câu hỏi nhiều ý, bộ phân tích tách thành các yêu cầu nhỏ, tìm kiếm riêng rồi gộp thứ hạng bằng Reciprocal Rank Fusion. Hệ thống có thể bổ sung đoạn cùng Điều hoặc đoạn được tham chiếu trong giới hạn đã cấu hình để không bỏ sót mức phạt, điểm bị trừ hoặc biện pháp liên quan.

Trước khi gọi mô hình ngôn ngữ, hệ thống kiểm tra các đoạn đã truy xuất. Nếu số lượng hoặc tham chiếu không đạt điều kiện, API trả thông báo chưa đủ căn cứ và dừng xử lý. Nếu đạt, OpenRouter nhận câu hỏi cùng các đoạn pháp luật để tạo phần diễn giải. Backend dựng trích dẫn từ metadata. Supabase lưu thông tin đăng ký, đăng nhập, hồ sơ, phiên chat, tin nhắn, phản hồi và bookmark. Corpus pháp luật và chỉ mục Qdrant được lưu riêng.

Bộ đánh giá gồm 40 trường hợp kiểm thử. Các chỉ số tự động đo khả năng tìm căn cứ, độ chính xác ở cấp văn bản, Điều, Khoản và Điểm, tính hợp lệ của trích dẫn, độ chính xác của hành vi từ chối trả lời và thời gian phản hồi. Nội dung câu trả lời được đánh giá thủ công để tách chất lượng truy xuất khỏi chất lượng diễn giải.

ABSTRACT

This thesis develops a Retrieval-Augmented Generation (RAG) system for questions about Vietnamese traffic law. The system retrieves relevant legal passages before sending the question and retrieved text to a language model. Each answer links to its legal source at document, Article, Clause, or Point level.

The legal corpus uses a source manifest and local Markdown files. The current manifest contains 17 traffic-law documents. The Markdown ingestion module recognizes Article, Clause, and Point markers, creates LangChain Document objects with legal metadata, and writes the processed chunks to JSONL. Each chunk has an identifier, source metadata, and a SHA-256 content hash.

For retrieval, the system uses a local Qdrant hybrid index. Dense embeddings handle semantic similarity, while sparse BM25 handles keywords and legal identifiers. Explicit Article, Clause, Point, and document references are checked against metadata. Compound questions are split into a limited number of legal intents, retrieved separately, and merged with Reciprocal Rank Fusion. The retriever may add a limited number of sibling or cross-reference passages.

Before generation, the backend checks whether the retrieved material satisfies the configured evidence conditions. If the check fails, the API returns an abstention response. If it passes, the selected passages are sent to a ChatOpenRouter model with a Vietnamese prompt. Citations come from stored metadata. Supabase stores authentication and application data, including profiles, chat sessions, messages, feedback, and bookmarks. The legal corpus and Qdrant index are stored separately.

The evaluation set contains 40 cases. It measures retrieval hits, legal-coordinate accuracy, citation validity, abstention behavior, and latency. Answer correctness is reviewed manually. The design documentation covers the data model, use cases, system-flow diagrams, API, deployment, and user-interface views.

MỤC LỤC

LỜI CẢM ƠN.....	i
LỜI CAM ĐOAN	ii
TÓM TẮT	iii
ABSTRACT	iv
DANH MỤC BẢNG.....	vi
DANH MỤC HÌNH	vii
DANH MỤC TỪ VIẾT TẮT	viii
NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN	x
CHƯƠNG I. MỞ ĐẦU	1
1.1 Đặt vấn đề	1
1.2 Lý do chọn đề tài	2
1.3 Mục tiêu nghiên cứu.....	2
1.3.1 Mục tiêu tổng quát.....	2
1.3.2 Mục tiêu cụ thể	2
1.4 Câu hỏi nghiên cứu	3
1.5 Đối tượng nghiên cứu	3
1.6 Phạm vi nghiên cứu	3
1.6.1 Phạm vi dữ liệu.....	3
1.6.2 Phạm vi chức năng	3
1.6.3 Phạm vi kỹ thuật	3
1.6.4 Phạm vi đánh giá	3
1.7 Phương pháp nghiên cứu	4
1.7.1 Khảo sát tài liệu kỹ thuật.....	4
1.7.2 Phân tích yêu cầu hệ thống	4
1.7.3 Thiết kế hệ thống	4
1.7.4 Kiểm thử và đánh giá	4
1.8 Hướng giải quyết.....	5
1.9 Kế hoạch thực hiện	5
1.10 Cấu trúc khóa luận	6
CHƯƠNG II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ	7
2.1 Tổng quan về Retrieval-Augmented Generation	7
2.2 Cấu trúc văn bản pháp luật và cách chia dữ liệu	7

2.2.1	Điều, Khoản và Điểm	7
2.2.2	Metadata của một đoạn pháp luật	8
2.3	Manifest, Markdown và JSONL	9
2.4	Tra cứu định danh pháp lý chính xác	9
2.5	Tìm kiếm sparse bằng BM25	10
2.6	Tìm kiếm dense bằng embedding	11
2.7	Tìm kiếm kết hợp trong Qdrant	11
2.8	Phân tích câu hỏi bằng quy tắc xác định trước	12
2.9	Reciprocal Rank Fusion	13
2.10	Bổ sung đoạn cùng Điều và tham chiếu liên quan	13
2.11	Kiểm tra căn cứ và cơ chế từ chối trả lời	14
2.12	Sinh câu trả lời có căn cứ qua OpenRouter	15
2.13	Trích dẫn được tạo từ metadata	15
2.14	FastAPI và Pydantic	16
2.15	Python 3.11 và các thành phần backend cơ bản	17
2.16	HTTP, REST và dữ liệu JSON	17
2.17	ASGI và Uvicorn	18
2.18	LangChain trong phạm vi dự án	18
2.19	Qdrant, FastEmbed và chỉ mục tìm kiếm	19
2.20	Supabase, PostgreSQL và Row Level Security	19
2.21	Cơ sở dữ liệu quan hệ và PostgreSQL	20
2.22	Supabase Auth, JWT và phân quyền	21
2.23	Next.js, React và TypeScript	21
2.23.1	React component, props và state	22
2.23.2	TypeScript và kiểm tra kiểu tĩnh	23
2.23.3	PDF.js và React Markdown	23
2.24	Docker Compose và cấu hình môi trường	24
2.25	Quản lý thư viện và công cụ chất lượng mã nguồn	24
2.26	Tổ chức các lớp dữ liệu	25
2.27	Kiểm thử phần mềm	26
2.28	Đánh giá hệ thống RAG	26
2.28.1	Retrieval hit tại k	26
2.28.2	Độ chính xác theo tọa độ pháp lý	26
2.28.3	Citation validity	26
2.28.4	Abstention accuracy	26
2.28.5	Độ trễ	26
2.28.6	Đánh giá nội dung thủ công	26
2.29	Tài liệu kỹ thuật dùng để xây dựng hệ thống	27
2.30	Tổng hợp công nghệ sử dụng	27
2.31	Kết luận chương	28
CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG		29

3.1	Mục đích của chương	29
3.2	Xác định tác nhân	29
3.3	Yêu cầu chức năng	29
3.4	Yêu cầu phi chức năng.....	30
3.5	Sơ đồ phân rã chức năng	30
3.6	Sơ đồ luồng dữ liệu.....	31
3.6.1	DFD mức ngữ cảnh.....	31
3.6.2	DFD mức 0	31
3.7	Sơ đồ Use Case	32
3.7.1	Use Case tổng quát	32
3.7.2	Use Case xác thực	32
3.7.3	Use Case hỏi đáp pháp luật.....	33
3.7.4	Use Case nguồn pháp luật	33
3.7.5	Use Case lịch sử hội thoại	34
3.7.6	Use Case feedback và bookmark	34
3.7.7	Use Case ingestion và index bằng CLI	34
3.8	Sơ đồ hoạt động	35
3.8.1	Hoạt động đăng ký và đăng nhập	35
3.8.2	Hoạt động hỏi đáp pháp luật.....	35
3.8.3	Hoạt động xem nguồn pháp luật	36
3.8.4	Hoạt động ingestion và tạo index	37
3.8.5	Hoạt động đánh giá	37
3.9	Sơ đồ tuần tự.....	38
3.9.1	Đăng nhập	38
3.9.2	Hỏi đáp end-to-end	38
3.9.3	Mở citation.....	39
3.9.4	Ingestion và index	39
3.9.5	Feedback và bookmark	40
3.10	Kiến trúc thành phần	40
3.11	Kiến trúc triển khai	41
3.12	Tổ chức module backend và frontend	41
3.12.1	Backend	41
3.12.2	Frontend	42
3.13	Thiết kế cơ sở dữ liệu.....	43
3.13.1	Nguyên tắc phân tách dữ liệu.....	43
3.13.2	Mô hình dữ liệu khái niệm CDM/MCD	43
3.13.3	Mô hình dữ liệu logic LDM/MLD	43
3.13.4	Mô hình dữ liệu vật lý PDM/MPD.....	44
3.13.5	Sơ đồ ERD	45
3.13.6	Bảng profiles	45
3.13.7	Bảng chat_sessions	45
3.13.8	Bảng messages	46

3.13.9	Bảng feedback	47
3.13.10	Bảng bookmarks	47
3.13.11	Chỉ mục, trigger và ràng buộc toàn vẹn	48
3.13.12	Row Level Security	48
3.14	Thiết kế dữ liệu retrieval trong Qdrant	48
3.15	Thiết kế manifest và chunks JSONL	49
3.15.1	Manifest	49
3.15.2	Chunks JSONL	49
3.16	Thiết kế API	50
3.17	Thiết kế các giải thuật chính	51
3.17.1	Giải thuật tách Điều, Khoản và Điểm	51
3.17.2	Giải thuật retrieval	51
3.17.3	Giải thuật phân tích câu hỏi nhiều ý	52
3.17.4	Giải thuật gộp RRF	52
3.17.5	Giải thuật evidence gate	52
3.17.6	Giải thuật dựng citation	52
3.18	Thiết kế bảo mật	52
3.19	Thiết kế kiểm thử và đánh giá	53
3.20	Kết luận chương	54
CHƯƠNG IV. KẾT QUẢ THỰC HIỆN VÀ GIAO DIỆN		55
4.1	Tổng quan kết quả triển khai	55
4.2	Giao diện trang đầu	55
4.3	Đăng ký và đăng nhập	55
4.3.1	Đăng ký	55
4.3.2	Đăng nhập	56
4.4	Giao diện hỏi đáp pháp luật	57
4.4.1	Trạng thái chưa có câu hỏi	57
4.4.2	Câu trả lời có căn cứ	57
4.4.3	Trường hợp không đủ căn cứ	58
4.5	Hiển thị citation	58
4.6	Trang nguồn pháp luật	59
4.6.1	Danh sách văn bản	59
4.6.2	Tìm kiếm và lọc	59
4.6.3	Viewer Markdown	60
4.6.4	Viewer PDF	60
4.7	Lịch sử hội thoại	61
4.8	Feedback và bookmark	61
4.9	API và khả năng vận hành	62
4.9.1	Health check	62
4.9.2	Tài liệu OpenAPI/Swagger	62
4.10	Kết quả kiểm thử	63

4.11	Kết quả đánh giá khóa luận	64
4.12	Phân tích lỗi	65
4.13	Kết luận chương	65
CHƯƠNG V. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		66
5.1	Kết quả đạt được	66
5.2	Hạn chế.....	66
5.3	Hướng phát triển	66
5.4	Kết luận chung	66
TÀI LIỆU THAM KHẢO		67

DANH MỤC BẢNG

1.2	Kế hoạch thực hiện đề tài.....	5
2.1	Các công nghệ chính trong hệ thống	27
3.1	Các tác nhân chính của hệ thống	29
3.2	Yêu cầu chức năng dự kiến dùng để đối chiếu implementation.....	29
3.3	Thiết kế bảng profiles	45
3.4	Thiết kế bảng chat_sessions	46
3.5	Thiết kế bảng messages	46
3.6	Thiết kế bảng feedback	47
3.7	Thiết kế bảng bookmarks.....	47
3.8	Các trường metadata chính trong chunk/Qdrant payload	48
3.9	Các endpoint chính của hệ thống.....	50
4.1	Các chỉ số dùng trong bộ đánh giá 40 trường hợp	64

DANH MỤC HÌNH

1.1	Bối cảnh của bài toán hỏi đáp pháp luật giao thông có kiểm chứng	1
1.2	Phạm vi dữ liệu, chức năng và đánh giá của khóa luận	4
1.3	Quy trình nghiên cứu và phát triển hệ thống	5
1.4	Kế hoạch thực hiện theo các giai đoạn	6
2.1	Mô hình RAG tổng quát: câu hỏi, truy xuất, căn cứ và sinh câu trả lời .	7
2.2	Cấu trúc đơn vị pháp lý dùng trong dữ liệu của đề tài	8
2.3	Cấu trúc một chunk pháp luật và metadata đi kèm	8
2.4	Luồng tạo dữ liệu: manifest và Markdown đến chunks JSONL	9
2.5	Tra cứu chính xác theo số hiệu văn bản, Điều, Khoản và Điểm	10
2.6	Minh họa cách BM25 ưu tiên từ khóa đặc trưng	10
2.7	Minh họa tìm kiếm ngữ nghĩa bằng dense embedding	11
2.8	Tìm kiếm kết hợp dense và sparse trong Qdrant	12
2.9	Phân tích câu hỏi và tách các yêu cầu pháp lý nhỏ	12
2.10	Minh họa gộp nhiều danh sách kết quả bằng RRF	13
2.11	Bổ sung căn cứ cùng Điều và tham chiếu liên quan	14
2.12	Bước kiểm tra căn cứ trước khi sinh câu trả lời	14
2.13	Luồng sinh câu trả lời từ các căn cứ đã truy xuất	15
2.14	Tạo và hiển thị trích dẫn từ metadata nguồn	16
2.15	Các nhóm API chính của backend FastAPI	16
2.16	Các nhóm chức năng Python được dùng trong backend	17
2.17	Vòng đời một yêu cầu HTTP giữa trình duyệt và FastAPI	18
2.18	Bảo vệ dữ liệu ứng dụng bằng Supabase Auth và RLS	19
2.19	Khóa chính, khóa ngoại, chỉ mục và ràng buộc trong dữ liệu ứng dụng .	20
2.20	Luồng Supabase Auth, JWT, auth.uid và Row Level Security	21
2.21	Kiến trúc giao diện Next.js của hệ thống	22
2.22	Component và luồng state của giao diện hỏi đáp	22
2.23	Luồng mở citation đến nguồn Markdown hoặc PDF viewer	23
2.24	Môi trường phát triển với Docker Compose và các dịch vụ liên quan ..	24
2.25	Chuỗi công cụ cài đặt, kiểm tra kiểu, lint và test của dự án	25
2.26	Kiến trúc dữ liệu: nguồn pháp luật, chỉ mục Qdrant và dữ liệu ứng dụng Supabase	25
2.27	Các nhóm chỉ số trong bộ đánh giá 40 trường hợp	27
3.1	Sơ đồ phân rã chức năng của hệ thống	30
3.2	Sơ đồ luồng dữ liệu mức ngữ cảnh	31
3.3	Sơ đồ luồng dữ liệu mức 0	31
3.4	Sơ đồ Use Case tổng quát của hệ thống	32
3.5	Use Case đăng ký và đăng nhập	32
3.6	Use Case hỏi đáp pháp luật	33
3.7	Use Case tra cứu văn bản và điều khoản pháp luật	33

3.8	Use Case quản lý lịch sử hội thoại	34
3.9	Use Case phản hồi và đánh dấu câu trả lời	34
3.10	Use Case vận hành dữ liệu và xây chỉ mục	35
3.11	Sơ đồ hoạt động đăng ký và đăng nhập	35
3.12	Sơ đồ hoạt động xử lý một câu hỏi pháp luật	36
3.13	Sơ đồ hoạt động tra cứu nguồn pháp luật	36
3.14	Sơ đồ hoạt động tạo chunks và Qdrant index	37
3.15	Sơ đồ hoạt động chạy bộ đánh giá khóa luận	37
3.16	Sơ đồ tuần tự đăng nhập	38
3.17	Sơ đồ tuần tự xử lý câu hỏi từ giao diện đến câu trả lời	38
3.18	Sơ đồ tuần tự mở nguồn từ một citation	39
3.19	Sơ đồ tuần tự tạo chunks và chỉ mục Qdrant	39
3.20	Sơ đồ tuần tự lưu feedback và bookmark	40
3.21	Sơ đồ thành phần của hệ thống	40
3.22	Sơ đồ triển khai hệ thống	41
3.23	Sơ đồ module backend	42
3.24	Sơ đồ module frontend	42
3.25	Mô hình dữ liệu khái niệm CDM/MCD	43
3.26	Mô hình dữ liệu logic LDM/MLD	44
3.27	Mô hình dữ liệu vật lý PDM/MPD	44
3.28	Sơ đồ thực thể liên kết ERD của dữ liệu ứng dụng	45
3.29	Cấu trúc point và payload trong Qdrant	49
3.30	Cấu trúc manifest nguồn pháp luật	49
3.31	Cấu trúc một dòng trong chunks JSONL	50
3.32	Bản đồ endpoint và nhóm chức năng API	51
3.33	Luồng xác thực, phân quyền và bảo vệ khóa bí mật	53
3.34	Mô hình các lớp kiểm thử và đánh giá	53
4.1	Giao diện trang đầu của hệ thống	55
4.2	Giao diện đăng ký tài khoản	56
4.3	Giao diện đăng nhập	56
4.4	Giao diện chat khi bắt đầu phiên	57
4.5	Giao diện câu trả lời pháp luật có trích dẫn	57
4.6	Giao diện khi hệ thống từ chối trả lời do thiếu căn cứ	58
4.7	Citation và thông tin văn bản nguồn	58
4.8	Danh sách nguồn pháp luật	59
4.9	Tìm kiếm văn bản và điều khoản	59
4.10	Hiển thị nguồn pháp luật dạng Markdown	60
4.11	Hiển thị PDF và điều hướng trang	60
4.12	Giao diện quản lý phiên chat	61
4.13	Giao diện gửi phản hồi cho câu trả lời	61
4.14	Giao diện đánh dấu câu trả lời	62
4.15	Kết quả health/readiness API	62

4.16 Tài liệu API tự động của FastAPI	63
4.17 Kết quả kiểm thử của hệ thống.....	63
4.18 Tổng hợp kết quả đánh giá hệ thống	64

DANH MỤC TỪ VIẾT TẮT

AI	Artificial Intelligence - Trí tuệ nhân tạo
API	Application Programming Interface - Giao diện để các phần mềm trao đổi dữ liệu
BM25	Phương pháp xếp hạng kết quả tìm kiếm theo mức độ trùng khớp từ khóa
CLI	Command-Line Interface - Giao diện dòng lệnh
LLM	Large Language Model - Mô hình ngôn ngữ lớn
MRR	Mean Reciprocal Rank - Chỉ số đánh giá vị trí của kết quả đúng đầu tiên
nDCG	Normalized Discounted Cumulative Gain - Chỉ số đánh giá chất lượng thứ tự kết quả tìm kiếm
NLP	Natural Language Processing - Xử lý ngôn ngữ tự nhiên
OCR	Optical Character Recognition - Nhận dạng chữ từ hình ảnh hoặc PDF quét
PDF	Portable Document Format - Định dạng tài liệu PDF
RAG	Retrieval-Augmented Generation - Tạo câu trả lời có kết hợp tìm kiếm tài liệu
RRF	Reciprocal Rank Fusion - Cách gộp thứ hạng từ nhiều phương pháp tìm kiếm
SQL	Structured Query Language - Ngôn ngữ truy vấn cơ sở dữ liệu
UI	User Interface - Giao diện người dùng

**ĐÁNH GIÁ KẾT QUẢ THỰC HIỆN
NIÊN LUẬN, TIỂU LUẬN, KHOÁ LUẬN**
(Học kỳ: III, Năm 2026)

TÊN ĐỀ TÀI: HỆ THỐNG RAG HỖ TRỢ TRA CỨU

PHÁP LUẬT GIAO THÔNG VIỆT NAM

GIÁO VIÊN HƯỚNG DẪN:

STT	HỌ VÀ TÊN	MSCB
1	ThS. Nguyễn Chí Cường	

SINH VIÊN THỰC HIỆN:

STT	HỌ VÀ TÊN	MSSV	THƯỞNG (Tối đa 1,0 điểm)	ĐIỂM
1	Quách Trường Phúc	227060168		

I. HÌNH THỨC (Tối đa 1,5 điểm)

Bìa (tối đa 0,5 điểm)

Đúng mẫu quy định

Bố cục (tối đa 1,0 điểm)

Nhận xét của giảng viên hướng dẫn và giảng viên chấm, mục lục, cấu trúc chương, mục và tiểu mục, phụ lục nếu có, tài liệu tham khảo.

II. NỘI DUNG (Tối đa 5,0 điểm)

Giới thiệu (tối đa 0,5 điểm)

Mô tả bài toán, mục tiêu cần đạt và hướng giải quyết.

Lý thuyết (tối đa 1,0 điểm)

Các khái niệm, phương pháp và kiến thức lý thuyết được vận dụng vào đề tài.

Ứng dụng (tối đa 3,0 điểm)

Đặc tả hệ thống (0,5 điểm).

Các mô hình dữ liệu và các ràng buộc toàn vẹn (1,0 điểm).

Sơ đồ chức năng, lưu đồ xử lý, diễn giải các màn hình và báo biểu nếu có (1,0 điểm).

Giới thiệu chương trình (0,5 điểm).

Kết luận (tối đa 0,5 điểm)

Nhận xét kết quả đạt được, hạn chế và hướng phát triển.

III. CHƯƠNG TRÌNH DEMO (Tối đa 3,5 điểm)

Giao diện thân thiện với người dùng (0,5 điểm)

Hướng dẫn sử dụng (0,5 điểm)

Kết quả thực hiện đúng với kết quả của phần ứng dụng (2,5 điểm)

Giải thuật đúng, thực thi chính xác, cơ sở dữ liệu và các ràng buộc hợp lý, màn hình cập nhật đúng, báo biểu phù hợp.

Ghi chú:

- Điểm trong khung “sinh viên thực hiện” là điểm kết quả cuối cùng của từng sinh viên trong quá trình thực hiện Niên luận/Tiểu luận/Khoá luận.
- Nếu sinh viên demo chương trình và trả lời vấn đáp không đạt yêu cầu của giáo viên hướng dẫn thì sinh viên sẽ nhận điểm F cho học phần này.

Cần Thơ, ngày tháng năm 2026

GIÁO VIÊN CHẤM

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần Thơ, ngày, tháng, năm 2026

Giáo viên hướng dẫn

ThS. Nguyễn Chí Cường

CHƯƠNG I. MỞ ĐẦU

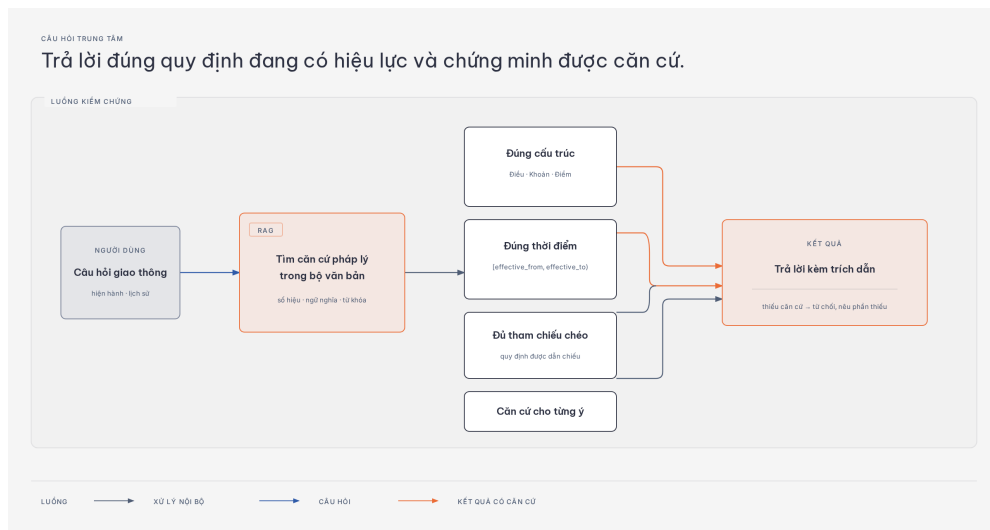
1.1 Đặt vấn đề

Mô hình ngôn ngữ tạo câu trả lời từ câu hỏi viết bằng ngôn ngữ tự nhiên. Khi dữ liệu cung cấp không đủ, mô hình vẫn có thể sinh nội dung nghe hợp lý nhưng không có căn cứ để kiểm tra. Trong khóa luận, trường hợp này được gọi là *câu trả lời không được dữ liệu nguồn hỗ trợ*. Căn cứ pháp luật dùng cho câu trả lời phải đến từ các đoạn đã được hệ thống truy xuất.

Trong pháp luật giao thông, cùng một câu hỏi về mức phạt có thể cho kết quả khác nhau theo loại phương tiện và quy định áp dụng. Ví dụ, câu hỏi “vượt đèn đỏ bị phạt bao nhiêu” cần xác định đúng văn bản, Điều, Khoản, Điểm và các biện pháp đi kèm. Một con số đứng riêng không cho người dùng biết căn cứ nào đã được dùng. Hệ thống vì thế trả kèm thông tin nguồn để người dùng mở lại văn bản gốc.

Retrieval-Augmented Generation (RAG) trong phạm vi khóa luận là cách kết hợp tìm kiếm tài liệu với sinh câu trả lời. Câu hỏi được phân tích trước, hệ thống tìm các đoạn pháp luật phù hợp, kiểm tra căn cứ rồi gọi mô hình ngôn ngữ để diễn giải. Trích dẫn được tạo từ metadata của dữ liệu nguồn, không do mô hình sinh ra.

Văn bản pháp luật Việt Nam được tổ chức theo Điều, Khoản và Điểm. Số hiệu như “168/2024/NĐ-CP” cũng cần được giữ nguyên khi tra cứu. Hệ thống dùng nội dung ngữ nghĩa từ dense embedding, từ khóa từ sparse BM25 và metadata pháp lý để đối chiếu số hiệu cùng vị trí Điều, Khoản, Điểm.



Hình 1.1: Bối cảnh của bài toán hỏi đáp pháp luật giao thông có kiểm chứng

1.2 Lý do chọn đề tài

Người dùng thường biết hành vi cần hỏi nhưng không biết chính xác văn bản chứa quy định đó. Kết quả tìm kiếm trên web có thể lẫn bài giải thích, bản sao và nguồn chính thức. Hệ thống trong đề tài giới hạn việc trả lời vào corpus đã chuẩn bị và cho phép mở lại căn cứ được dùng.

Câu hỏi pháp luật cũng có nhiều dạng. Một câu có thể nêu thẳng “Điều 7 Nghị định 168/2024/NĐ-CP”, chỉ mô tả hành vi, hoặc hỏi cùng lúc mức phạt và số điểm bị trừ. Dense search xử lý tốt các cách diễn đạt gần nghĩa. BM25 giữ lợi thế khi truy vấn chứa từ khóa, số hiệu hoặc ký hiệu xuất hiện trực tiếp trong văn bản. Qdrant cho phép kết hợp hai kiểu tìm kiếm này trong hybrid search [6].

Khóa luận yêu cầu câu trả lời phải đi kèm thông tin đủ để kiểm tra lại. Citation lưu tên văn bản, Điều, Khoản, Điểm và thông tin trang hoặc tệp nguồn khi có. Những trường này cũng được dùng trong bộ đánh giá để so sánh kết quả trả về với vị trí pháp lý mong đợi.

1.3 Mục tiêu nghiên cứu

1.3.1 Mục tiêu tổng quát

Xây dựng một hệ thống RAG hỗ trợ tra cứu pháp luật giao thông Việt Nam, cho phép người dùng đặt câu hỏi bằng tiếng Việt, nhận câu trả lời dựa trên dữ liệu pháp luật cục bộ và xem được trích dẫn có thể kiểm tra lại.

1.3.2 Mục tiêu cụ thể

1. Xây dựng quy trình đọc manifest và tài liệu Markdown, nhận diện Điều, Khoản, Điểm và tạo dữ liệu chunk có metadata rõ ràng.
2. Tạo chỉ mục Qdrant kết hợp dense embedding và sparse BM25 để phục vụ tìm kiếm theo ý nghĩa và từ khóa.
3. Nhận diện tham chiếu pháp lý xuất hiện trực tiếp trong câu hỏi, ví dụ số hiệu văn bản, Điều, Khoản và Điểm.
4. Phân tích một số dạng câu hỏi nhiều ý thành các yêu cầu nhỏ, thực hiện tìm kiếm riêng và gộp kết quả bằng RRF.
5. Chỉ cho phép sinh câu trả lời khi bước kiểm tra căn cứ đạt yêu cầu, khi thiếu căn cứ phải trả về trạng thái từ chối trả lời.
6. Tạo trích dẫn từ metadata, hỗ trợ mở nguồn Markdown hoặc PDF tương ứng.
7. Xây dựng giao diện Web gồm hỏi đáp, nguồn pháp luật, lịch sử hội thoại, phản hồi và đánh dấu câu trả lời.
8. Xây dựng cơ sở dữ liệu ứng dụng trên Supabase cho hồ sơ, phiên chat, tin nhắn, phản hồi và bookmark, kết hợp chính sách RLS để giới hạn dữ liệu theo người dùng.
9. Chuẩn bị bộ đánh giá 40 trường hợp và các chỉ số tự động cho retrieval, tọa độ pháp lý, citation, abstention và độ trễ.

1.4 Câu hỏi nghiên cứu

Khóa luận tập trung vào các câu hỏi sau:

1. Làm thế nào để kết hợp tìm kiếm theo từ khóa, theo ý nghĩa và theo định danh pháp lý trong cùng một hệ thống RAG?
2. Việc tách câu hỏi nhiều ý và gộp các danh sách kết quả ảnh hưởng như thế nào đến khả năng thu thập đủ căn cứ?
3. Có thể giảm nguy cơ tạo câu trả lời không có nguồn bằng cách đặt bước kiểm tra căn cứ trước mô hình ngôn ngữ như thế nào?
4. Cách tổ chức metadata nào giúp trích dẫn có thể mở lại đúng văn bản, Điều, Khoản hoặc Điểm?
5. Các chỉ số nào phù hợp để đánh giá một hệ thống tra cứu pháp luật ngoài việc đánh giá văn phong của câu trả lời?

1.5 Đối tượng nghiên cứu

Đối tượng nghiên cứu gồm quy trình RAG cho dữ liệu pháp luật giao thông, cách chia văn bản theo Điều, Khoản và Điểm, tìm kiếm sparse và dense, gộp thứ hạng, kiểm tra căn cứ, sinh câu trả lời có nguồn, thiết kế API, giao diện Web, cơ sở dữ liệu ứng dụng và quy trình đánh giá có thể chạy lặp lại.

1.6 Phạm vi nghiên cứu

1.6.1 Phạm vi dữ liệu

Tập data/sources/manifest.json của đề tài hiện khai báo 17 văn bản giao thông, gồm Nghị định và Thông tư. Dữ liệu Markdown cục bộ nằm trong data/corpus/mds/. Các đoạn đã xử lý được ghi vào data/processed/chunks.jsonl. PDF được dùng trong giao diện nguồn khi metadata tương ứng có sẵn. Danh sách văn bản được xem là dữ liệu nghiên cứu của đề tài, không phải toàn bộ hệ thống pháp luật Việt Nam.

1.6.2 Phạm vi chức năng

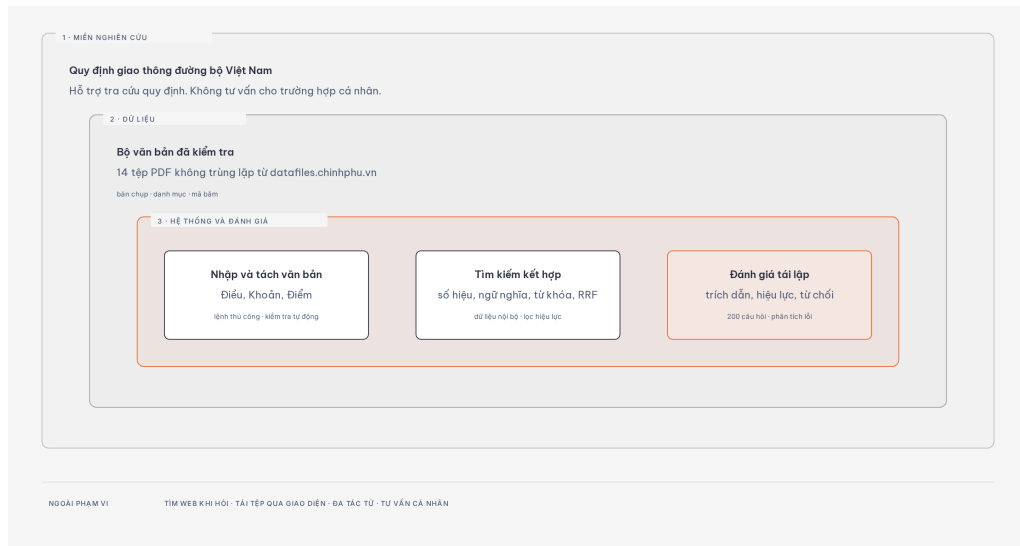
Hệ thống có các chức năng đăng ký, đăng nhập, hỏi đáp pháp luật, xem nguồn và điều khoản, quản lý lịch sử hội thoại, gửi phản hồi và đánh dấu câu trả lời. Dự án còn có các lệnh CLI để tạo chunk, xây lại chỉ mục và chạy bộ đánh giá.

1.6.3 Phạm vi kỹ thuật

Backend sử dụng Python 3.11, FastAPI và Pydantic. Tầng RAG sử dụng LangChain Document, LangChain Qdrant, Qdrant cục bộ, OpenRouter cho embedding và sinh câu trả lời, cùng FastEmbed BM25 cho tìm kiếm sparse. Dữ liệu ứng dụng dùng Supabase. Frontend dùng Next.js App Router, React và TypeScript.

1.6.4 Phạm vi đánh giá

Bộ dữ liệu đánh giá khóa luận hiện gồm 40 trường hợp. Hệ thống tự tính retrieval hit, độ chính xác ở cấp văn bản/Điều/Khoản/Điểm, tính hợp lệ của citation, abstention accuracy và độ trễ. Việc nhận xét nội dung câu trả lời cuối cùng được thực hiện thủ công, do đó khóa luận không tự suy diễn một “điểm đúng nội dung” từ các chỉ số retrieval.



Hình 1.2: Phạm vi dữ liệu, chức năng và đánh giá của khóa luận

1.7 Phương pháp nghiên cứu

1.7.1 Khảo sát tài liệu kỹ thuật

Cơ sở kỹ thuật của đề tài được xây dựng từ các khái niệm về lập trình, phân tích hệ thống, công nghệ phần mềm và phát triển Web đã học trong chương trình Công nghệ thông tin của Trường Đại học Tây Đô [1, 2, 3, 4, 5]. Các đặc điểm của FastAPI, Qdrant, Supabase, Next.js, React, TypeScript, PDF.js, Docker và OpenRouter được mô tả theo tài liệu chính thức của từng nền tảng.

1.7.2 Phân tích yêu cầu hệ thống

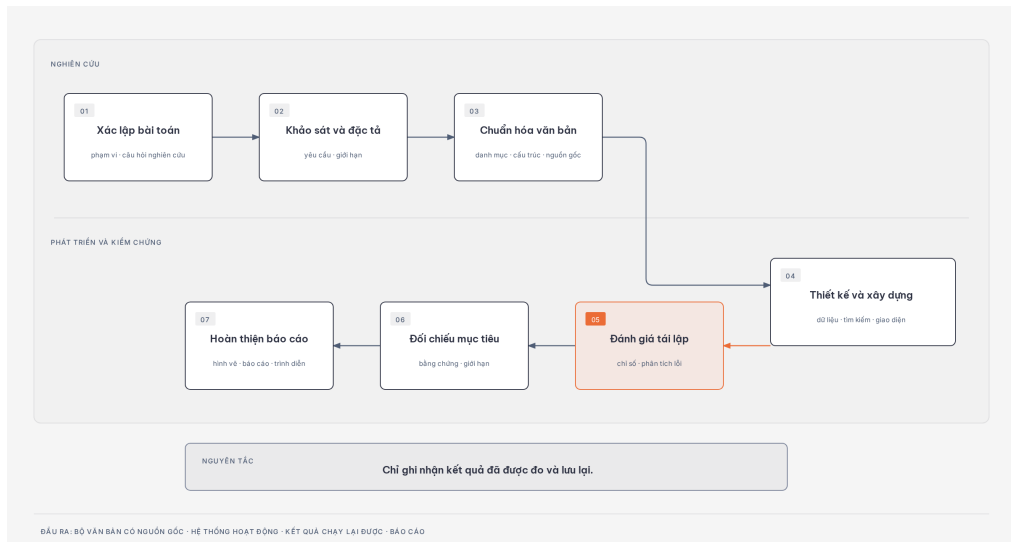
Yêu cầu hệ thống được phân tích theo tác nhân, dữ liệu đầu vào, dữ liệu đầu ra, quy tắc xử lý và ràng buộc của từng chức năng. Các yêu cầu chức năng và phi chức năng được dùng làm cơ sở cho use case, luồng dữ liệu, thiết kế API, mô hình dữ liệu và kiểm thử.

1.7.3 Thiết kế hệ thống

Từ yêu cầu chức năng, đề tài xây dựng Use Case, FDD, DFD, Activity Diagram, Sequence Diagram, Component Diagram, Deployment Diagram và các mô hình dữ liệu CDM, LDM, PDM/ERD. Mỗi mô hình mô tả một góc nhìn riêng, từ chức năng, luồng xử lý, tương tác giữa các thành phần đến cấu trúc dữ liệu.

1.7.4 Kiểm thử và đánh giá

Backend sử dụng pytest cho unit test và integration test. Giao diện dùng Playwright cho các luồng kiểm thử trình duyệt. Bộ đánh giá 40 trường hợp cho phép kiểm tra khả năng truy xuất căn cứ, vị trí pháp lý, trích dẫn, hành vi từ chối trả lời và thời gian phản hồi.



Hình 1.3: Quy trình nghiên cứu và phát triển hệ thống

1.8 Hướng giải quyết

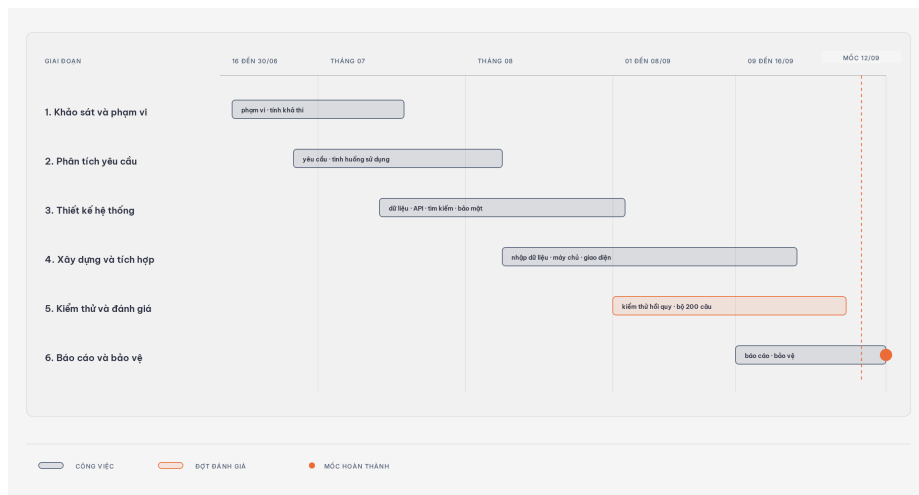
Giải pháp có hai dòng dữ liệu. Dòng dữ liệu pháp luật đi từ manifest và Markdown đến chunks JSONL rồi chỉ mục Qdrant. Dòng dữ liệu ứng dụng lưu người dùng, phiên chat, tin nhắn, feedback và bookmark trong Supabase. Việc tách hai nhóm dữ liệu giúp phân biệt kho tìm kiếm pháp luật với dữ liệu người dùng.

Khi người dùng gửi câu hỏi, hệ thống xác định các ý pháp lý cần tìm rồi thực hiện hybrid search. Nếu câu hỏi có tham chiếu cụ thể, retriever đối chiếu thêm metadata. Service dùng RRF để gộp nhiều danh sách kết quả, sau đó kiểm tra căn cứ. Chỉ khi đủ căn cứ, model generation mới được gọi. Backend dựng citation từ metadata và trả cùng câu trả lời.

1.9 Kế hoạch thực hiện

Bảng 1.2: Kế hoạch thực hiện đề tài

Giai đoạn	Nội dung	Đầu ra
1	Khảo sát tài liệu, xác định câu hỏi nghiên cứu và phạm vi	Đề cương, danh sách nguồn tham khảo
2	Chuẩn bị corpus, manifest, Markdown và quy tắc tách Điều/Khoản/Điểm	chunks JSONL có metadata
3	Xây dựng Qdrant hybrid retrieval và đối chiếu tham chiếu pháp lý	Chỉ mục và retriever
4	Xây dựng phân tích câu hỏi, RRF, mở rộng căn cứ, evidence gate	Luồng hỏi đáp backend
5	Tích hợp OpenRouter generation và citation metadata	API chat có căn cứ
6	Xây dựng Supabase auth, lịch sử chat, feedback, bookmark	Dữ liệu ứng dụng và API
7	Xây dựng Next.js UI, nguồn pháp luật và PDF viewer	Giao diện Web
8	Kiểm thử, chạy bộ 40 trường hợp và hoàn thiện demo	Kết quả đánh giá và bản demo



Hình 1.4: Kế hoạch thực hiện theo các giai đoạn

1.10 Cấu trúc khóa luận

Khóa luận gồm năm chương, đi từ bài toán, cơ sở lý thuyết và công nghệ đến phân tích, thiết kế, kết quả thực hiện, kết luận và hướng phát triển.

CHƯƠNG II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Tổng quan về Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) trong đề tài gồm hai bước chính. Hệ thống tìm các đoạn dữ liệu liên quan rồi gửi câu hỏi cùng các đoạn đó cho mô hình ngôn ngữ. Dữ liệu pháp luật nằm ngoài mô hình nên có thể mở lại và kiểm tra. Phần cài đặt đi từ dữ liệu nguồn, tạo chunk và lập chỉ mục Qdrant đến truy xuất, kiểm tra căn cứ và sinh câu trả lời.

Cách tổ chức này cho phép cập nhật corpus và xây lại chỉ mục mà không huấn luyện lại mô hình ngôn ngữ. Mỗi câu trả lời có thể gắn với đoạn nguồn đã truy xuất. Chất lượng câu trả lời phụ thuộc vào retrieval và phần diễn giải của model. Đề tài kiểm tra căn cứ trước generation và trả trạng thái từ chối khi dữ liệu chưa đủ.



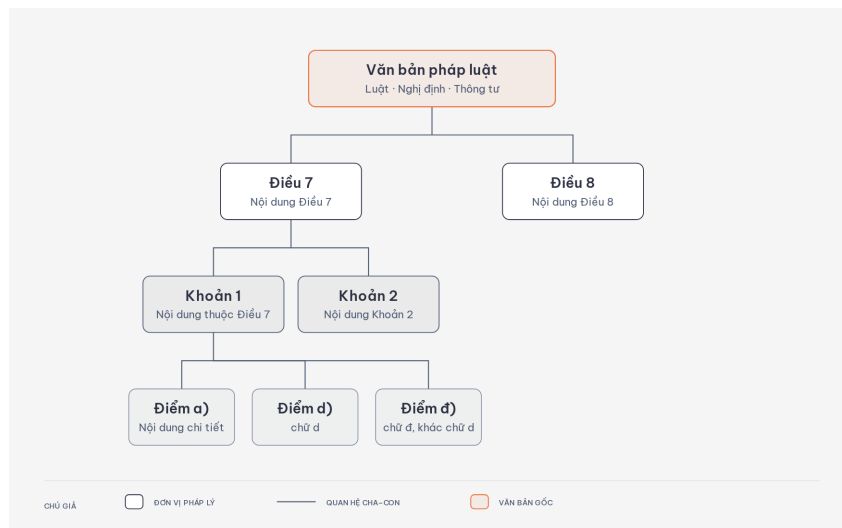
Hình 2.1: Mô hình RAG tổng quát: câu hỏi, truy xuất, căn cứ và sinh câu trả lời

2.2 Cấu trúc văn bản pháp luật và cách chia dữ liệu

2.2.1 Điều, Khoản và Điểm

Trong dữ liệu giao thông, số hiệu văn bản cùng vị trí Điều, Khoản và Điểm tạo thành thông tin định danh thường xuất hiện trong câu hỏi tra cứu. Mỗi chunk vì thế lưu metadata vị trí cùng với nội dung văn bản.

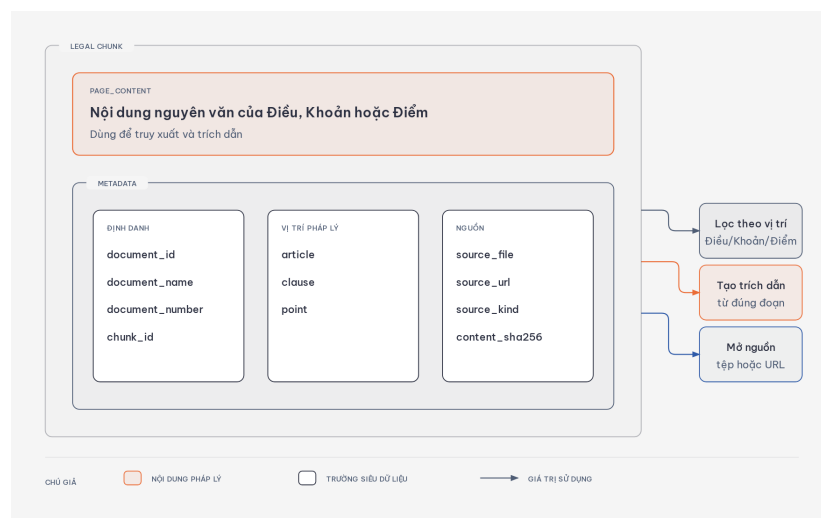
Bộ đọc Markdown dùng biểu thức chính quy để nhận diện các mẫu “Điều n”, “n.” hoặc “n)” cho Khoản và “a)”, “b)”, “đ)” cho Điểm. Khi gặp một đơn vị mới, buffer hiện tại được ghi thành một LangChain Document. Cách xử lý có tính xác định, cùng một đầu vào và cùng bộ quy tắc sẽ cho cùng cách chia.



Hình 2.2: Cấu trúc đơn vị pháp lý dùng trong dữ liệu của đề tài

2.2.2 Metadata của một đoạn pháp luật

Mỗi chunk ngoài page_content còn có metadata. Nhóm định danh gồm document_id, document_name, document_number và chunk_id. Nhóm vị trí pháp lý gồm article, clause và point. Nhóm nguồn gồm source_file, source_url, source_kind và content_sha256. Metadata giúp hệ thống lọc theo Điều/Khoản/Điểm, dựng trích dẫn và mở đúng nguồn.

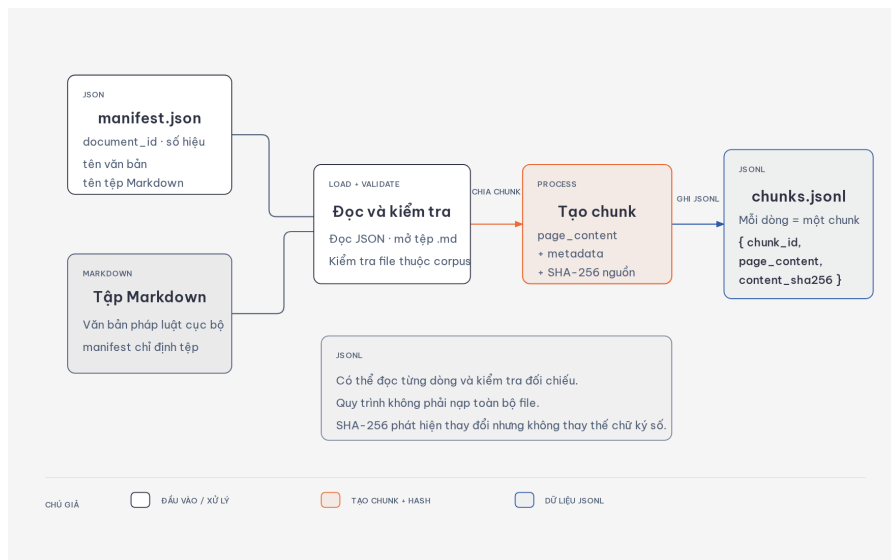


Hình 2.3: Cấu trúc một chunk pháp luật và metadata đi kèm

2.3 Manifest, Markdown và JSONL

Manifest là tập JSON liệt kê các văn bản mà hệ thống sử dụng. Mỗi phần tử có `document_id`, số hiệu, tên văn bản và tên file Markdown. Tập Markdown là nguồn cục bộ dùng để tạo chunk. Sau khi xử lý, mỗi chunk được ghi thành một dòng JSON độc lập trong `chunks.jsonl`. JSONL phù hợp cho pipeline dữ liệu vì có thể đọc từng dòng, dễ kiểm tra diff và không cần nạp toàn bộ file vào bộ nhớ.

Mã SHA-256 được tính từ nội dung nguồn và gắn vào metadata. Hàm băm không thay thế chữ ký số, nhưng hữu ích để phát hiện dữ liệu có thay đổi giữa các lần xử lý.

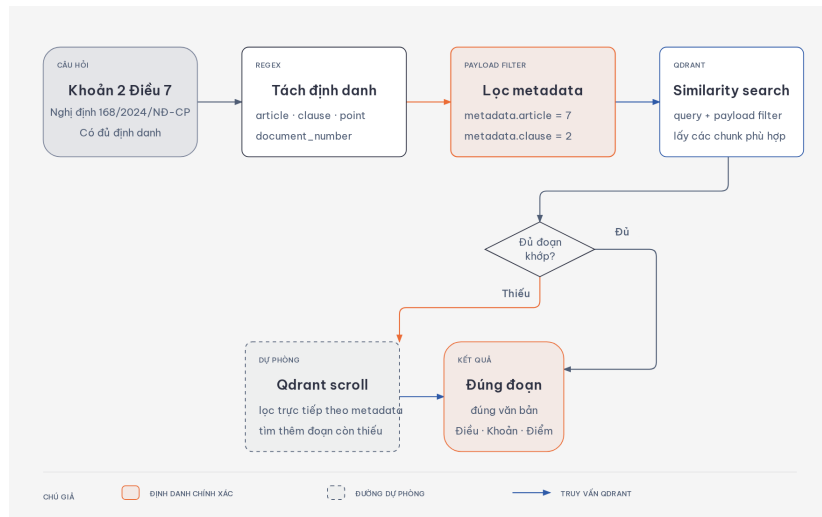


Hình 2.4: Luồng tạo dữ liệu: manifest và Markdown đến chunks JSONL

2.4 Tra cứu định danh pháp lý chính xác

Một câu hỏi có thể chứa tham chiếu rõ ràng như “Khoản 2 Điều 7 Nghị định 168/2024/NĐ-CP”. Trong trường hợp này, chỉ dựa vào vector có thể trả về đoạn gần nghĩa nhưng sai vị trí. Vì vậy, retriever của dự án dùng biểu thức chính quy để lấy `article`, `clause`, `point` và số văn bản. Các giá trị này được chuyển thành bộ lọc payload của Qdrant hoặc dùng để kiểm tra metadata sau retrieval.

Cơ chế này có đường dự phòng. Nếu similarity search không trả đủ kết quả cho một tham chiếu rõ ràng, hệ thống dùng Qdrant scroll với filter metadata để tìm đúng chunk. Cách xử lý ưu tiên định danh pháp lý khi người dùng đã cung cấp Điều, Khoản, Điểm hoặc số hiệu văn bản.



Hình 2.5: Tra cứu chính xác theo số hiệu văn bản, Điều, Khoản và Điểm

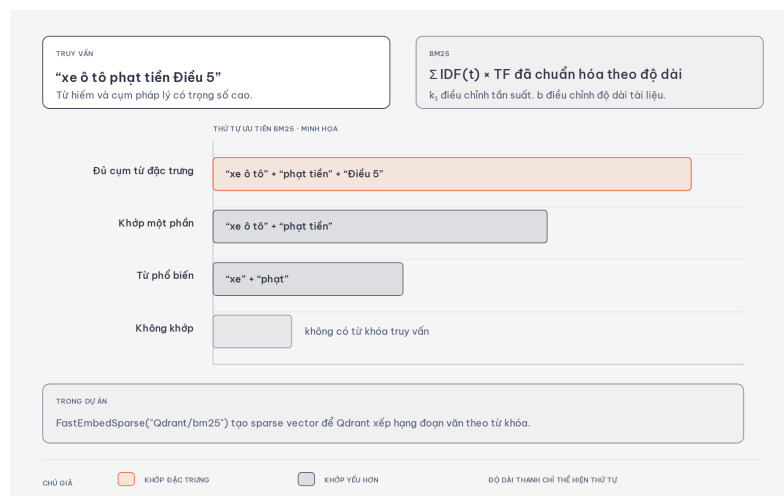
2.5 Tìm kiếm sparse bằng BM25

BM25 là phương pháp xếp hạng văn bản dựa trên mức độ xuất hiện của từ trong tài liệu, độ hiếm của từ trong toàn bộ tập dữ liệu và độ dài của tài liệu. Tài liệu chính thức của Qdrant mô tả BM25 là một cơ chế xếp hạng dùng sparse vector, trong đó trọng số từ khóa chịu ảnh hưởng của tần suất từ và IDF [7]. Một dạng công thức thường dùng có thể trình bày như sau:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}. \quad (2.1)$$

Trong đó $f(t, d)$ là số lần từ t xuất hiện trong tài liệu d , $|d|$ là độ dài tài liệu và avgdl là độ dài trung bình. k_1 và b là tham số điều chỉnh.

BM25 hữu ích trong đề tài vì các chuỗi như số Nghị định, số Điều, tên hành vi và cụm từ xử phạt cần được khớp chính xác. Qdrant hỗ trợ sparse vector và dự án sử dụng FastEmbedSparse("Qdrant/bm25") để tạo biểu diễn sparse.



Hình 2.6: Minh họa cách BM25 ưu tiên từ khóa đặc trưng

2.6 Tìm kiếm dense bằng embedding

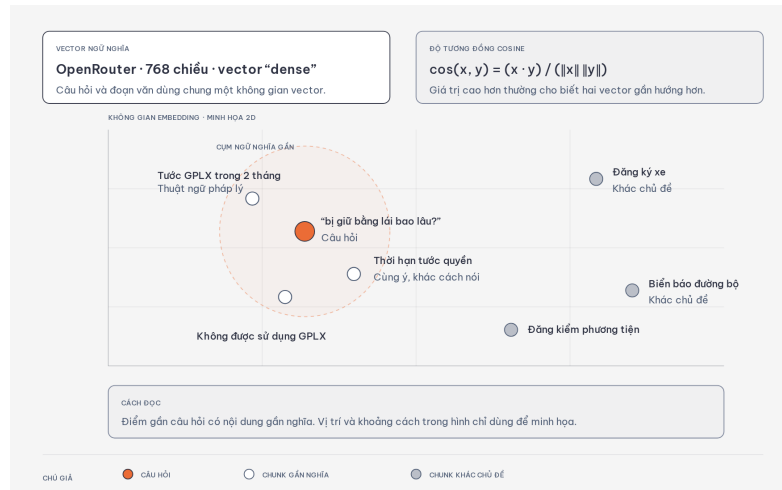
Dense embedding biểu diễn một đoạn văn bản bằng vector số có số chiều cố định. Hệ thống so sánh các vector để tìm những đoạn có biểu diễn gần câu hỏi, kể cả khi từ ngữ không trùng hoàn toàn. Qdrant lưu dense vector cùng sparse vector trong một point [8]. Dense search được dùng cho các câu hỏi diễn đạt khác từ ngữ so với văn bản gốc.

Trong dự án, dense embedding được gọi qua endpoint tương thích OpenAI của OpenRouter và cấu hình 768 chiều. Qdrant lưu vector dense dưới tên dense. Tìm kiếm dense bổ sung cho BM25 trong các trường hợp người dùng diễn đạt khác với văn bản gốc, ví dụ dùng từ đời thường thay cho cụm từ pháp lý.

Độ tương đồng thường được tính từ khoảng cách vector. Với cosine similarity, hai vector x và y có thể so sánh bằng:

$$\cos(x, y) = \frac{x \cdot y}{\|x\| \|y\|}. \quad (2.2)$$

Giá trị càng cao thường thể hiện hướng vector càng gần nhau.

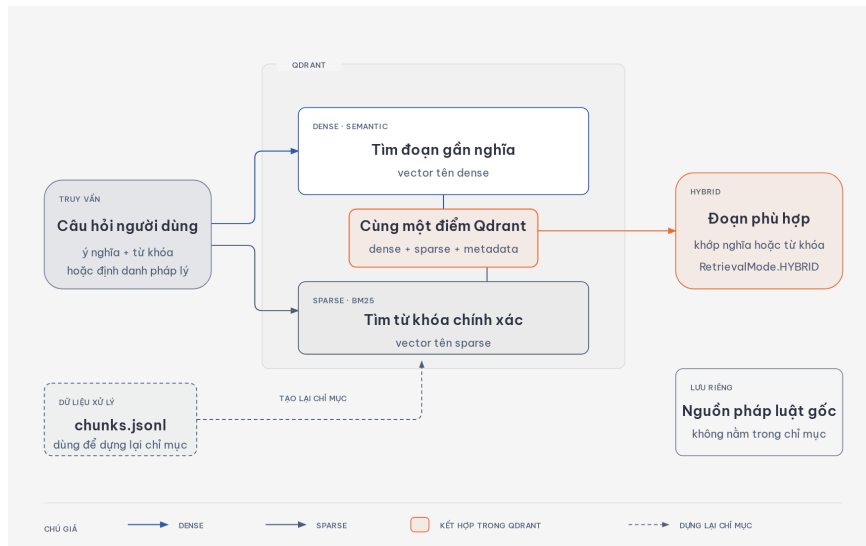


Hình 2.7: Minh họa tìm kiếm ngữ nghĩa bằng dense embedding

2.7 Tìm kiếm kết hợp trong Qdrant

Qdrant cho phép lưu nhiều dạng vector trong cùng một point và hỗ trợ kết hợp dense với sparse. Tài liệu Qdrant mô tả hybrid search như cách kết hợp semantic search với lexical search, nhờ đó hệ thống có thể xử lý cả câu hỏi diễn đạt theo ý nghĩa và truy vấn chứa từ khóa/định danh chính xác [6].

Hệ thống cấu hình QdrantVectorStore ở chế độ RetrievalMode.HYBRID, với vector dense tên dense và sparse tên sparse. Qdrant giữ chỉ mục phục vụ tìm kiếm. Khi corpus thay đổi, collection được xây dựng lại từ dữ liệu chunks JSONL. Nguồn pháp luật gốc vẫn được lưu riêng với chỉ mục tìm kiếm.

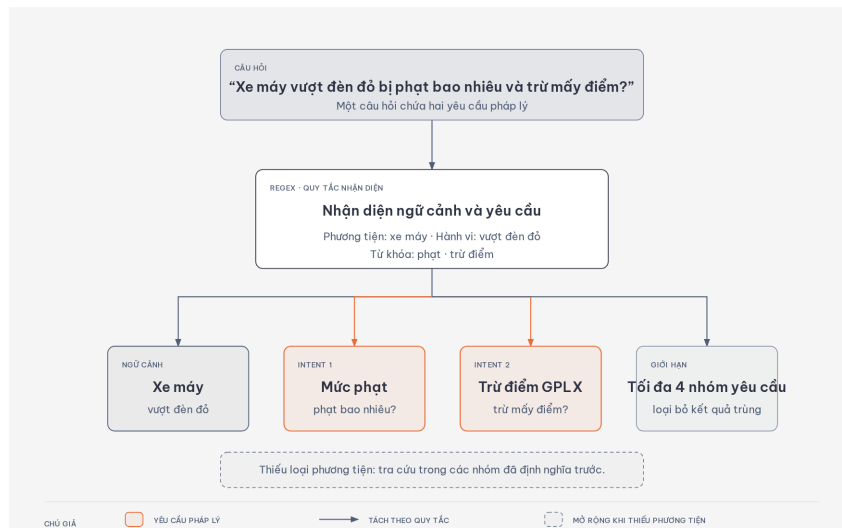


Hình 2.8: Tìm kiếm kết hợp dense và sparse trong Qdrant

2.8 Phân tích câu hỏi bằng quy tắc xác định trước

Mô-đun phân tích câu hỏi có nhiệm vụ xác định các ý pháp lý cần tìm. Hệ thống dùng biểu thức chính quy để phát hiện loại phương tiện, các từ khóa như phạt tiền, trừ điểm, tước quyền sử dụng và một số hành vi phổ biến. Câu hỏi có thể được tách thành tối đa bốn intent để giới hạn số lượng truy vấn phát sinh.

Ví dụ, câu hỏi “xe máy vượt đèn đỏ bị phạt bao nhiêu và trừ mấy điểm” được tách thành yêu cầu về mức phạt và yêu cầu về trừ điểm. Khi câu hỏi xử phạt không nêu loại phương tiện, hệ thống tạo thêm truy vấn cho một số nhóm phương tiện đã được định nghĩa trước. Cách tách và mở rộng truy vấn được xác định bằng quy tắc.



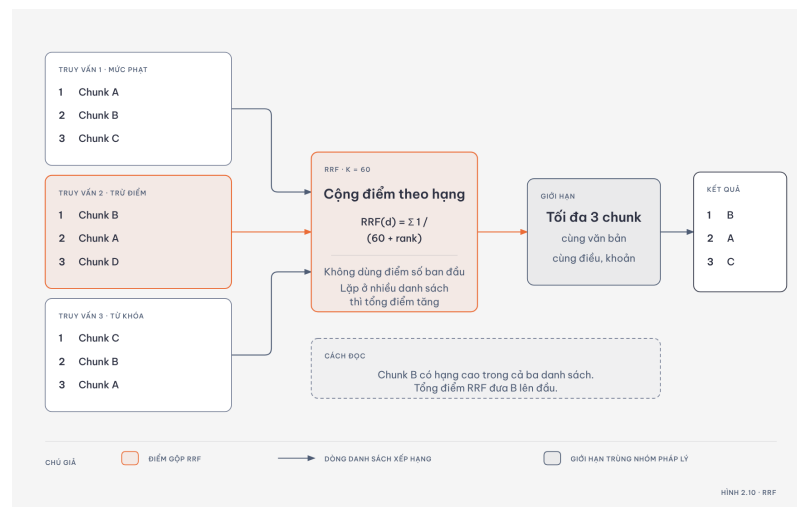
Hình 2.9: Phân tích câu hỏi và tách các yêu cầu pháp lý nhỏ

2.9 Reciprocal Rank Fusion

Khi một câu hỏi được tách thành nhiều truy vấn, mỗi truy vấn trả về một danh sách chunk đã xếp hạng. Reciprocal Rank Fusion (RRF) là cách gộp nhiều danh sách dựa trên thứ hạng của từng kết quả. Qdrant cũng hỗ trợ RRF trong hybrid query và mô tả cơ chế này là cộng điểm theo vị trí của một kết quả trong các danh sách khác nhau [6]. Công thức tổng quát:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}. \quad (2.3)$$

Trong service hiện tại, hằng số k được dùng ở dạng $60 + \text{rank}$. Chunk xuất hiện ở vị trí cao trong nhiều danh sách sẽ có điểm gộp lớn hơn. Sau khi gộp, service còn giới hạn số chunk trong cùng một nhóm văn bản/Điều/Khoản để giảm việc một vị trí pháp lý chiếm toàn bộ context.

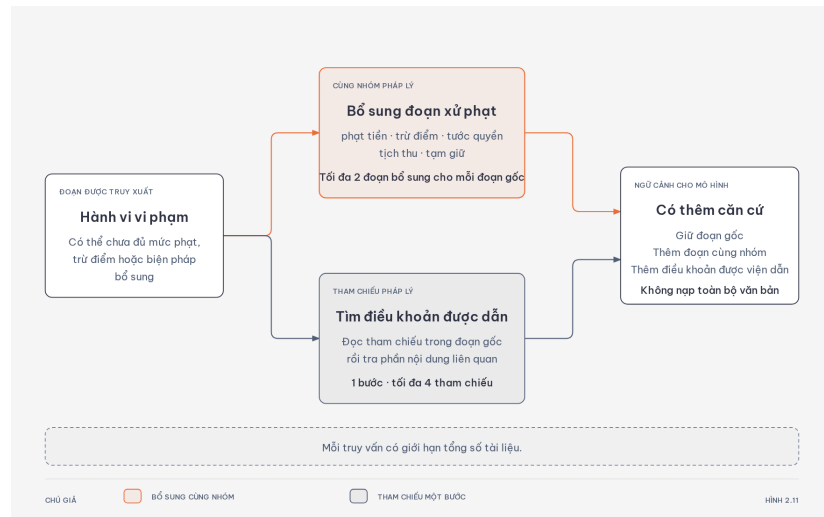


Hình 2.10: Minh họa gộp nhiều danh sách kết quả bằng RRF

2.10 Bổ sung đoạn cùng Điều và tham chiếu liên quan

Một đoạn nêu hành vi vi phạm có thể không chứa phần trừ điểm hoặc biện pháp bổ sung. Vì vậy, retriever có cơ chế tìm thêm một số đoạn cùng Điều chứa các từ khóa xử phạt như “phạt tiền”, “trừ điểm”, “tước quyền”, “tịch thu” hoặc “tạm giữ”. Số lượng được giới hạn để tránh mở rộng vô hạn.

Ngoài sibling completion, dự án có mô-đun cross-reference để bổ sung các đoạn được tham chiếu. Mục tiêu là làm context đủ để model trả lời nhiều ý mà không cần gửi toàn bộ văn bản.

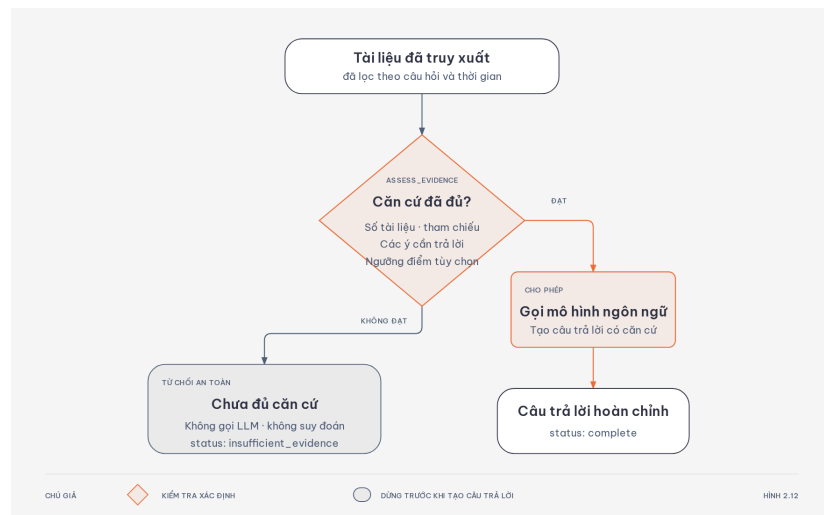


Hình 2.11: Bổ sung căn cứ cùng Điều và tham chiếu liên quan

2.11 Kiểm tra căn cứ và cơ chế từ chối trả lời

Trước generation, hàm `assess_evidence` kiểm tra số lượng tài liệu, tham chiếu bắt buộc và ngưỡng score nếu được cấu hình. Khi không đạt, service trả thông báo “Chưa đủ căn cứ...” và không gọi LLM. Cách làm này thường được gọi là *abstention*: hệ thống chủ động không kết luận khi thiếu bằng chứng.

Khi dữ liệu không đủ, trả về trạng thái từ chối an toàn hơn việc suy đoán một mức phạt hoặc căn cứ pháp lý. Evidence gate chỉ kiểm tra các điều kiện được lập trình sẵn nên không bảo đảm mọi câu trả lời sau đó đều đúng. Vai trò của nó là chặn các trường hợp thiếu tài liệu hoặc thiếu tham chiếu trước khi gọi model.



Hình 2.12: Bước kiểm tra căn cứ trước khi sinh câu trả lời

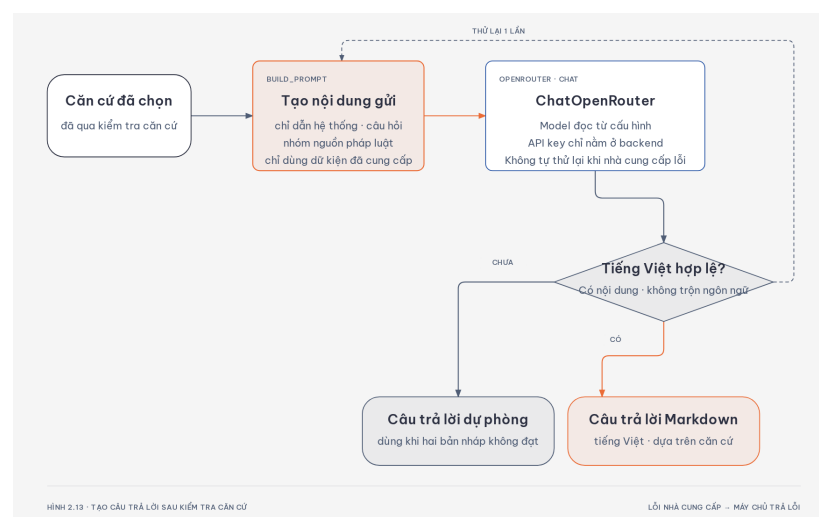
2.12 Sinh câu trả lời có căn cứ qua OpenRouter

Mô-đun generator tạo prompt gồm thông điệp hệ thống, câu hỏi và các nhóm nguồn pháp luật. Prompt yêu cầu trả lời bằng tiếng Việt, không tự tạo số Điều, mức tiền, điểm hoặc nguồn. Model được gọi qua ChatOpenRouter. Tên model nằm trong cấu hình và có thể thay đổi mà không làm thay đổi luồng xử lý chính.

OpenRouter được dùng qua hai loại API. API embedding biến văn bản thành vector. API chat nhận danh sách message và trả nội dung sinh. Backend đọc khóa truy cập từ biến môi trường và không gửi khóa này xuống frontend. Tài liệu OpenRouter mô tả riêng endpoint embedding và endpoint chat/completions [12, 20].

Generation diễn ra sau retrieval và kiểm tra căn cứ. Model chỉ nhận các chunk đã được service chọn. Khi provider không khả dụng hoặc response không hợp lệ, backend trả lỗi theo luồng xử lý đã định nghĩa. Generator còn kiểm tra một số dấu hiệu trộn ngôn ngữ rõ ràng. Nếu phát hiện, model được gọi lại với yêu cầu viết tiếng Việt và giữ nguyên thông tin từ nguồn đã cung cấp.

Retriever tìm dữ liệu, evidence gate kiểm tra điều kiện, còn model diễn đạt câu trả lời. Vì mỗi phần có đầu vào và đầu ra riêng, các phần này có thể được kiểm thử độc lập.

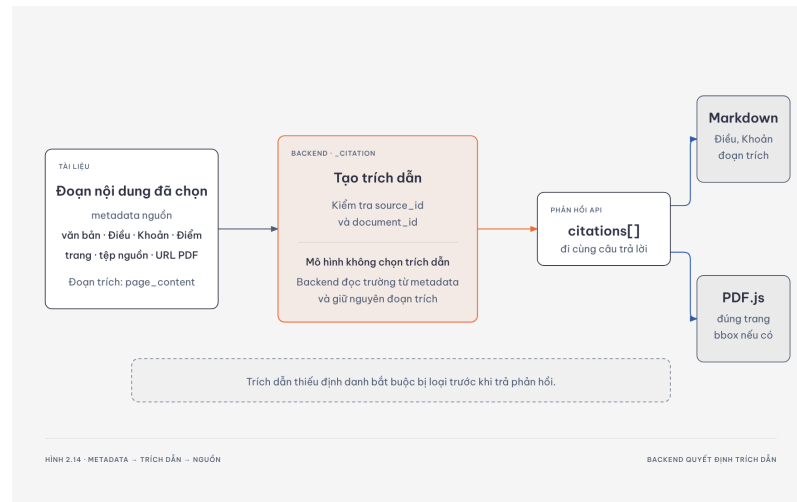


Hình 2.13: Luồng sinh câu trả lời từ các căn cứ đã truy xuất

2.13 Trích dẫn được tạo từ metadata

Service dựng citation trực tiếp từ metadata của từng Document. Một citation gồm tên hoặc số văn bản, Điều, Khoản, Điểm, trang, source file, đoạn trích và PDF URL nếu có. Model viết phần diễn giải. Backend quyết định citation nào được đưa vào response và lấy các trường đó từ dữ liệu đã lưu.

Giao diện có thể dùng metadata citation để mở đúng tài liệu nguồn. Với Markdown, người dùng xem đoạn pháp luật có cấu trúc, với PDF, viewer dùng PDF.js để hiển thị trang và có thể đánh dấu tọa độ khi metadata tương ứng tồn tại.

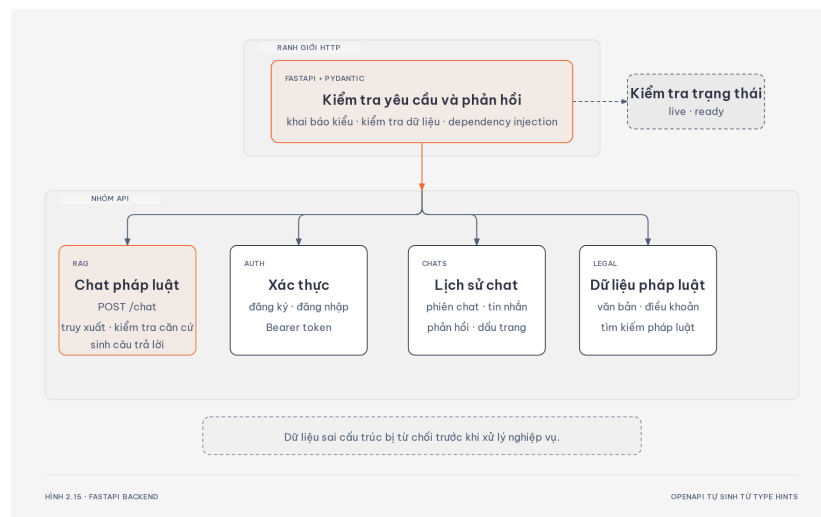


Hình 2.14: Tạo và hiển thị trích dẫn từ metadata nguồn

2.14 FastAPI và Pydantic

FastAPI là web framework Python dựa trên OpenAPI và sử dụng type hints để khai báo request/response. Tài liệu chính thức nêu các khả năng như tự tạo OpenAPI, validation dữ liệu, dependency injection và tài liệu API tự động [9]. Trong đề tài, FastAPI được dùng để cung cấp API chat, auth, chat history, feedback, bookmark, legal documents, legal provisions, search và health check.

Pydantic kiểm tra cấu trúc dữ liệu vào và ra. Request sai schema bị từ chối trước khi service xử lý nghiệp vụ. Các trường như `top_k`, `effective_date`, UUID phiên chat và nội dung message được kiểm tra theo kiểu dữ liệu đã khai báo.



Hình 2.15: Các nhóm API chính của backend FastAPI

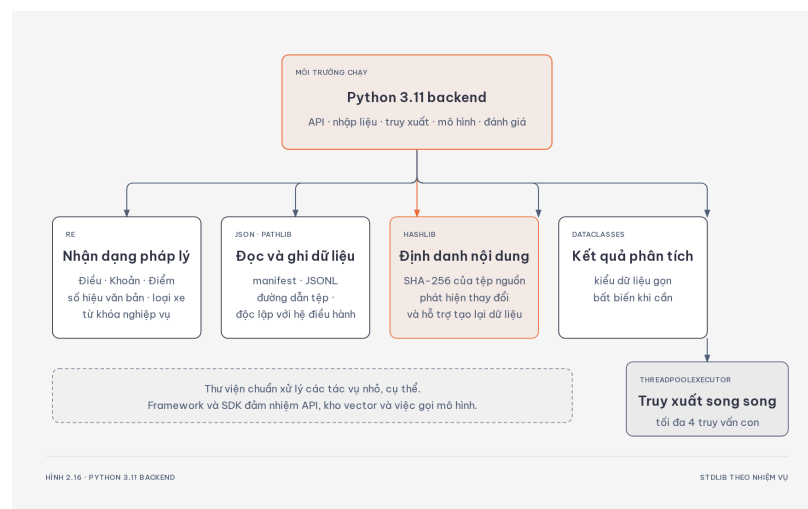
2.15 Python 3.11 và các thành phần backend cơ bản

Python là ngôn ngữ chính của backend. Tài liệu chính thức mô tả Python là ngôn ngữ bậc cao, có cấu trúc dữ liệu sẵn có, hỗ trợ lập trình hướng đối tượng và phù hợp cho phát triển nhanh [13]. Trong đề tài, Python 3.11 được dùng cho API, đọc dữ liệu, xử lý biểu thức chính quy, retrieval, gọi model và đánh giá.

Backend sử dụng một số thành phần của thư viện chuẩn Python cho các nhiệm vụ sau:

- `re` xử lý biểu thức chính quy để nhận dạng Điều, Khoản, Điểm, số hiệu văn bản, loại phương tiện và từ khóa nghiệp vụ,
- `json` đọc manifest và ghi dữ liệu JSON/JSONL,
- `hashlib` tạo SHA-256 cho nội dung nguồn,
- `pathlib.Path` quản lý đường dẫn tệp theo cách độc lập hơn với hệ điều hành,
- `dataclasses` biểu diễn các kết quả phân tích nhỏ, bất biến khi phù hợp,
- `concurrent.futures.ThreadPoolExecutor` cho phép thực hiện một số truy vấn con song song khi câu hỏi có nhiều ý.

Thread pool được dùng để chạy một số lời gọi retrieval độc lập song song. Service giới hạn số worker theo số truy vấn con và mức tối đa được cấu hình.



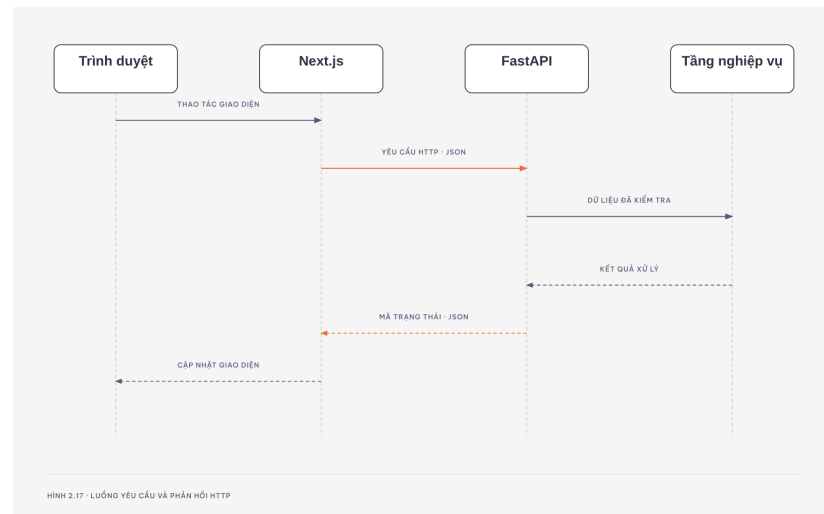
Hình 2.16: Các nhóm chức năng Python được dùng trong backend

2.16 HTTP, REST và dữ liệu JSON

Frontend và backend trao đổi qua HTTP. Một request thường gồm phương thức như GET hoặc POST, đường dẫn, header và body. Backend trả status code cùng response body. Với API JSON, dữ liệu được biểu diễn bằng object, array, chuỗi, số, boolean hoặc null. Các schema Pydantic giúp định nghĩa cấu trúc request/response để hai phía thống nhất cách trao đổi.

API được tổ chức theo tài nguyên. Phiên chat, message, feedback, bookmark và văn bản pháp luật có endpoint riêng. GET dùng cho thao tác đọc. POST dùng cho tạo mới hoặc gọi nghiệp vụ. PATCH cập nhật một phần tài nguyên và DELETE xử lý xóa theo contract của route. Endpoint chat điều phối analyzer, retrieval, evidence và generation nên không ánh xạ trực tiếp vào một bảng dữ liệu.

FastAPI sinh tài liệu OpenAPI tự động từ route và model dữ liệu [9]. OpenAPI mô tả endpoint, tham số, request body và schema dữ liệu, nhờ đó client có thể kiểm tra hợp đồng API theo một định dạng thống nhất.



Hình 2.17: Vòng đời một yêu cầu HTTP giữa trình duyệt và FastAPI

2.17 ASGI và Uvicorn

FastAPI là ứng dụng ASGI. Có thể hiểu ASGI là giao diện chuẩn giữa ứng dụng Python bất đồng bộ và web server. Uvicorn nhận kết nối mạng, chuyển request cho ứng dụng FastAPI và gửi response trở lại client. Trong môi trường phát triển của đề tài, backend được chạy bằng lệnh `uvicorn app.main:app --reload`.

Việc tách web server và application giúp code nghiệp vụ không phải tự xử lý socket HTTP ở mức thấp. Backend tập trung vào route, validation, service và integration, còn Uvicorn chịu trách nhiệm vòng đời kết nối. Đây cũng là lý do health endpoint được tách thành liveness và readiness: một tiến trình có thể còn chạy nhưng dependency như Supabase hoặc Qdrant chưa sẵn sàng.

2.18 LangChain trong phạm vi dự án

LangChain cung cấp các adapter được backend sử dụng trực tiếp. `langchain_core.documents.Document` chứa `page_content` và `metadata`. `langchain-qdrant` kết nối retriever với Qdrant. `langchain-openrouter` cung cấp lớp gọi model generation. Luồng xử lý câu hỏi được viết trong các mô-đun `service`, `analyzer`, `retriever`, `evidence` và `generator` của backend.

Cách dùng này giúp backend tận dụng adapter của thư viện nhưng vẫn giữ logic pháp luật trong các mô-đun riêng của đề tài. Các quy tắc nhận diện tham chiếu, tách câu hỏi, tạo citation và abstention đều có thể kiểm thử trực tiếp.

2.19 Qdrant, FastEmbed và chỉ mục tìm kiếm

Qdrant là cơ sở dữ liệu vector phục vụ retrieval. Dự án chạy Qdrant local với đường dẫn cấu hình bởi `QDRANT_PATH`, collection mặc định là `traffic_law`. Mỗi point chứa dense vector, sparse vector và payload metadata. Tài liệu Qdrant mô tả hybrid search là cách kết hợp tìm kiếm ngữ nghĩa bằng dense vector với tìm kiếm từ khóa bằng sparse vector, sau đó gộp các danh sách kết quả [6].

Phần tìm kiếm sparse dùng lớp `FastEmbedSparse` với mô hình `Qdrant/bm25`. `FastEmbed` nhận văn bản và tạo biểu diễn sparse để Qdrant lưu và tìm kiếm. Backend cũng cài `ONNX Runtime` để chạy mô hình được đóng gói theo định dạng `ONNX` khi thư viện yêu cầu. Trong phạm vi khóa luận, phần này được xem như tầng thực thi mô hình của `FastEmbed`, không phải một thuật toán riêng của hệ thống.

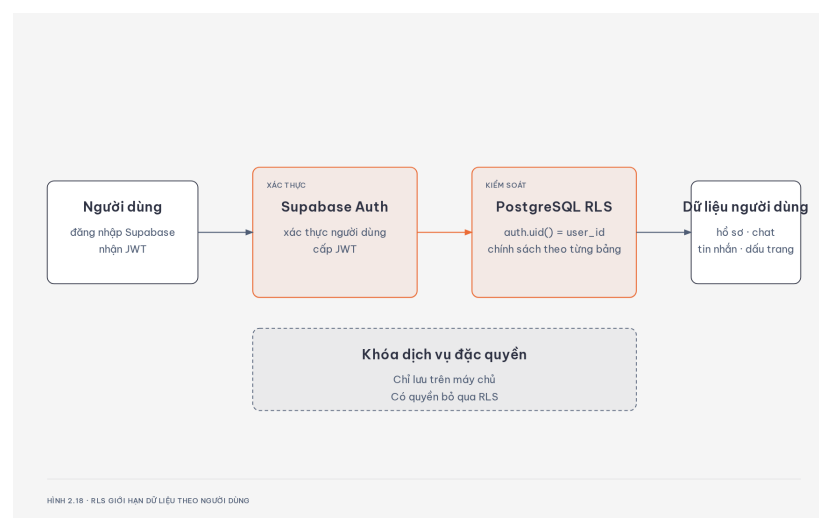
Payload lưu metadata đi cùng vector. Retriever dùng `article`, `clause` và `point` để tạo bộ lọc khi câu hỏi có tham chiếu pháp lý. Vector search và đối chiếu cấu trúc được thực hiện trong cùng luồng retrieval.

Qdrant giữ dữ liệu dẫn xuất phục vụ tìm kiếm. Bản gốc của corpus nằm trong manifest và các tệp nguồn. Tài khoản cùng lịch sử chat nằm trong Supabase. Khi corpus hoặc cấu hình embedding thay đổi, collection được tạo lại từ `chunks.jsonl`.

2.20 Supabase, PostgreSQL và Row Level Security

Supabase cung cấp Auth và cơ sở dữ liệu PostgreSQL cho dữ liệu ứng dụng. Schema của đề tài gồm các bảng `profiles`, `chat_sessions`, `messages`, `feedback` và `bookmarks`. Các bảng dùng `UUID`, khóa ngoại đến `auth.users`, `CHECK` constraint, `UNIQUE` constraint, index và trigger cập nhật thời gian.

Row Level Security (RLS) là cơ chế PostgreSQL cho phép giới hạn hàng dữ liệu mà một người dùng được đọc hoặc sửa. Supabase khuyến nghị dùng RLS trên các bảng được truy cập qua Data API và kết hợp với Auth để kiểm soát dữ liệu theo `auth.uid()` [10]. Trong schema của dự án, mỗi bảng có policy chỉ cho phép người dùng thao tác với dữ liệu thuộc chính tài khoản của mình. Service-role key được giữ ở backend và không đưa ra trình duyệt.



Hình 2.18: Bảo vệ dữ liệu ứng dụng bằng Supabase Auth và RLS

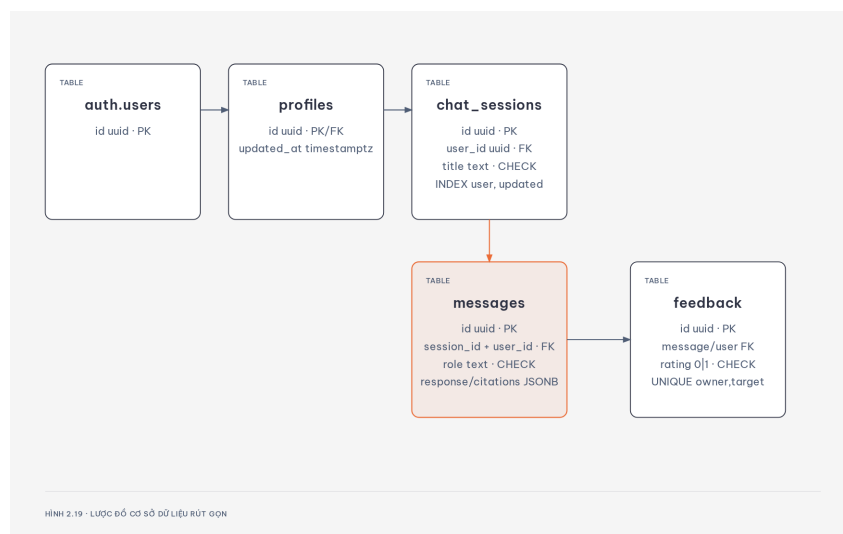
2.21 Cơ sở dữ liệu quan hệ và PostgreSQL

Cơ sở dữ liệu quan hệ tổ chức dữ liệu theo bảng. Mỗi hàng là một bản ghi và mỗi cột là một thuộc tính. Khóa chính định danh bản ghi. Khóa ngoại liên kết bản ghi với bảng khác. Schema của đề tài dùng UUID cho dữ liệu người dùng, phiên chat, tin nhắn, phản hồi và bookmark.

Tính toàn vẹn được bảo vệ bằng nhiều loại ràng buộc. NOT NULL buộc trường phải có giá trị, UNIQUE ngăn trùng lặp theo điều kiện thiết kế, CHECK giới hạn miền giá trị, FOREIGN KEY giữ quan hệ tham chiếu hợp lệ, DEFAULT cung cấp giá trị mặc định. Ví dụ, `feedback.rating` chỉ nhận 0 hoặc 1, còn `messages.role` chỉ nhận `user` hoặc `assistant`.

Chỉ mục (index) là cấu trúc phụ giúp một số truy vấn tìm dữ liệu nhanh hơn, đôi lại cần thêm không gian lưu trữ và chi phí cập nhật. Schema tạo index theo các cặp thường dùng như người dùng và thời gian cập nhật hoặc session và thời gian tạo. Trigger là hàm chạy tự động khi có sự kiện trên bảng. Đề tài dùng trigger để cập nhật `updated_at`, chạm thời gian phiên chat khi có message mới và tạo profile sau khi Auth tạo user.

PostgreSQL hỗ trợ kiểu JSONB. Đề tài dùng JSONB cho response, citations và metadata vì cấu trúc các trường này thay đổi linh hoạt hơn dữ liệu quan hệ cố định. Quan hệ sở hữu user, session và message vẫn được biểu diễn bằng khóa ngoại.



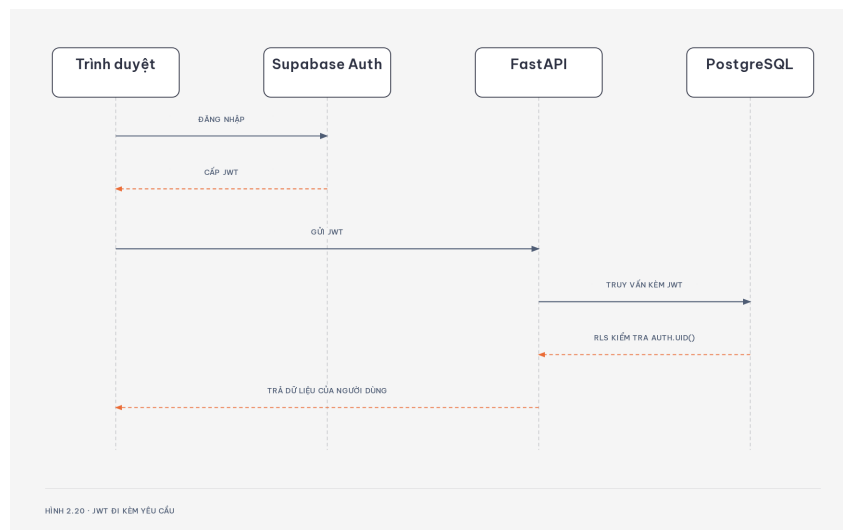
Hình 2.19: Khóa chính, khóa ngoại, chỉ mục và ràng buộc trong dữ liệu ứng dụng

2.22 Supabase Auth, JWT và phân quyền

Supabase Auth quản lý danh tính người dùng và phát hành token xác thực. Theo tài liệu Supabase, authentication xác định danh tính người dùng, còn authorization xác định quyền truy cập tài nguyên. Supabase Auth dùng JWT và kết hợp với RLS trong PostgreSQL [14].

Trong đề tài, Supabase quản lý bảng `auth.users`. Bảng `profiles` dùng cùng UUID để lưu thêm thông tin hiển thị. Các bảng nghiệp vụ giữ `user_id` để xác định chủ sở hữu. Khi request có phiên đăng nhập hợp lệ, policy RLS so sánh `auth.uid()` với `user_id`. Quyền truy cập được kiểm tra ở cơ sở dữ liệu và không phụ thuộc vào việc giao diện có hiển thị nút thao tác hay không.

Service-role key có quyền cao và có thể bỏ qua RLS, vì vậy tài liệu Supabase yêu cầu giữ loại khóa này ở môi trường server [10]. Frontend chỉ dùng loại khóa công khai phù hợp với browser và dựa vào JWT của người dùng để truy cập dữ liệu theo policy.

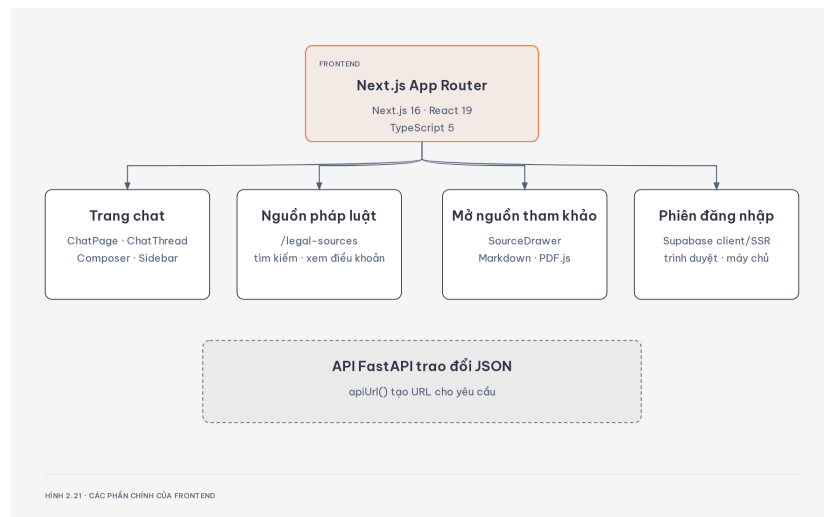


Hình 2.20: Luồng Supabase Auth, JWT, auth.uid và Row Level Security

2.23 Next.js, React và TypeScript

Next.js là framework React dùng để xây dựng ứng dụng web. App Router sử dụng cấu trúc thư mục để định nghĩa route và hỗ trợ Server/Client Components [11]. Frontend của dự án dùng Next.js 16, React 19 và TypeScript 5. Các route chính hiện có gồm trang chat và trang nguồn pháp luật.

TypeScript bổ sung kiểm tra kiểu tĩnh cho JavaScript, giúp giảm một số lỗi khi truyền dữ liệu giữa component và API. PDF.js được dùng để hiển thị PDF trong trình duyệt. React Markdown hỗ trợ trình bày câu trả lời dạng Markdown. Supabase client/SSR hỗ trợ phiên đăng nhập và giao tiếp với Supabase phù hợp theo môi trường server hoặc browser.

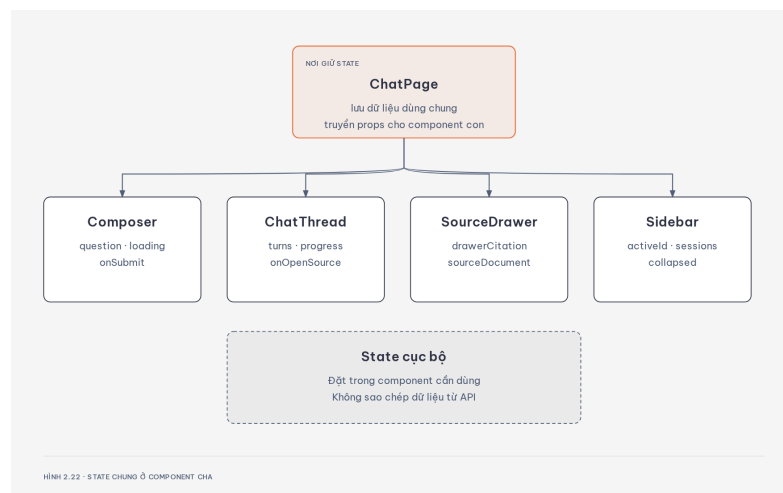


Hình 2.21: Kiến trúc giao diện Next.js của hệ thống

2.23.1 React component, props và state

React xây giao diện từ các component. Tài liệu React mô tả component là một phần giao diện có logic và cách hiển thị riêng, có thể lồng vào nhau để tạo thành trang [15]. Props truyền dữ liệu từ component cha xuống component con. State lưu dữ liệu thay đổi theo thao tác người dùng. Trang chat dùng state cho nội dung ô nhập, trạng thái gửi, danh sách tin nhắn và citation đang được chọn.

Quản lý state cần tránh dữ liệu lặp. Nếu cùng một thông tin vừa nằm trong response API vừa được sao chép vào nhiều state độc lập, khả năng lệch dữ liệu tăng. React khuyến nghị tổ chức state có chủ đích và đưa state chung lên vị trí phù hợp khi nhiều component cần dùng [16].



Hình 2.22: Component và luồng state của giao diện hỏi đáp

2.23.2 TypeScript và kiểm tra kiểu tĩnh

TypeScript bổ sung kiểm tra kiểu tĩnh cho JavaScript trước khi chạy chương trình [17]. Frontend khai báo kiểu cho message, citation, session và response API. Trình biên dịch có thể phát hiện các lỗi như sai tên field hoặc truyền dữ liệu không đúng kiểu trước khi chạy trên trình duyệt.

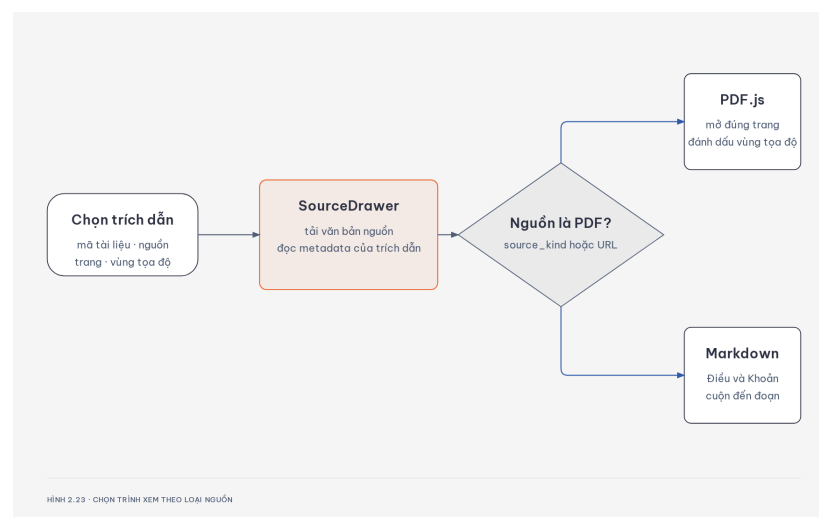
TypeScript kiểm tra kiểu trong mã frontend. Dữ liệu đi qua mạng vẫn được backend kiểm tra bằng Pydantic vì request thực tế có thể không tuân theo type đã khai báo ở phía client.

2.23.3 PDF.js và React Markdown

PDF.js là dự án của Mozilla dùng JavaScript để phân tích và hiển thị PDF. Tài liệu PDF.js mô tả core layer, display layer và viewer layer. Display layer cung cấp API để render trang và đọc thông tin tài liệu [18]. Đề tài dùng gói pdfjs-dist cho phần xem nguồn PDF trong ứng dụng.

Citation có thể chứa trang và thông tin nguồn. Khi người dùng chọn một citation, giao diện điều hướng đến viewer phù hợp. Với nguồn Markdown, ứng dụng hiển thị nội dung pháp luật có cấu trúc. Với PDF, viewer có thể mở đúng trang và, nếu metadata có tọa độ, dùng tọa độ đó để làm nổi bật vùng liên quan. Đây là chức năng giao diện, nội dung citation vẫn do backend dựng từ metadata.

React Markdown được dùng để render câu trả lời Markdown thành các phần tử giao diện có cấu trúc như tiêu đề, đoạn, danh sách và nhấn mạnh. Cần phân biệt rõ Markdown của câu trả lời với Markdown corpus: một bên là định dạng trình bày response, bên còn lại là nguồn dữ liệu pháp luật cục bộ dùng cho ingestion.



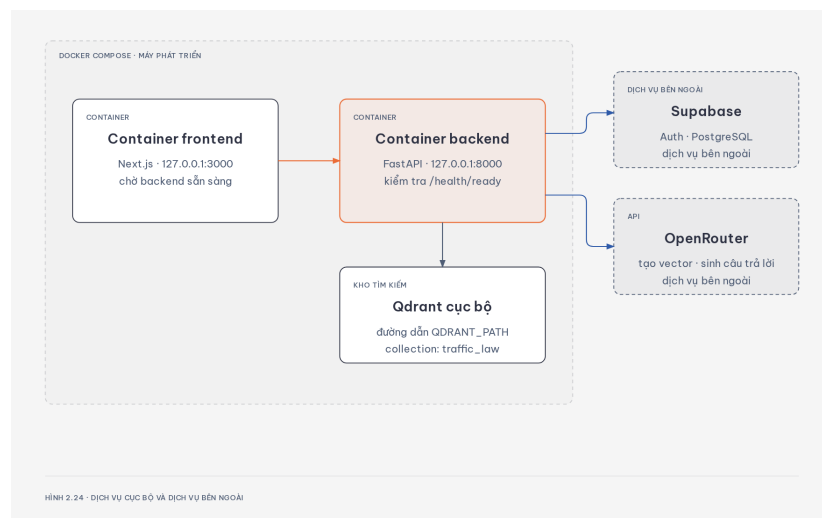
Hình 2.23: Luồng mở citation đến nguồn Markdown hoặc PDF viewer

2.24 Docker Compose và cấu hình môi trường

Docker đóng gói ứng dụng và dependency thành môi trường chạy có cấu hình rõ ràng. Docker Compose dùng một cấu hình để định nghĩa và chạy ứng dụng gồm nhiều container [19]. Đề tài hỗ trợ lệnh `docker compose --env-file .env up --build` cho vòng phát triển đầy đủ.

Biến môi trường lưu URL dịch vụ, khóa API, tên model và đường dẫn Qdrant. Khóa bí mật được tách khỏi logic nghiệp vụ để có thể thay đổi cấu hình triển khai mà không sửa chương trình.

Docker Compose tổ chức các thành phần chạy cục bộ theo tệp cấu hình của đề tài. Supabase và OpenRouter được truy cập như các dịch vụ bên ngoài qua mạng.

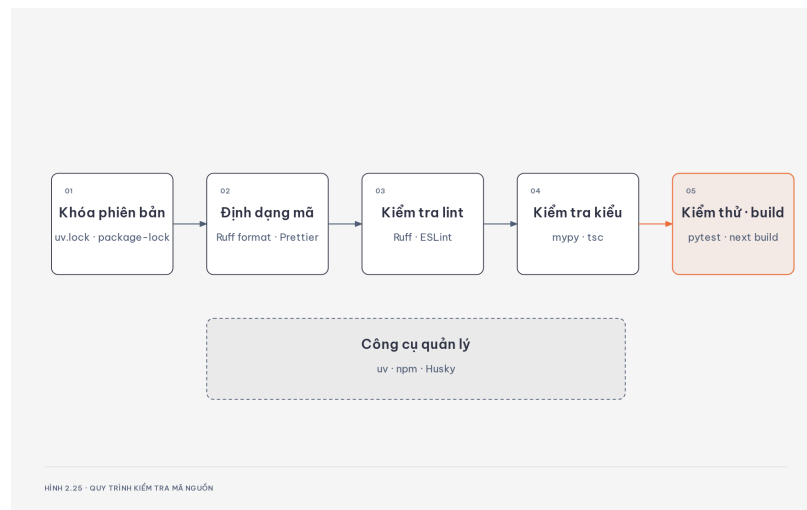


Hình 2.24: Môi trường phát triển với Docker Compose và các dịch vụ liên quan

2.25 Quản lý thư viện và công cụ chất lượng mã nguồn

Backend dùng uv để đồng bộ môi trường Python. Frontend dùng npm để quản lý thư viện JavaScript. Lock file cố định phiên bản dependency đã được chọn, giúp các môi trường cài đặt sử dụng cùng tập phiên bản thư viện.

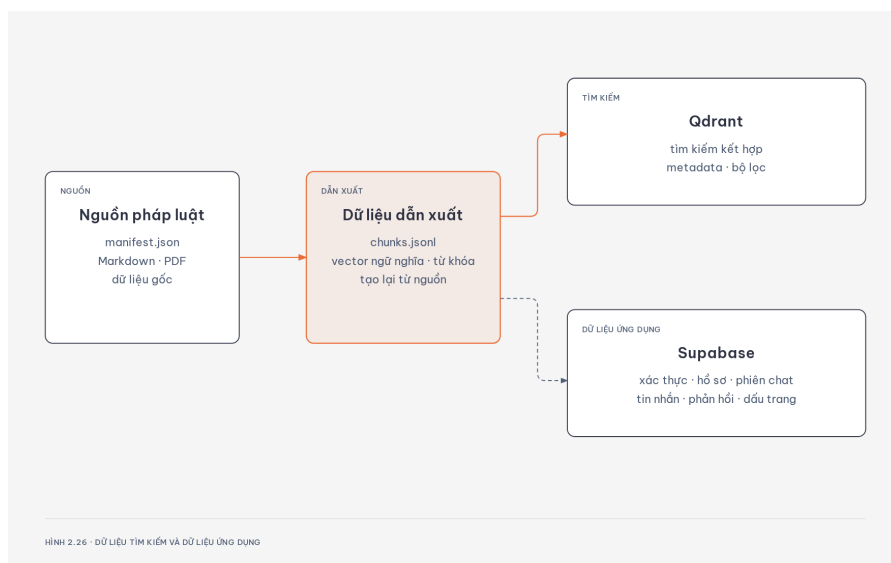
Ruff kiểm tra quy tắc lint cho Python. Mypy kiểm tra type hint và pytest chạy test backend. Ở frontend, TypeScript compiler kiểm tra kiểu, ESLint kiểm tra quy tắc lập trình, Prettier định dạng mã nguồn và Husky chạy các kiểm tra tự động trước một số thao tác Git. Nhóm công cụ này hỗ trợ phát hiện lỗi sớm và giữ cách tổ chức mã nguồn nhất quán.



Hình 2.25: Chuỗi công cụ cài đặt, kiểm tra kiểu, lint và test của dự án

2.26 Tổ chức các lớp dữ liệu

Dữ liệu của hệ thống được chia theo nơi sử dụng. Manifest, Markdown và PDF là nguồn pháp luật. Chunks JSONL cùng Qdrant index phục vụ retrieval và có thể tạo lại. Supabase lưu tài khoản, phiên chat, message, feedback và bookmark. Việc xây lại Qdrant không làm thay đổi dữ liệu người dùng trong Supabase.



Hình 2.26: Kiến trúc dữ liệu: nguồn pháp luật, chỉ mục Qdrant và dữ liệu ứng dụng Supabase

2.27 Kiểm thử phần mềm

Backend dùng pytest cho unit test và integration test. Playwright được dùng khi cần kiểm tra luồng trình duyệt. Ruff và mypy hỗ trợ kiểm tra chất lượng Python, còn ESLint, TypeScript và Prettier hỗ trợ frontend. Kiểm thử được tách giữa logic thuần như parser, reference matching, metrics và các phần tích hợp với Qdrant, Supabase hoặc nhà cung cấp API.

Các phần xử lý theo quy tắc được kiểm thử bằng input và output cụ thể. Parser cần nhận đúng Điểm “đ”. Analyzer cần phân biệt xe máy với ô tô. Citation phải lấy đúng Điều, Khoản và Điểm từ metadata. Evidence gate phải từ chối khi danh sách tài liệu rỗng.

2.28 Đánh giá hệ thống RAG

2.28.1 Retrieval hit tại k

Retrieval hit tại k kiểm tra xem trong k kết quả được lấy có ít nhất một tọa độ pháp lý đúng hay không. Đây là chỉ số nhị phân theo từng case và được lấy trung bình trên bộ đánh giá.

2.28.2 Độ chính xác theo tọa độ pháp lý

Mô-đun đánh giá so sánh lần lượt văn bản, Điều, Khoản và Điểm. Cách tách theo cấp giúp phát hiện trường hợp retrieval tìm đúng văn bản nhưng sai Khoản, hoặc đúng Điều nhưng sai Điểm. Cách đánh giá này phù hợp với dữ liệu pháp luật, nơi vị trí cấu trúc có ý nghĩa đối với căn cứ được trả về.

2.28.3 Citation validity

Citation validity kiểm tra các citation trả về có được đánh dấu hợp lệ hay không. Chỉ số này không thay thế đánh giá nội dung câu trả lời, nhưng giúp phát hiện citation hỏng hoặc không khớp với nguồn đã truy xuất.

2.28.4 Abstention accuracy

Bộ dữ liệu có các trường hợp kỳ vọng hệ thống từ chối trả lời. Abstention accuracy so sánh trạng thái thực tế với kỳ vọng. Một hệ thống luôn trả lời có thể có trải nghiệm “mạnh” nhưng không phù hợp nếu dữ liệu thiếu.

2.28.5 Độ trễ

Mô-đun đánh giá tính mean, p50 và p95 latency. P50 cho biết trung vị, còn p95 phản ánh nhóm truy vấn có thời gian xử lý cao hơn. Trong hệ thống RAG, độ trễ đến từ retrieval, quá trình gọi mô hình và các truy vấn con khi câu hỏi có nhiều intent.

2.28.6 Đánh giá nội dung thủ công

Answer correctness được xác định qua bước review thủ công. Mô-đun đánh giá chỉ tính các metric có thể xác định trực tiếp từ prediction và expected data. Bộ 40 case cho phép chạy lại cùng một quy trình để so sánh các lần thử nghiệm.



Hình 2.27: Các nhóm chỉ số trong bộ đánh giá 40 trường hợp

2.29 Tài liệu kỹ thuật dùng để xây dựng hệ thống

Các khái niệm về lập trình, phân tích hệ thống, công nghệ phần mềm và phát triển Web được tham khảo từ các bài giảng của Khoa Kỹ thuật - Công nghệ, Trường Đại học Tây Đô [1, 2, 3, 4, 5]. Những nội dung này hỗ trợ phần mô hình hóa hệ thống, thiết kế phần mềm và phát triển giao diện Web.

Đối với công nghệ đang sử dụng, khóa luận tham khảo tài liệu chính thức của từng nền tảng. Tài liệu Qdrant được dùng để giải thích dense vector, sparse vector, BM25, hybrid search và RRF [8, 7, 6]. Tài liệu FastAPI được dùng cho phần HTTP API, OpenAPI và validation [9]. Tài liệu Supabase được dùng để giải thích xác thực người dùng và Row Level Security [14, 10]. Tài liệu Next.js, React và TypeScript được dùng cho phần giao diện Web [11, 15, 17].

Các thành phần triển khai gồm API, cơ sở dữ liệu ứng dụng, xử lý dữ liệu pháp luật, tìm kiếm, sinh câu trả lời và đánh giá. Mỗi thành phần được mô tả theo chức năng và dữ liệu mà hệ thống sử dụng.

2.30 Tổng hợp công nghệ sử dụng

Bảng dưới đây tổng hợp các công nghệ đang có trong hệ thống và vai trò của từng thành phần.

Bảng 2.1: Các công nghệ chính trong hệ thống

Nhóm	Công nghệ	Vai trò trong đề tài
Ngôn ngữ backend	Python 3.11	Viết API, ingestion, retrieval, generation và evaluation
Quản lý Python	uv	Cài đặt và khóa dependency backend
API	FastAPI, Uvicorn, Pydantic	Xử lý HTTP, validation và chạy server
RAG adapter	LangChain	Document, kết nối Qdrant và lớp gọi mô hình
Vector search	Qdrant	Hybrid dense/sparse retrieval và payload filter

Sparse retrieval	FastEmbed BM25	Tạo sparse representation cho từ khóa
Model provider	OpenRouter	Dense embedding và generation qua API
Application data	Supabase/PostgreSQL	Auth, profile, chat, message, feedback, bookmark
Frontend	Next.js 16, React 19, TypeScript 5	Giao diện Web theo App Router
PDF viewer	pdfjs-dist	Hiển thị PDF trong trang nguồn pháp luật
Kiểm thử	pytest, pytest-asyncio, Playwright	Kiểm thử backend và trình duyệt
Chất lượng mã nguồn	Ruff, mypy, ESLint, Prettier	Kiểm tra quy tắc, kiểu dữ liệu và định dạng
Đóng gói	Docker Compose	Chạy frontend, backend và dịch vụ phát triển theo cùng cấu hình

2.31 Kết luận chương

Chương 2 mô tả cách hệ thống tổ chức chunk pháp luật, kết hợp BM25 với dense embedding, thực hiện hybrid search và RRF, phân tích câu hỏi, kiểm tra căn cứ và tạo citation. Chương cũng trình bày các thành phần triển khai và cách kiểm thử, đánh giá.

CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Mục đích của chương

Chương này trình bày phân tích và thiết kế hệ thống từ yêu cầu chức năng đến kiến trúc, luồng dữ liệu, mô hình tương tác, thiết kế cơ sở dữ liệu và API. Các sơ đồ FDD, DFD, Use Case, Activity, Sequence, Component và Deployment được dùng để mô tả hệ thống ở các mức khác nhau.

3.2 Xác định tác nhân

Bảng 3.1: Các tác nhân chính của hệ thống

Tác nhân	Vai trò
Khách chưa đăng nhập	Xem trang công khai, đăng ký, đăng nhập, phạm vi chi tiết phụ thuộc route giao diện
Người dùng đã đăng nhập	Hỏi đáp, xem citation, xem nguồn pháp luật, quản lý phiên chat, gửi feedback và bookmark
Backend FastAPI	Kiểm tra request, gọi RAG service, quản lý dữ liệu ứng dụng và trả API response
Supabase	Xác thực người dùng và lưu dữ liệu ứng dụng
Qdrant	Lưu chỉ mục dense/sparse và trả kết quả retrieval
OpenRouter	Cung cấp embedding và model sinh câu trả lời qua API
Người vận hành CLI	Chạy lệnh tạo chunks, xây chỉ mục và chạy đánh giá

3.3 Yêu cầu chức năng

Các yêu cầu chức năng được nhóm theo thao tác mà người dùng hoặc tiến trình vận hành thực hiện. Bảng dưới đây dùng làm căn cứ để đối chiếu với API và giao diện ở các phần sau.

Bảng 3.2: Yêu cầu chức năng dự kiến dùng để đối chiếu implementation

Mã	Chức năng	Mô tả
FR01	Đăng ký tài khoản	Người dùng tạo tài khoản thông qua Supabase Auth, profile được tạo tương ứng
FR02	Đăng nhập	Xác thực thông tin và tạo phiên đăng nhập
FR03	Gửi câu hỏi pháp luật	Gửi câu hỏi, top-k và ngày hiệu lực tùy chọn tới API chat
FR04	Nhận câu trả lời có citation	Hiển thị answer, status và danh sách citation do backend trả về
FR05	Từ chối khi thiếu căn cứ	Trả trạng thái insufficient evidence khi evidence gate không đạt
FR06	Quản lý phiên chat	Tạo, xem, đổi tên hoặc xóa mềm session theo API hiện hành
FR07	Lưu tin nhắn	Ghi message người dùng và assistant vào Supabase
FR08	Gửi phản hồi	Ghi rating và comment cho message
FR09	Bookmark	Đánh dấu message cần xem lại

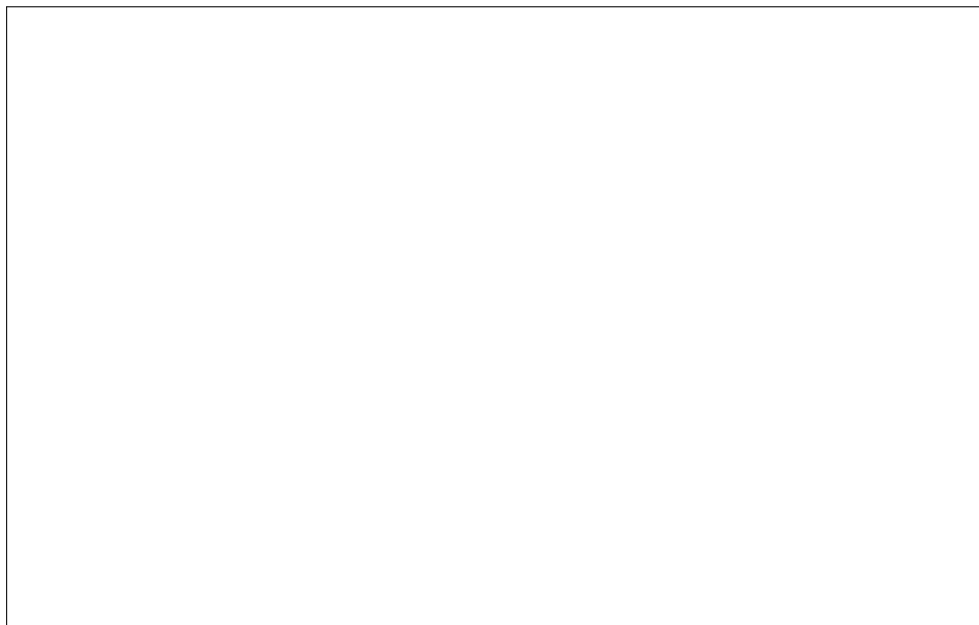
FR10	Xem danh sách văn bản	Đọc legal documents từ corpus API
FR11	Xem các điều khoản	Mở provisions của một document
FR12	Tìm trong nguồn pháp luật	Tìm theo text, số văn bản hoặc Điều theo contract API
FR13	Xem PDF/Markdown	Mở nguồn tương ứng với citation và metadata
FR14	Tạo chunks	CLI đọc manifest và Markdown, ghi chunks JSONL
FR15	Xây chỉ mục	CLI tạo collection Qdrant mới từ chunks
FR16	Chạy đánh giá	Mô-đun đánh giá xử lý 40 case qua API hoặc prediction JSONL

3.4 Yêu cầu phi chức năng

Yêu cầu phi chức năng tập trung vào khả năng truy vết, bảo mật, khả năng chạy lại và bảo trì. Citation phải giữ metadata nguồn. Service-role key chỉ xuất hiện ở backend, còn RLS giới hạn dữ liệu theo người dùng. Các bước ingestion, index và evaluation có lệnh chạy riêng. Logic RAG được chia thành analyzer, references, retrieval, evidence, generator và service. Khi thiếu căn cứ, API trả trạng thái abstention.

3.5 Sơ đồ phân rã chức năng

FDD mô tả hệ thống theo các nhóm chức năng lớn gồm quản lý người dùng, hỏi đáp pháp luật, nguồn pháp luật, dữ liệu chat và vận hành dữ liệu. Sơ đồ tập trung vào quan hệ phân rã chức năng thay vì thứ tự thời gian.



Hình 3.1: Sơ đồ phân rã chức năng của hệ thống

3.6 Sơ đồ luồng dữ liệu

3.6.1 DFD mức ngữ cảnh

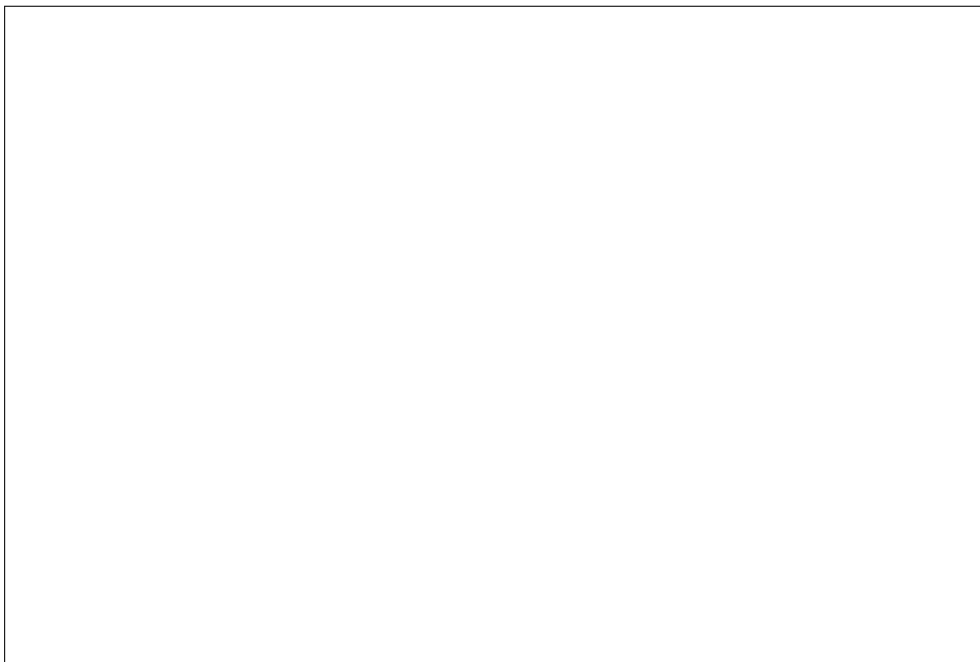
DFD mức ngữ cảnh xem toàn hệ thống như một tiến trình duy nhất và thể hiện trao đổi dữ liệu với người dùng, Supabase, Qdrant, OpenRouter và nguồn dữ liệu cục bộ.



Hình 3.2: Sơ đồ luồng dữ liệu mức ngữ cảnh

3.6.2 DFD mức 0

DFD mức 0 tách hệ thống thành các tiến trình xác thực, quản lý chat, xử lý RAG, tra cứu nguồn pháp luật và ingestion/index. Các kho dữ liệu gồm Supabase, Qdrant, manifest/Markdown và chunks JSONL.



Hình 3.3: Sơ đồ luồng dữ liệu mức 0

3.7 Sơ đồ Use Case

3.7.1 Use Case tổng quát

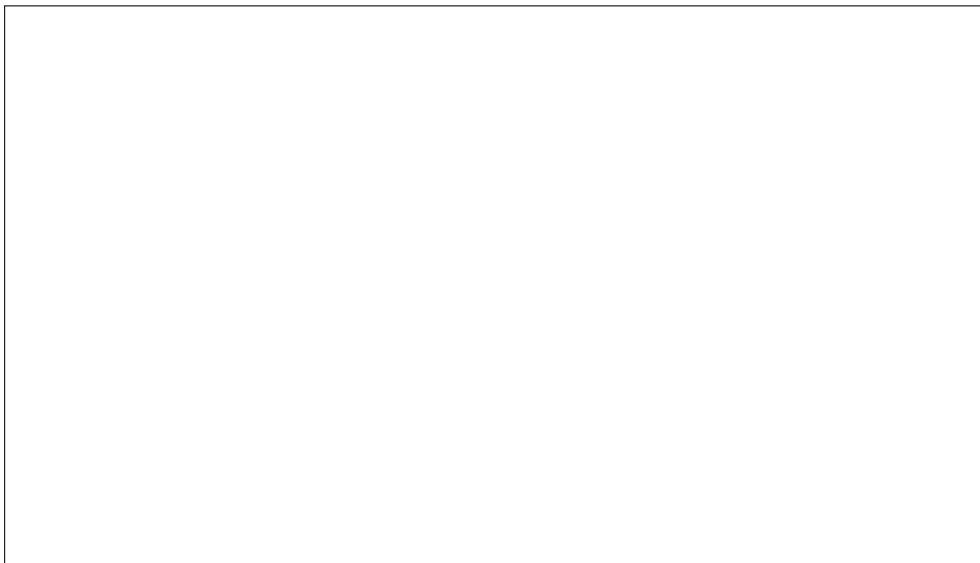
Use Case tổng quát thể hiện các tác nhân chính và phạm vi chức năng tương ứng trong hệ thống.



Hình 3.4: Sơ đồ Use Case tổng quát của hệ thống

3.7.2 Use Case xác thực

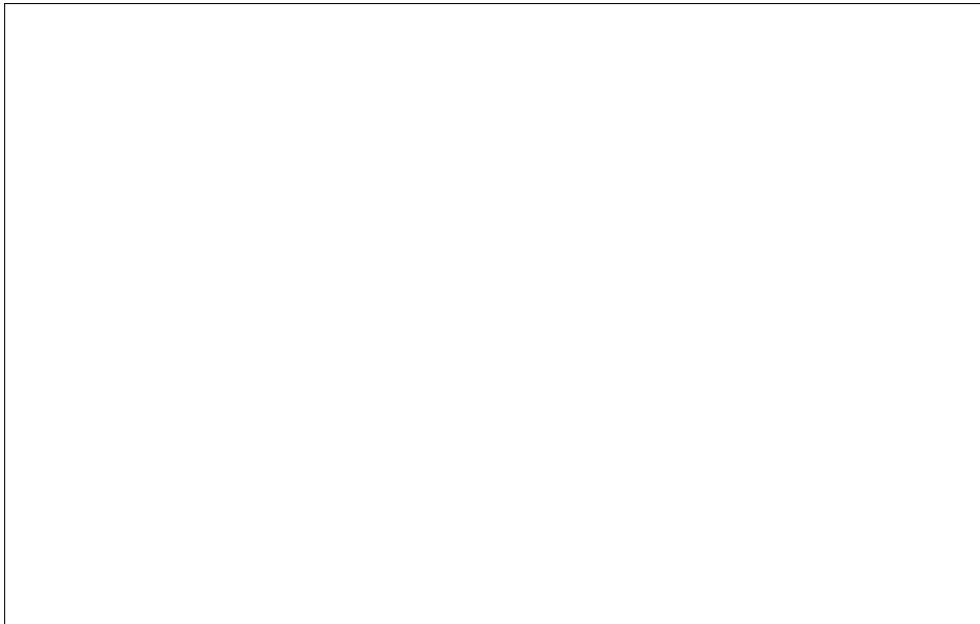
Use Case này gồm đăng ký, đăng nhập, lấy thông tin người dùng hiện tại và đăng xuất ở phía giao diện nếu có. Mọi thao tác dữ liệu cá nhân sau xác thực phải gắn với user ID.



Hình 3.5: Use Case đăng ký và đăng nhập

3.7.3 Use Case hỏi đáp pháp luật

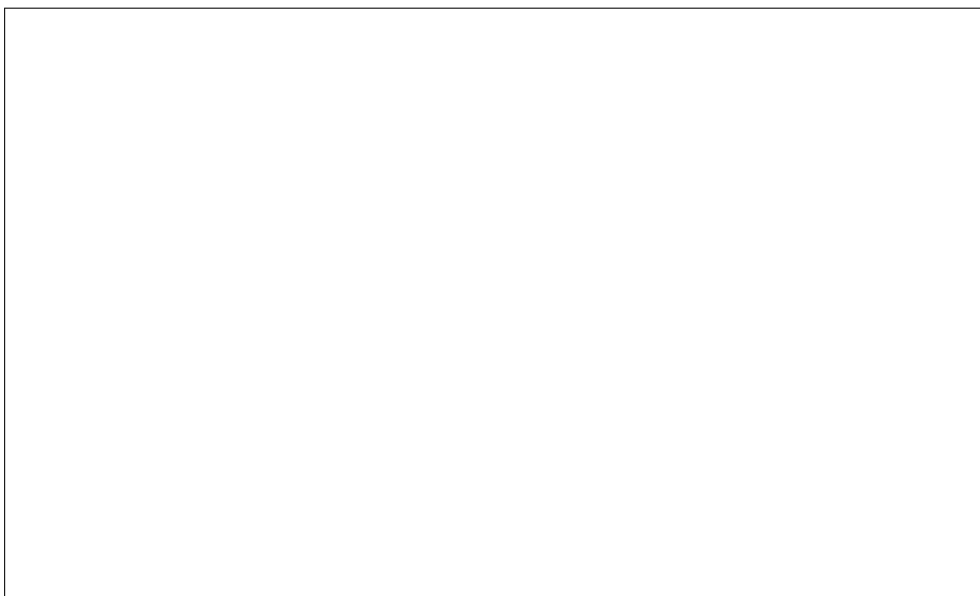
Luồng chính gồm nhập câu hỏi, gửi request, phân tích câu hỏi, retrieval, evidence gate, generation, trả citation, hiển thị và mở nguồn. Nhánh ngoại lệ là không đủ căn cứ hoặc provider lỗi.



Hình 3.6: Use Case hỏi đáp pháp luật

3.7.4 Use Case nguồn pháp luật

Người dùng có thể xem danh sách văn bản, tìm kiếm, mở điều khoản và xem tệp nguồn.



Hình 3.7: Use Case tra cứu văn bản và điều khoản pháp luật

3.7.5 Use Case lịch sử hội thoại

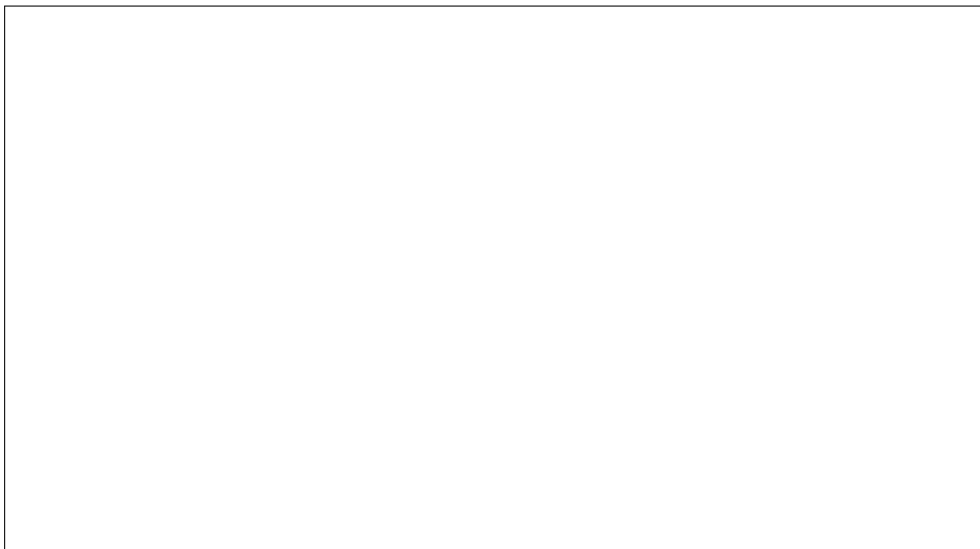
Người dùng tạo phiên chat, xem lại phiên cũ, đổi tiêu đề và xóa phiên theo quyền được cấp.



Hình 3.8: Use Case quản lý lịch sử hội thoại

3.7.6 Use Case feedback và bookmark

Người dùng đánh giá một câu trả lời hoặc lưu câu trả lời để xem lại.



Hình 3.9: Use Case phản hồi và đánh dấu câu trả lời

3.7.7 Use Case ingestion và index bằng CLI

Luồng vận hành bắt đầu từ dữ liệu nguồn, tạo tập chunk rồi xây dựng chỉ mục Qdrant.

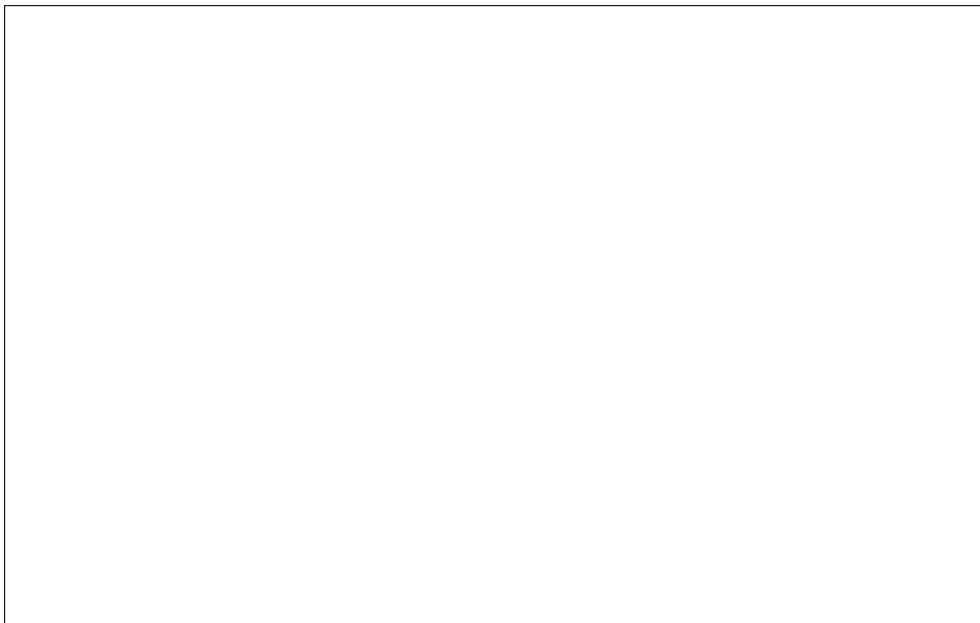


Hình 3.10: Use Case vận hành dữ liệu và xây chỉ mục

3.8 Sơ đồ hoạt động

3.8.1 Hoạt động đăng ký và đăng nhập

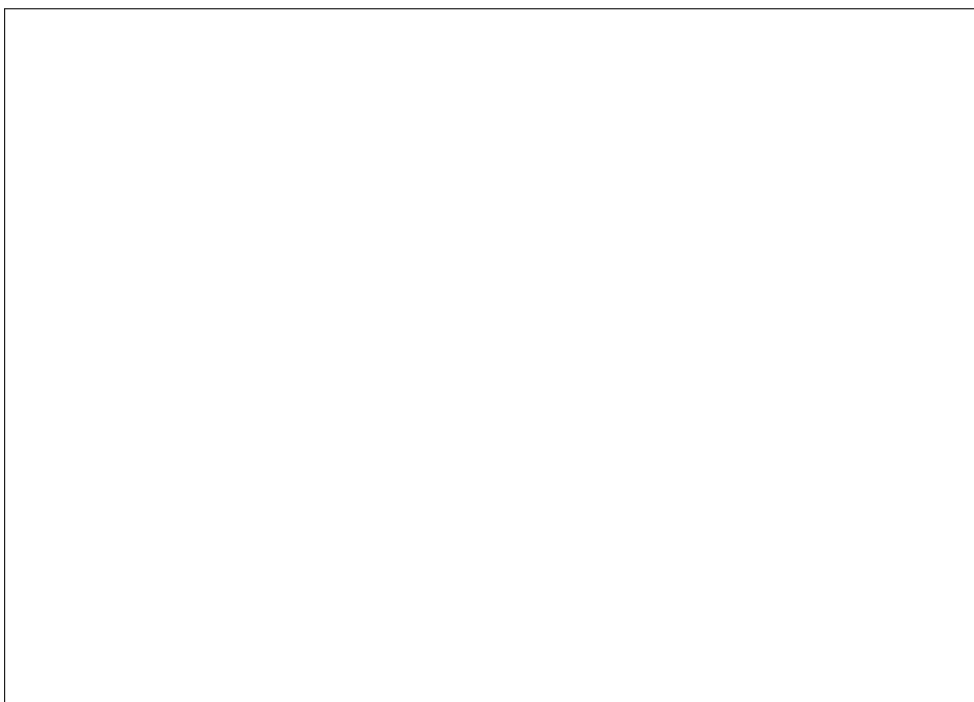
Người dùng nhập dữ liệu, hệ thống kiểm tra thông tin, xác thực với Supabase rồi trả kết quả về giao diện.



Hình 3.11: Sơ đồ hoạt động đăng ký và đăng nhập

3.8.2 Hoạt động hỏi đáp pháp luật

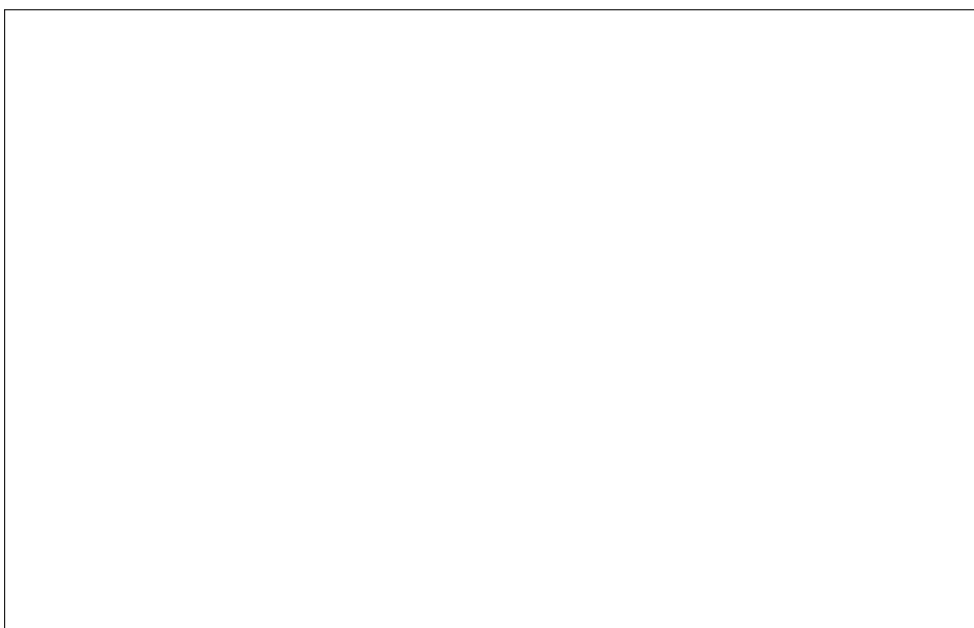
Sơ đồ hoạt động tách hai nhánh theo kết quả kiểm tra căn cứ. Backend chỉ gọi LLM khi evidence gate cho phép tiếp tục.



Hình 3.12: Sơ đồ hoạt động xử lý một câu hỏi pháp luật

3.8.3 Hoạt động xem nguồn pháp luật

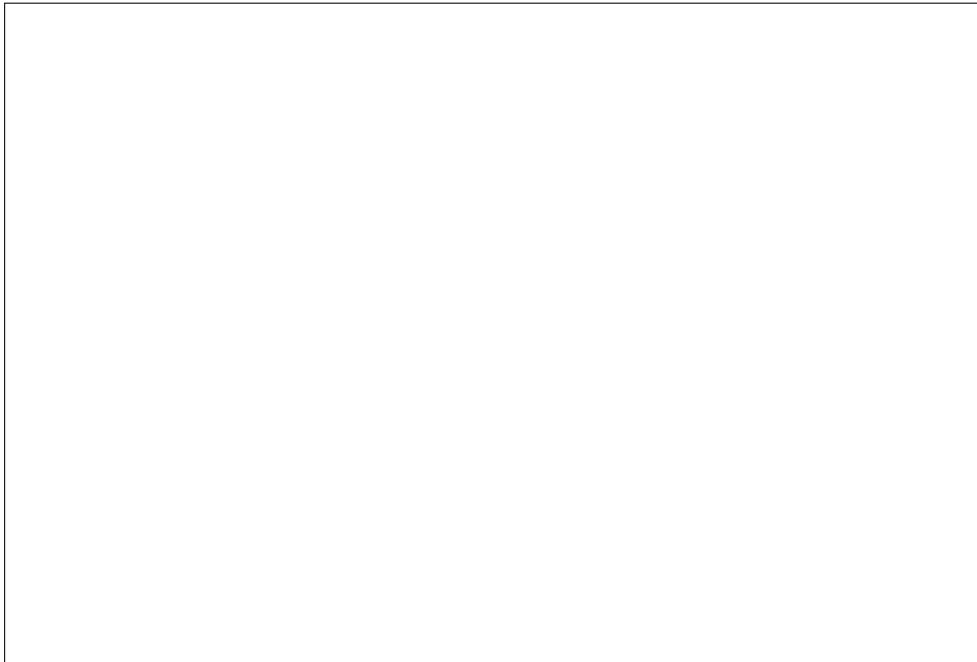
Sau khi tìm văn bản, người dùng mở nội dung Markdown hoặc trang PDF tương ứng.



Hình 3.13: Sơ đồ hoạt động tra cứu nguồn pháp luật

3.8.4 Hoạt động ingestion và tạo index

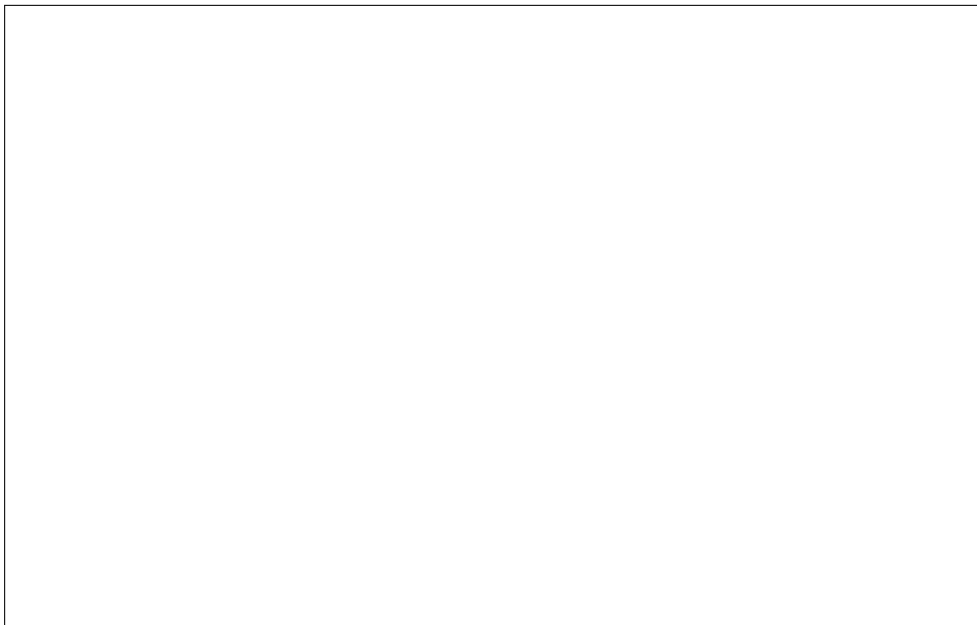
Quy trình đọc manifest, tạo chunk, ghi JSONL rồi xây dựng collection Qdrant.



Hình 3.14: Sơ đồ hoạt động tạo chunks và Qdrant index

3.8.5 Hoạt động đánh giá

Mô-đun đánh giá đọc bộ câu hỏi, gọi hệ thống, ghi dự đoán và tính các chỉ số tự động.



Hình 3.15: Sơ đồ hoạt động chạy bộ đánh giá khóa luận

3.9 Sơ đồ tuần tự

3.9.1 Đăng nhập

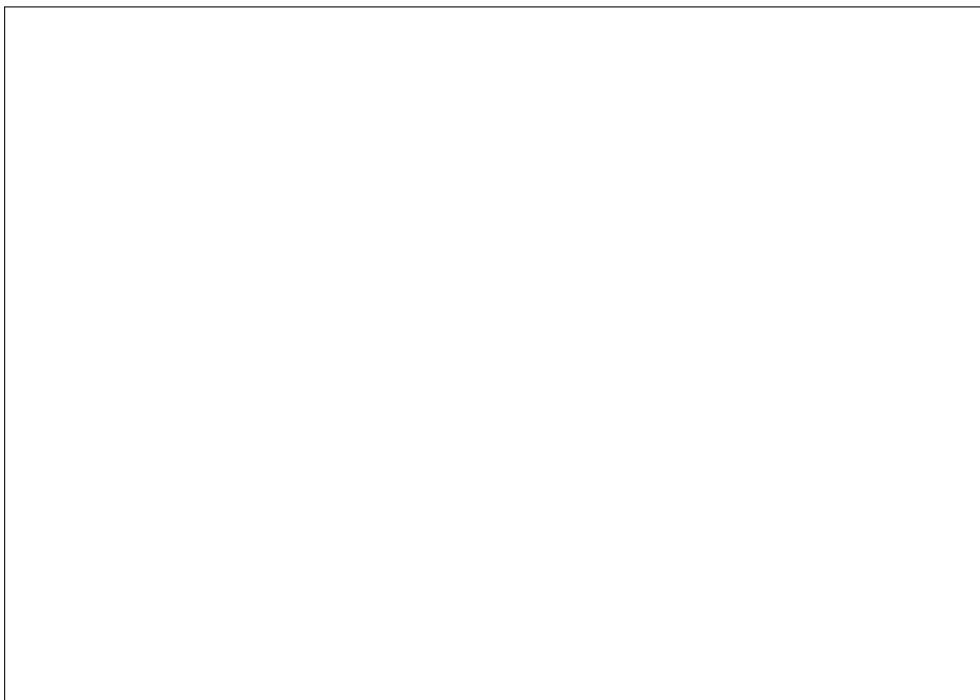
Sơ đồ thể hiện giao tiếp giữa trình duyệt, backend và dịch vụ xác thực Supabase trong quá trình đăng nhập.



Hình 3.16: Sơ đồ tuần tự đăng nhập

3.9.2 Hỏi đáp end-to-end

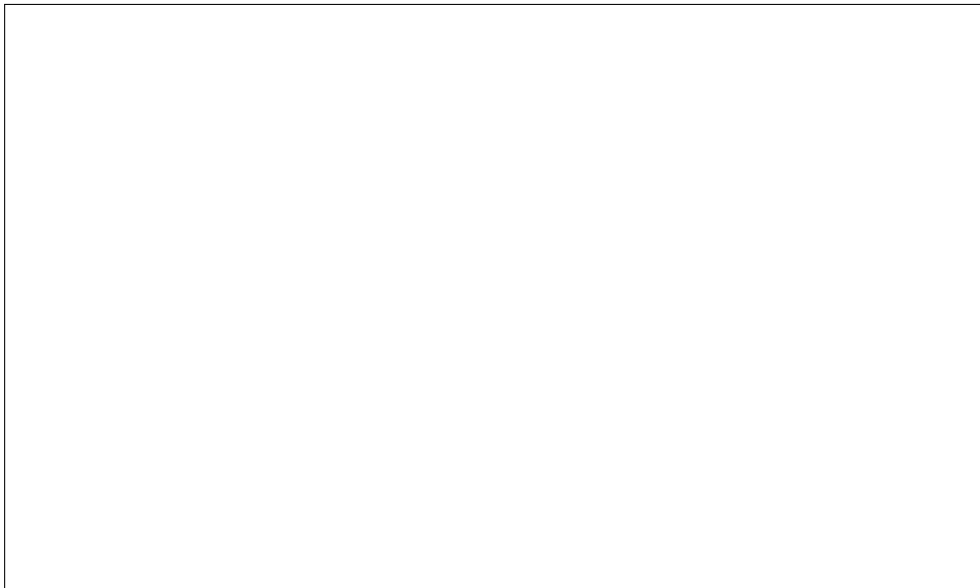
Sơ đồ tuần tự gồm Browser, Next.js, FastAPI, RAGService, Retriever, Qdrant, Evidence Gate, OpenRouter và Supabase ở bước lưu message. Backend tạo citation từ các Document đã dùng sau khi generation hoàn tất.



Hình 3.17: Sơ đồ tuần tự xử lý câu hỏi từ giao diện đến câu trả lời

3.9.3 Mở citation

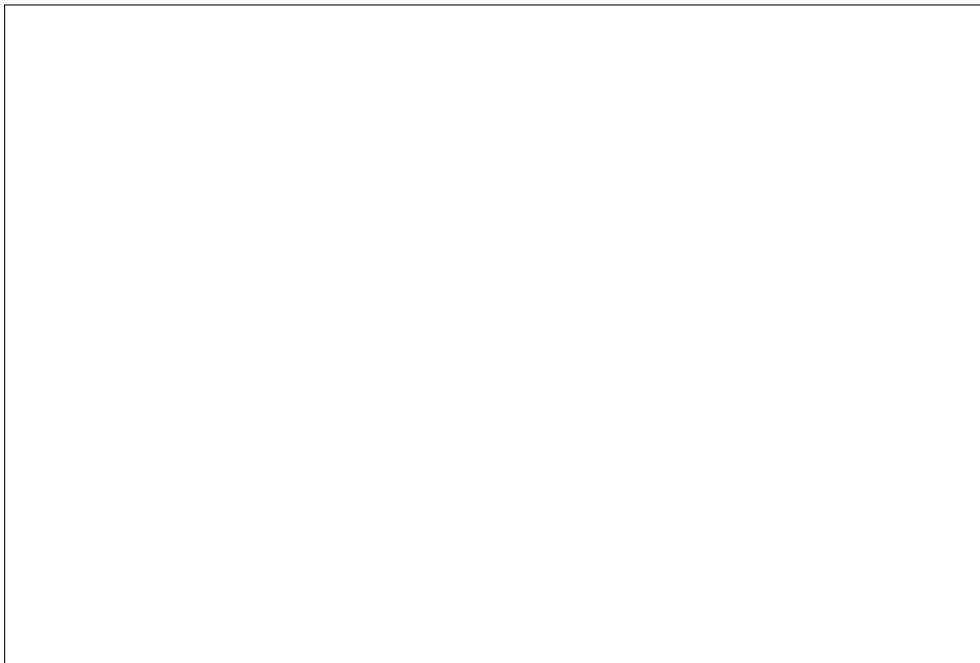
Người dùng chọn citation, frontend đọc metadata nguồn rồi mở văn bản hoặc trang PDF tương ứng.



Hình 3.18: Sơ đồ tuần tự mở nguồn từ một citation

3.9.4 Ingestion và index

Quy trình tạo chunk và ghi dữ liệu dẫn xuất vào Qdrant.



Hình 3.19: Sơ đồ tuần tự tạo chunks và chỉ mục Qdrant

3.9.5 Feedback và bookmark

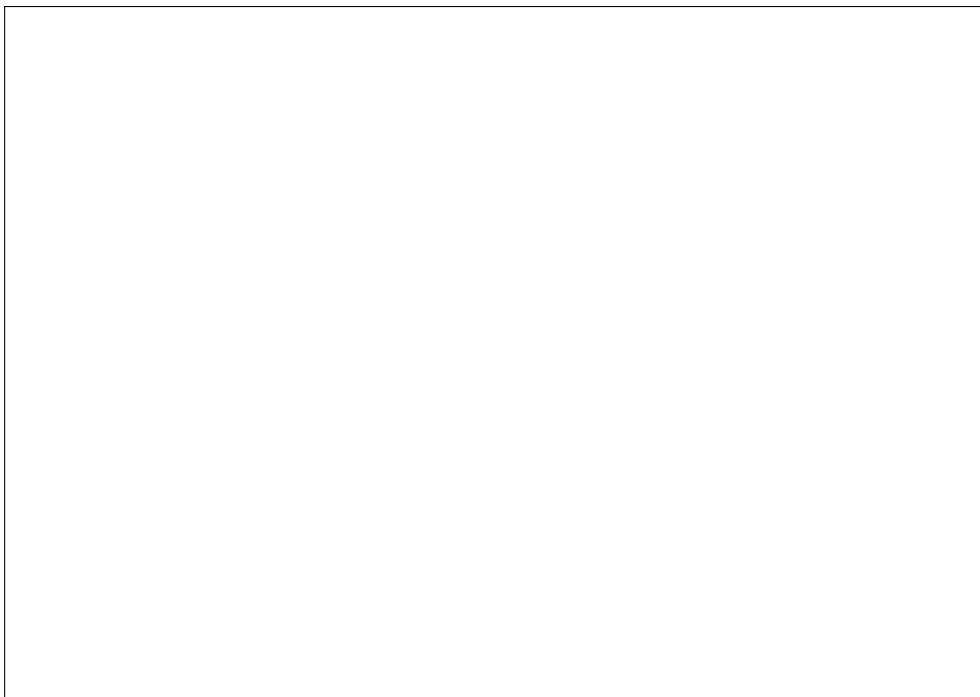
Frontend gửi request đến API, sau đó dữ liệu feedback và bookmark được lưu trong Supabase.



Hình 3.20: Sơ đồ tuần tự lưu feedback và bookmark

3.10 Kiến trúc thành phần

Kiến trúc thành phần gồm frontend Next.js, backend FastAPI, dịch vụ RAG, Qdrant, Supabase và OpenRouter. Frontend gửi yêu cầu đến FastAPI. Backend xử lý xác thực, hội thoại, nguồn pháp luật và hỏi đáp. Dịch vụ RAG sử dụng Qdrant để tìm kiếm và OpenRouter để gọi mô hình. Supabase lưu dữ liệu ứng dụng cùng thông tin xác thực. Dữ liệu pháp luật và chỉ mục tìm kiếm được quản lý tách khỏi các bảng ứng dụng.



Hình 3.21: Sơ đồ thành phần của hệ thống

3.11 Kiến trúc triển khai

Trong môi trường phát triển, Docker Compose chạy frontend và backend theo cấu hình của đề tài. Qdrant dùng local persistence khi cấu hình đường dẫn cục bộ. Supabase và OpenRouter được truy cập qua mạng. Sơ đồ deployment tách rõ trình duyệt, máy chạy ứng dụng và các dịch vụ bên ngoài. Corpus pháp luật không nằm trong Supabase.

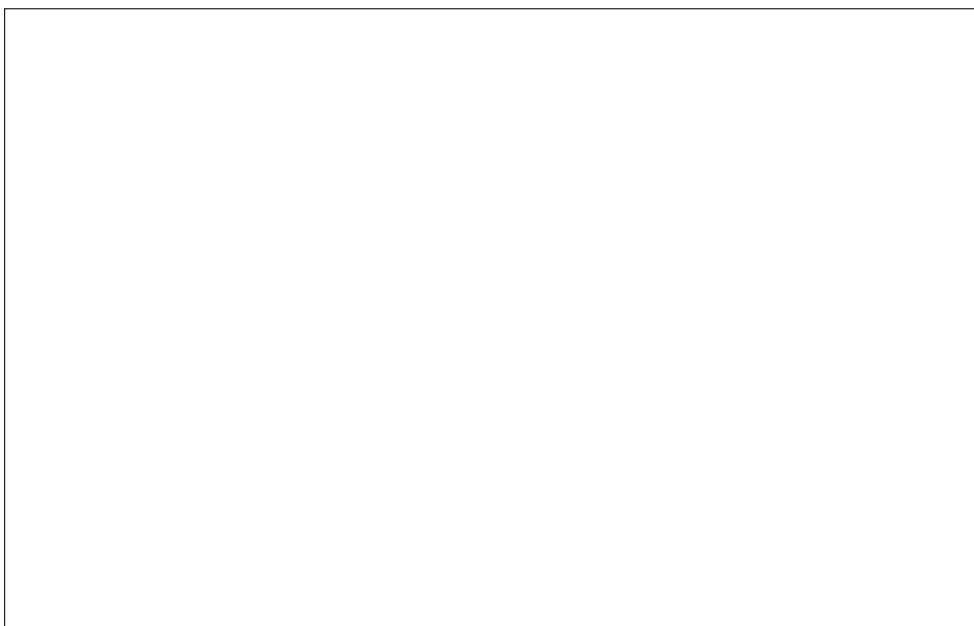


Hình 3.22: Sơ đồ triển khai hệ thống

3.12 Tổ chức module backend và frontend

3.12.1 Backend

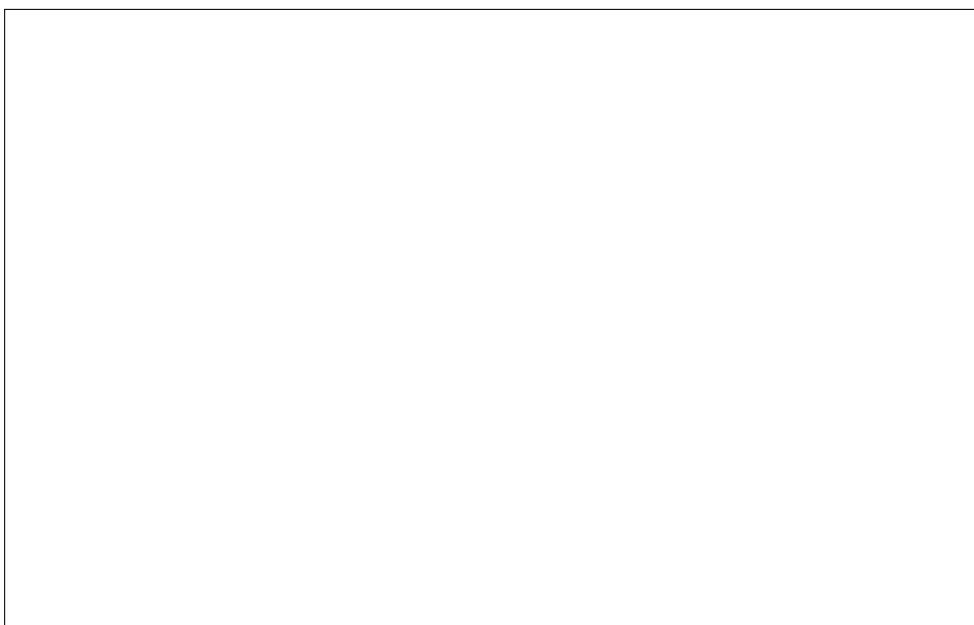
Backend được chia thành các package chính gồm ingestion, rag, legal, auth, chats, database, evaluation và config. Trong package rag, các module analyzer, references, retrieval, cross refs, evidence, generator và service được tách theo trách nhiệm xử lý.



Hình 3.23: Sơ đồ module backend

3.12.2 Frontend

Frontend theo App Router với các route /chat và /legal-sources, cùng component dùng lại cho citation, viewer và trạng thái người dùng.



Hình 3.24: Sơ đồ module frontend

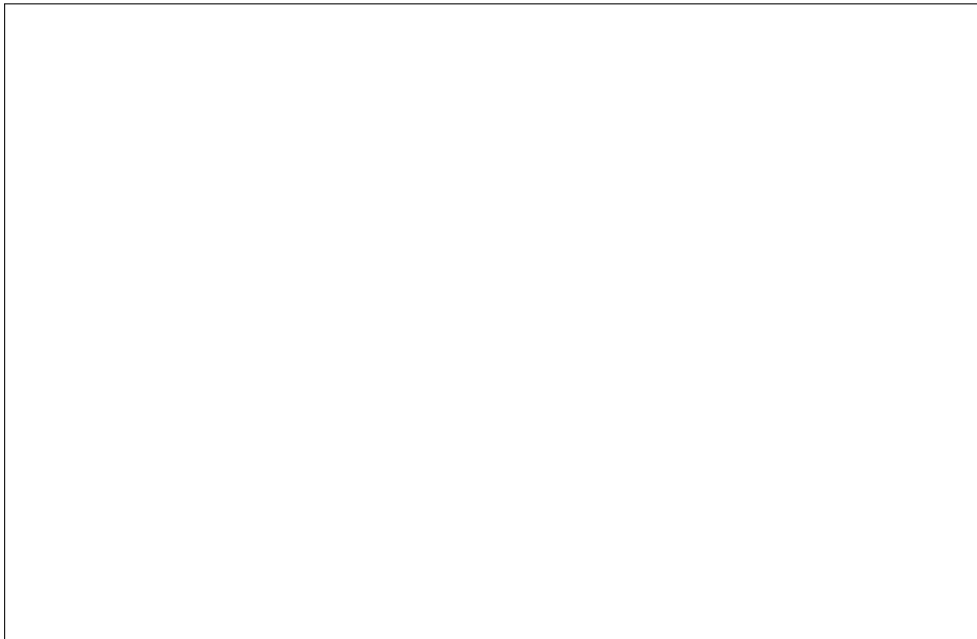
3.13 Thiết kế cơ sở dữ liệu

3.13.1 Nguyên tắc phân tách dữ liệu

Dữ liệu ứng dụng được lưu trong Supabase/PostgreSQL và có quan hệ sở hữu theo người dùng. Qdrant lưu vector cùng payload của chunk pháp luật. Manifest, Markdown và JSONL nằm ở lớp tệp dữ liệu. Phần mô hình cơ sở dữ liệu quan hệ dưới đây mô tả Supabase. Qdrant được trình bày riêng ở mục thiết kế dữ liệu retrieval.

3.13.2 Mô hình dữ liệu khái niệm CDM/MCD

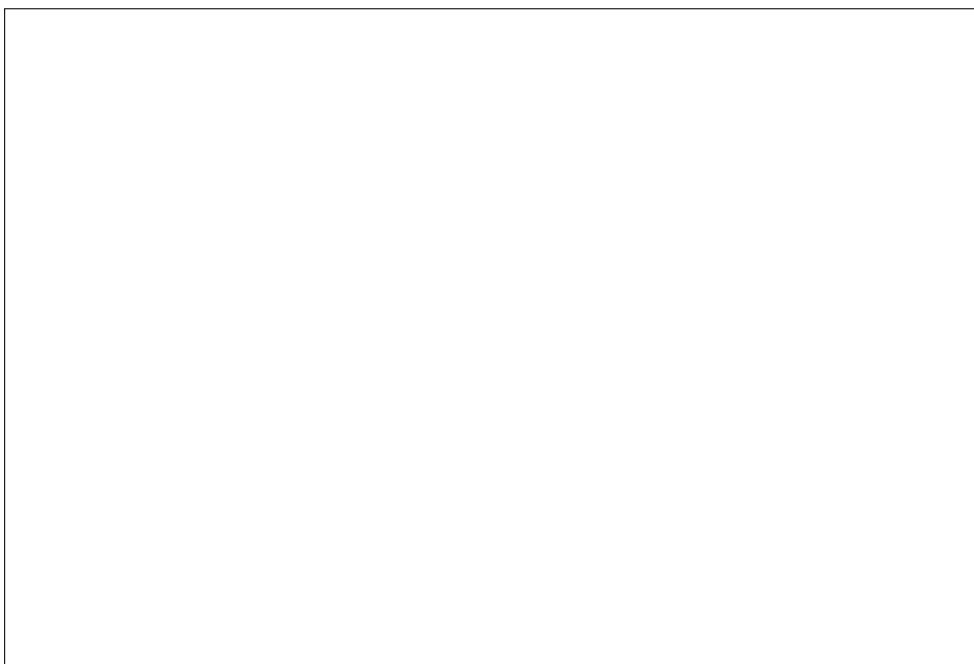
CDM gồm các thực thể Người dùng, Phiên chat, Tin nhắn, Phản hồi và Bookmark cùng quan hệ giữa chúng. `auth.users` là thực thể xác thực do Supabase quản lý.



Hình 3.25: Mô hình dữ liệu khái niệm CDM/MCD

3.13.3 Mô hình dữ liệu logic LDM/MLD

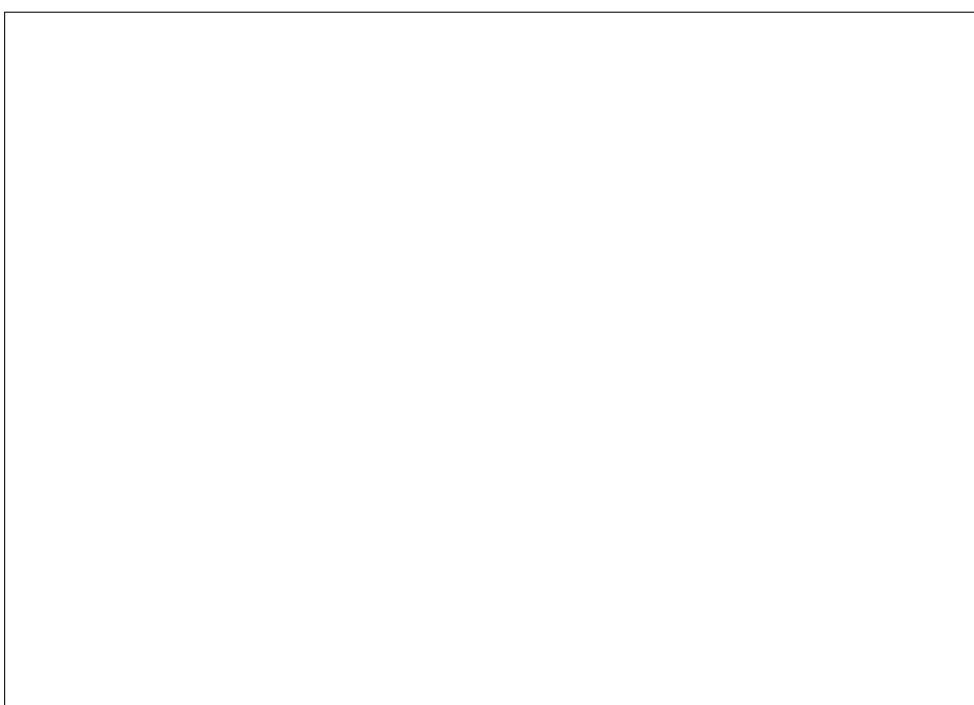
LDM bổ sung khóa chính, khóa ngoại, cardinality và các thuộc tính chính. Kiểu dữ liệu vật lý được trình bày ở PDM.



Hình 3.26: Mô hình dữ liệu logic LDM/MLD

3.13.4 Mô hình dữ liệu vật lý PDM/MPD

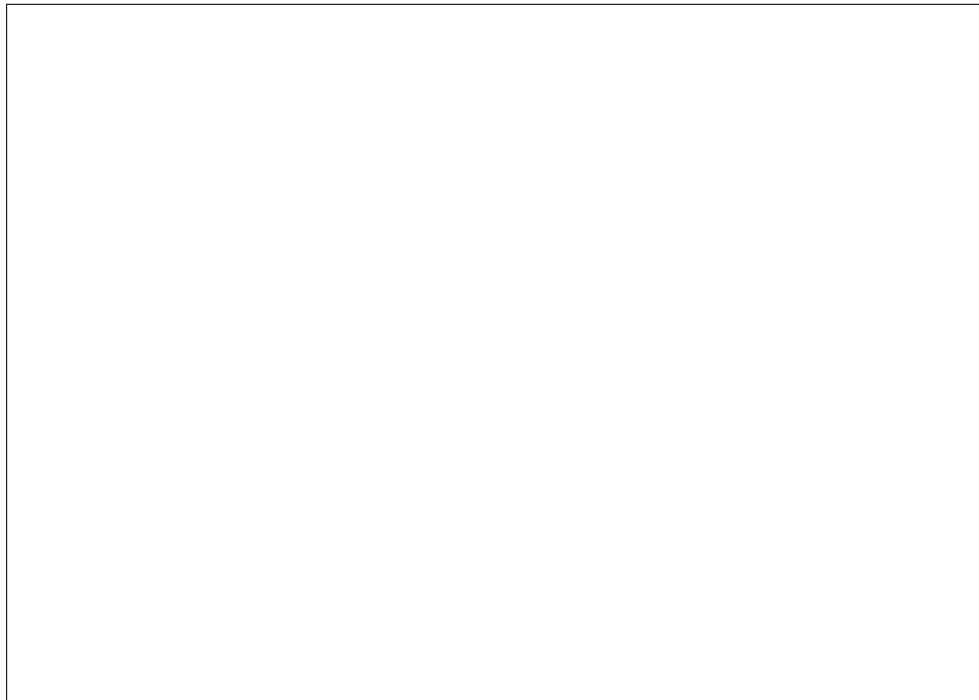
PDM thể hiện tên bảng, cột, kiểu dữ liệu, khóa chính, khóa ngoại, ràng buộc duy nhất và ràng buộc kiểm tra của cơ sở dữ liệu ứng dụng.



Hình 3.27: Mô hình dữ liệu vật lý PDM/MPD

3.13.5 Sơ đồ ERD

ERD tập trung vào năm bảng dữ liệu ứng dụng và các quan hệ khóa ngoại giữa phiên chat, tin nhắn, phản hồi, bookmark và tài khoản xác thực.



Hình 3.28: Sơ đồ thực thể liên kết ERD của dữ liệu ứng dụng

3.13.6 Bảng profiles

Bảng profiles mở rộng thông tin của `auth.users`. ID vừa là khóa chính vừa tham chiếu user của Supabase Auth.

Bảng 3.3: Thiết kế bảng profiles

STT	Thuộc tính	Khóa/ràng buộc	Kiểu dữ liệu	Diễn giải
1	id	PK, FK auth.users	UUID	Mã người dùng
2	display_name		TEXT	Tên hiển thị
3	avatar_url		TEXT	Đường dẫn ảnh đại diện
4	created_at	NOT NULL, DEFAULT	TIMESTAMPTZ	Thời điểm tạo
5	updated_at	NOT NULL, DEFAULT	TIMESTAMPTZ	Thời điểm cập nhật

3.13.7 Bảng chat_sessions

Bảng chat_sessions lưu từng phiên hội thoại và liên kết phiên với người dùng sở hữu.

Bảng 3.4: Thiết kế bảng chat_sessions

STT	Thuộc tính	Khóa/ràng buộc	Kiểu dữ liệu	Diễn giải
1	id	PK, DEFAULT	UUID	Mã phiên chat
2	user_id	FK auth.users, NOT NULL	UUID	Chủ sở hữu phiên chat
3	title	NOT NULL, DEFAULT, CHECK từ 1 đến 200 ký tự	TEXT	Tiêu đề phiên
4	deleted	NOT NULL, DEFAULT false	BOOLEAN	Cờ xóa mềm
5	created_at	NOT NULL, DEFAULT	TIMESTAMPTZ	Thời điểm tạo
6	updated_at	NOT NULL, DEFAULT	TIMESTAMPTZ	Lần cập nhật cuối

Bảng có ràng buộc unique theo cặp (id, user_id) để messages có thể tham chiếu đồng thời session và chủ sở hữu.

3.13.8 Bảng messages

Bảng messages lưu nội dung trao đổi, trạng thái xử lý, citation và metadata của từng tin nhắn.

Bảng 3.5: Thiết kế bảng messages

STT	Thuộc tính	Khóa/ràng buộc	Kiểu dữ liệu	Diễn giải
1	id	PK, DEFAULT	UUID	Mã tin nhắn
2	session_id	FK ghép với user_id, NOT NULL	UUID	Phiên chat chứa tin nhắn
3	user_id	FK auth.users, NOT NULL	UUID	Chủ sở hữu dữ liệu
4	role	NOT NULL, DEFAULT, CHECK user/assistant	TEXT	Vai trò tin nhắn
5	content	NOT NULL, CHECK không rỗng	TEXT	Nội dung hiển thị
6	status	NOT NULL, DEFAULT, CHECK độ dài	TEXT	Trạng thái xử lý
7	response		JSONB	Response có cấu trúc nếu cần lưu
8	citations	NOT NULL, DEFAULT []	JSONB	Danh sách citation

9	metadata	NOT NULL, DEFAULT {}	JSONB	Metadata bổ sung
10	created_at	NOT NULL, DEFAULT	TIMESTAMPTZ	Thời điểm tạo
11	updated_at	NOT NULL, DEFAULT	TIMESTAMPTZ	Thời điểm cập nhật

3.13.9 Bảng feedback

Bảng feedback lưu đánh giá nhị phân và ghi chú tùy chọn cho một tin nhắn.

Bảng 3.6: Thiết kế bảng feedback

STT	Thuộc tính	Khóa/ràng buộc	Kiểu dữ liệu	Diễn giải
1	id	PK, DEFAULT	UUID	Mã phản hồi
2	message_id	FK messages, NOT NULL	UUID	Tin nhắn được đánh giá
3	user_id	FK auth.users, NOT NULL	UUID	Người gửi phản hồi
4	rating	NOT NULL, CHECK 0 hoặc 1	INTEGER	Đánh giá nhị phân
5	comment		TEXT	Ghi chú tùy chọn
6	created_at	NOT NULL, DEFAULT	TIMESTAMPTZ	Thời điểm tạo
7	updated_at	NOT NULL, DEFAULT	TIMESTAMPTZ	Thời điểm cập nhật

Mỗi người dùng chỉ có một feedback cho một message nhờ ràng buộc duy nhất trên cặp (message_id, user_id).

3.13.10 Bảng bookmarks

Bảng bookmarks lưu các tin nhắn mà người dùng đánh dấu để xem lại.

Bảng 3.7: Thiết kế bảng bookmarks

STT	Thuộc tính	Khóa/ràng buộc	Kiểu dữ liệu	Diễn giải
1	id	PK, DEFAULT	UUID	Mã bookmark
2	message_id	FK messages, NOT NULL	UUID	Tin nhắn được đánh dấu
3	user_id	FK auth.users, NOT NULL	UUID	Chủ sở hữu bookmark
4	created_at	NOT NULL, DEFAULT	TIMESTAMPTZ	Thời điểm tạo

Unique (message_id, user_id) ngăn tạo bookmark trùng cho cùng người dùng.

3.13.11 Chỉ mục, trigger và ràng buộc toàn vẹn

Schema tạo index theo `user_id`, `updated_at`, `session_id` và `created_at` để hỗ trợ các truy vấn thường gặp. Trigger `set_updated_at` tự cập nhật thời gian sửa bản ghi. Trigger trên `messages` cập nhật `chat_sessions.updated_at` khi có message mới hoặc sửa. Trigger `handle_new_user` tạo profile sau khi Supabase Auth tạo user.

Các ràng buộc toàn vẹn gồm khóa chính, khóa ngoại, NOT NULL, UNIQUE, CHECK và DEFAULT. Việc dùng cả constraint ở database và validation ở API giúp giảm khả năng dữ liệu sai lệch vào hệ thống.

3.13.12 Row Level Security

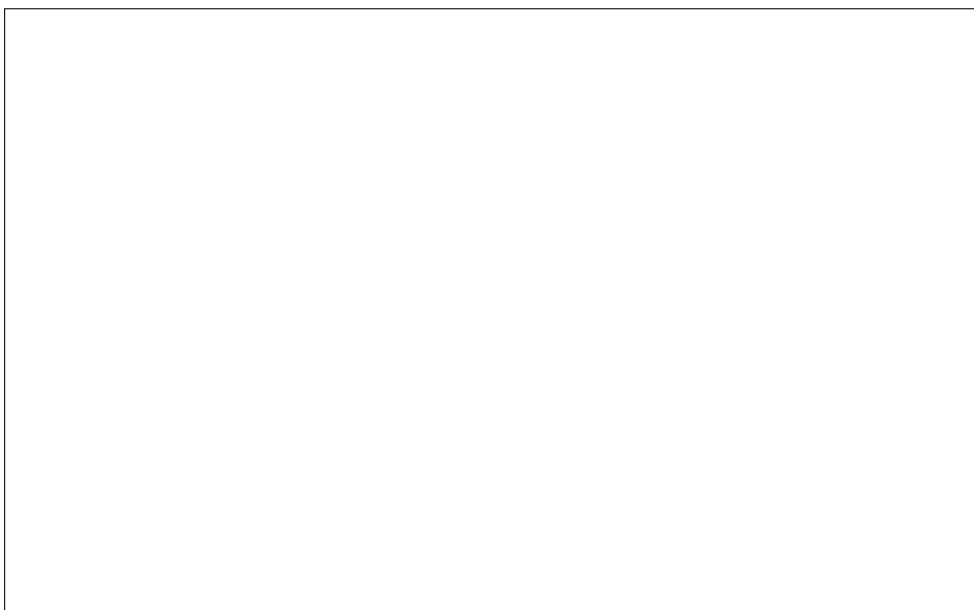
Tất cả năm bảng ứng dụng bật RLS. Policy dùng `auth.uid()` để giới hạn dữ liệu theo chủ sở hữu. Với `messages`, policy còn kiểm tra session tương ứng thuộc cùng người dùng. Sơ đồ bảo mật thể hiện browser dùng `publishable key` và backend giữ `service-role key`.

3.14 Thiết kế dữ liệu retrieval trong Qdrant

Qdrant lưu chỉ mục retrieval của hệ thống. Mỗi point có dense vector, sparse vector và payload metadata. Khi truy vấn chứa tham chiếu rõ ràng, retriever tạo bộ lọc trên các trường `metadata.article`, `metadata.clause` và `metadata.point`.

Bảng 3.8: Các trường metadata chính trong chunk/Qdrant payload

Trường	Dạng dữ liệu	Ý nghĩa
<code>chunk_id</code>	Chuỗi	Định danh chunk, dạng document-id + số thứ tự
<code>document_id</code>	Chuỗi	Mã nội bộ của văn bản
<code>document_name</code>	Chuỗi	Tên văn bản
<code>document_number</code>	Chuỗi	Số hiệu văn bản nếu có
<code>article</code>	Chuỗi	Số Điều
<code>clause</code>	Chuỗi/tùy chọn	Số Khoản
<code>point</code>	Chuỗi/tùy chọn	Nhãn Điểm
<code>source_file</code>	Chuỗi	Đường dẫn file nguồn
<code>source_url</code>	Chuỗi	URL nguồn nếu có
<code>source_kind</code>	Chuỗi	markdown hoặc pdf theo adapter nguồn
<code>content_sha256</code>	Chuỗi	SHA-256 của nội dung file Markdown nguồn

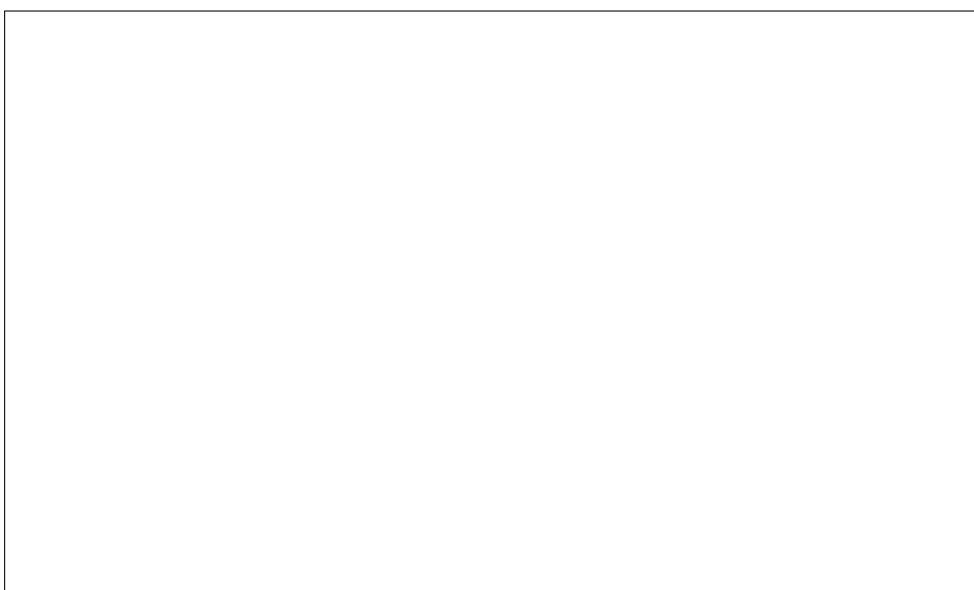


Hình 3.29: Cấu trúc point và payload trong Qdrant

3.15 Thiết kế manifest và chunks JSONL

3.15.1 Manifest

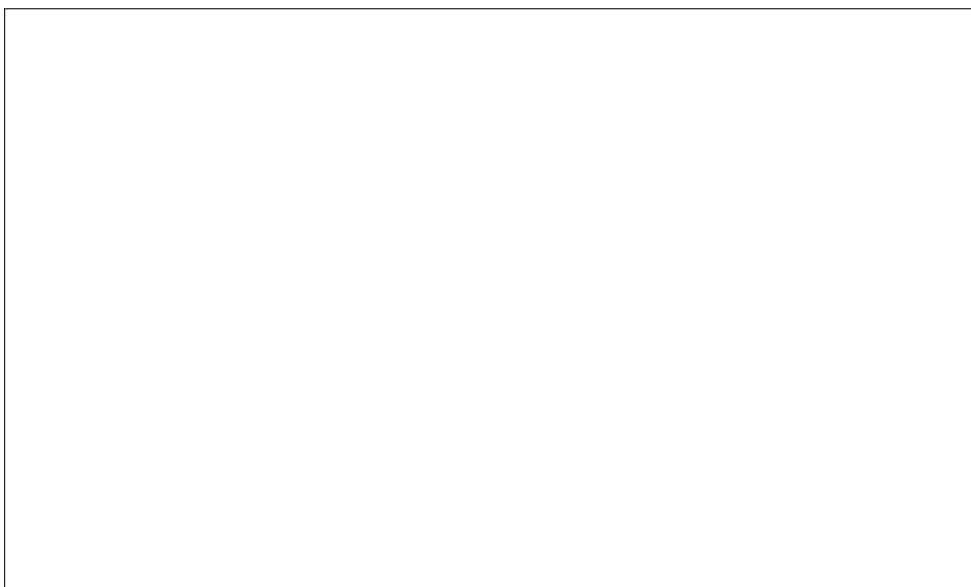
Manifest chứa danh sách văn bản và đường dẫn Markdown. Mỗi document entry liên kết định danh văn bản với tệp dữ liệu cục bộ tương ứng.



Hình 3.30: Cấu trúc manifest nguồn pháp luật

3.15.2 Chunks JSONL

Mỗi dòng JSONL gồm `page_content` và `metadata`. Dữ liệu này được dùng để xây dựng collection Qdrant.



Hình 3.31: Cấu trúc một dòng trong chunks JSONL

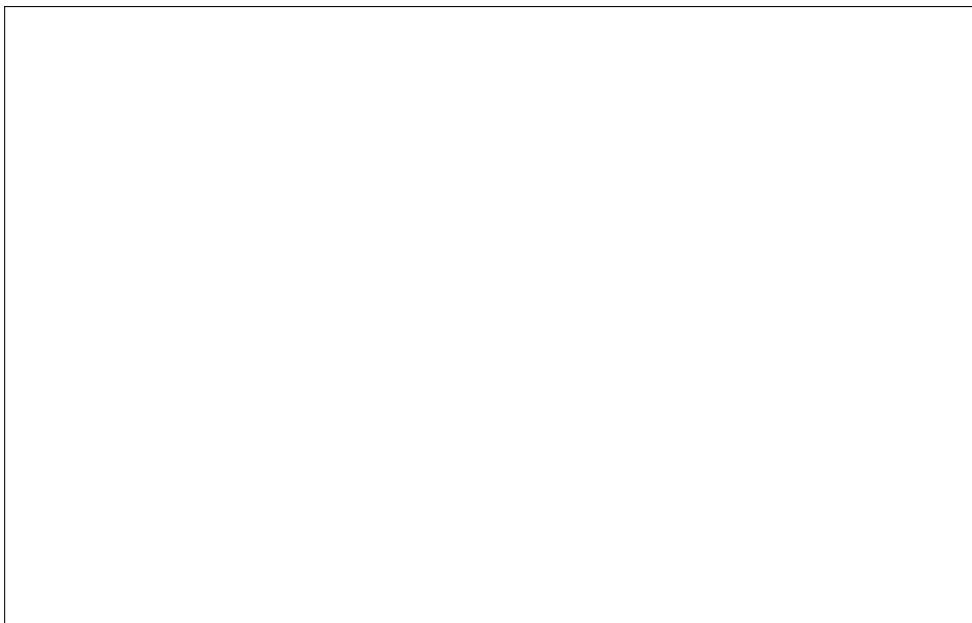
3.16 Thiết kế API

Các endpoint được chia theo nhóm hỏi đáp, xác thực, phiên chat và nguồn pháp luật. Bảng dưới đây ghi các endpoint chính dùng trong bản triển khai.

Bảng 3.9: Các endpoint chính của hệ thống

Phương thức	Endpoint	Mục đích
POST	/api/v1/chat	Gửi câu hỏi pháp luật
POST	/api/v1/auth/register	Đăng ký
POST	/api/v1/auth/login	Đăng nhập
GET	/api/v1/auth/me	Lấy người dùng hiện tại
GET, POST	/api/v1/chats	Lấy danh sách hoặc tạo phiên chat
GET, PATCH, DELETE	/api/v1/chats/{session_id}	Xem, sửa hoặc xóa phiên chat
POST	/api/v1/chats/{session_id}/messages	Lưu tin nhắn
POST	/api/v1/chats/{session_id}/messages/{message_id}/feedback	Lưu phản hồi
POST	/api/v1/chats/{session_id}/bookmarks	Tạo bookmark
GET	/api/v1/legal-documents	Danh sách văn bản
GET	/api/v1/legal-documents/{document_id}	Chi tiết văn bản
GET	/api/v1/legal-documents/{document_id}/provisions	Danh sách điều khoản
GET	/api/v1/legal-search	Tìm nguồn pháp luật
GET	/api/v1/health/live	Kiểm tra tiến trình API đang sống

GET	/api/v1/health/ready	Kiểm tra sẵn sàng gồm Supabase và Qdrant
-----	----------------------	--



Hình 3.32: Bản đồ endpoint và nhóm chức năng API

3.17 Thiết kế các giải thuật chính

3.17.1 Giải thuật tách Điều, Khoản và Điểm

Quy trình có thể trình bày theo các bước sau:

1. Đọc file Markdown và tách front matter nếu có.
2. Duyệt từng dòng, chuẩn hóa khoảng trắng.
3. Nếu dòng khớp mẫu Điều, ghi chunk trước đó và cập nhật số Điều, đặt Khoản/Điểm về rỗng.
4. Nếu dòng khớp mẫu Khoản trong một Điều, ghi chunk trước đó và cập nhật Khoản.
5. Nếu dòng khớp mẫu Điểm trong một Khoản, ghi chunk trước đó và cập nhật Điểm.
6. Nội dung thường được thêm vào buffer hiện tại.
7. Khi kết thúc file, ghi buffer cuối và gán metadata, chunk ID, SHA-256.

Độ phức tạp theo số dòng là tuyến tính $O(n)$ nếu bỏ qua chi phí regex theo chiều dài từng dòng.

3.17.2 Giải thuật retrieval

1. Phân tích tham chiếu pháp lý trong câu hỏi.
2. Nếu có tham chiếu, tạo filter Điều/Khoản/Điểm.
3. Gọi hybrid similarity search trên Qdrant với số kết quả lớn hơn top-k mục tiêu để có dư địa lọc.
4. Ưu tiên và lọc các kết quả khớp tham chiếu.
5. Nếu còn thiếu, dùng Qdrant scroll để tìm exact metadata.

6. Bổ sung tối đa một số sibling sanction và cross-reference.
7. Trả về danh sách giới hạn top-k.

3.17.3 Giải thuật phân tích câu hỏi nhiều ý

Analyzer chuẩn hóa câu hỏi, thay một số viết tắt và tách câu theo các từ nối đã định nghĩa. Sau đó mô-đun nhận diện các yêu cầu như mức phạt, trừ điểm hoặc tước quyền. Danh sách intent được khử trùng lặp và giới hạn số phần tử, vì vậy số truy vấn con có giới hạn xác định.

3.17.4 Giải thuật gộp RRF

Với mỗi danh sách kết quả, document ở hạng r nhận thêm $1/(60 + r)$. Score được cộng cho cùng document key. Sau khi sắp xếp giảm dần, service áp dụng diversity limit để mỗi bucket văn bản/Điều/Khoản không có quá ba chunk. Kết quả cuối được gắn nhãn intent để generator biết nhóm căn cứ nào phục vụ phần nào của câu hỏi.

3.17.5 Giải thuật evidence gate

Evidence gate chuyển iterable sang danh sách, kiểm tra số lượng tối thiểu, kiểm tra tham chiếu bắt buộc nếu có, và kiểm tra score nếu caller truyền ngưỡng. Chỉ khi tất cả điều kiện đạt mới trả `allowed=True`. Nếu không, service dừng generation và trả thông báo thiếu căn cứ.

3.17.6 Giải thuật dựng citation

Backend duyệt các document đã dùng, tạo identity ổn định bằng chunk ID hoặc source + content để loại trùng, sau đó lấy document name/number, Article, Clause, Point, page, source file, excerpt và PDF URL. Citation này được trả thành dữ liệu có cấu trúc cho frontend.

3.18 Thiết kế bảo mật

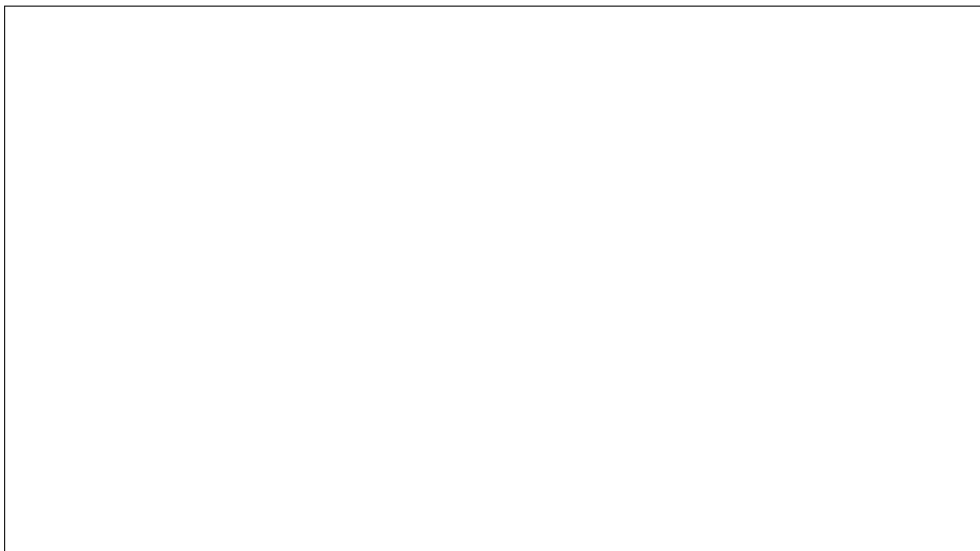
Các khóa bí mật được đọc từ biến môi trường. Trình duyệt chỉ dùng khóa Supabase công khai phù hợp cho client. Khóa service role và OpenRouter API key nằm ở backend. RLS giới hạn quyền trên từng hàng dữ liệu. API dùng trace ID để hỗ trợ theo dõi request mà không đưa khóa bí mật vào log. Các endpoint health tách kiểm tra tiến trình đang chạy với kiểm tra khả năng kết nối các dịch vụ phụ thuộc.



Hình 3.33: Luồng xác thực, phân quyền và bảo vệ khóa bí mật

3.19 Thiết kế kiểm thử và đánh giá

Kiểm thử được chia theo mức. Unit test áp dụng cho parser, regex tham chiếu, analyzer, evidence và metrics. Integration test áp dụng cho API, Supabase và Qdrant. E2E dùng Playwright để mô phỏng người dùng trên giao diện. Thesis evaluation là lớp riêng, dùng 40 case để đánh giá retrieval/citation/abstention/latency và có bước review thủ công.



Hình 3.34: Mô hình các lớp kiểm thử và đánh giá

3.20 Kết luận chương

Chương 3 mô tả yêu cầu, luồng dữ liệu, mô hình tương tác, kiến trúc, API và thiết kế dữ liệu. Supabase lưu dữ liệu ứng dụng; Qdrant và corpus pháp luật được quản lý riêng cho retrieval.

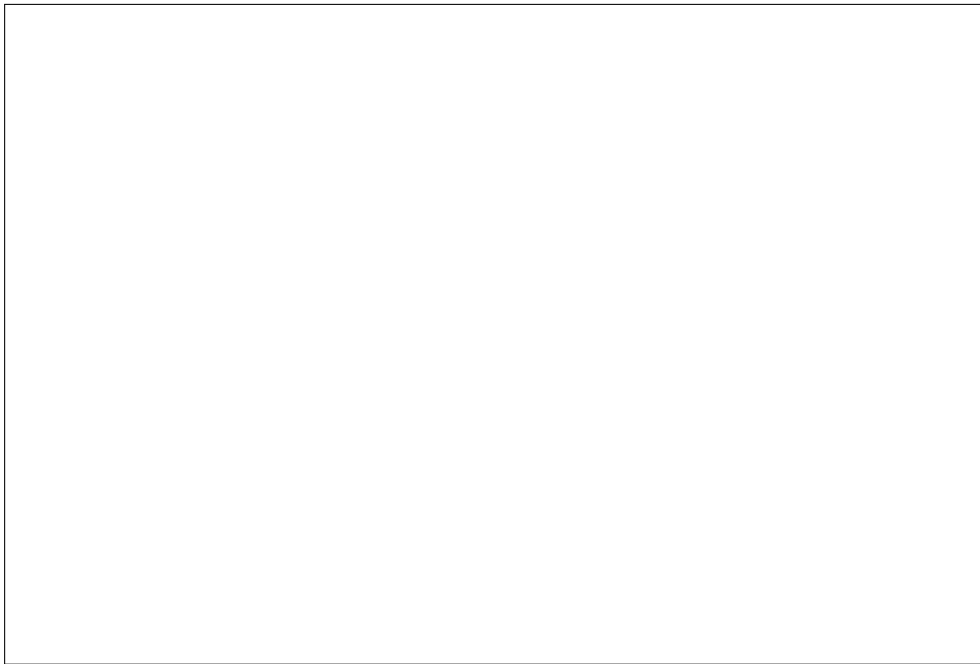
CHƯƠNG IV. KẾT QUẢ THỰC HIỆN VÀ GIAO DIỆN

4.1 Tổng quan kết quả triển khai

Chương này trình bày các chức năng đã triển khai ở giao diện người dùng, API vận hành, kiểm thử và đánh giá hệ thống. Các màn hình chính gồm đăng ký, đăng nhập, hỏi đáp, xem trích dẫn, tra cứu nguồn pháp luật, lịch sử hội thoại, phản hồi và đánh dấu câu trả lời.

4.2 Giao diện trang đầu

Trang đầu cung cấp khu vực điều hướng và lối vào chức năng hỏi đáp pháp luật.

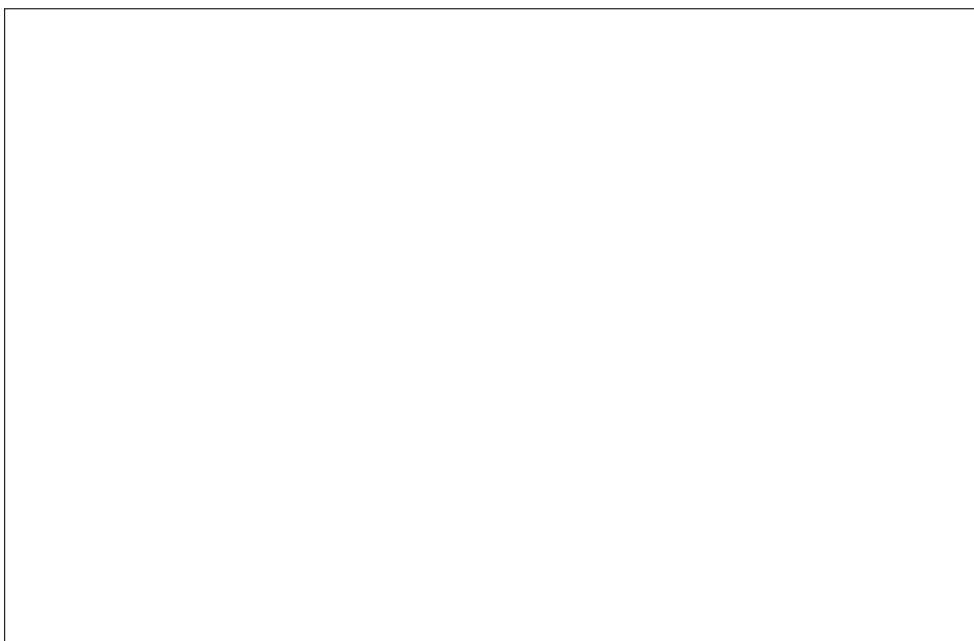


Hình 4.1: Giao diện trang đầu của hệ thống

4.3 Đăng ký và đăng nhập

4.3.1 Đăng ký

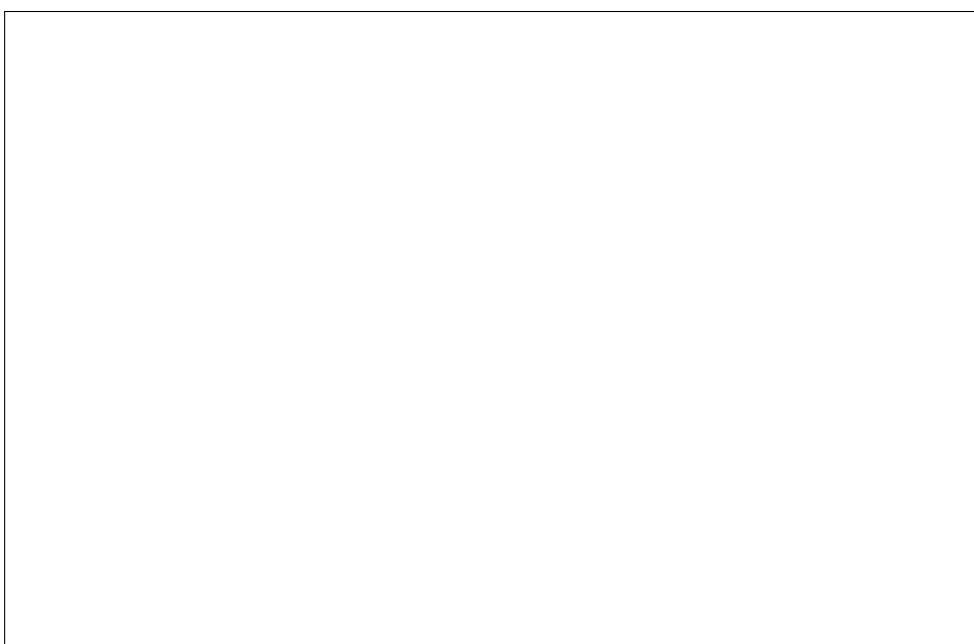
Màn hình đăng ký gồm các trường nhập liệu, thông báo validation và trạng thái phản hồi sau khi Supabase xử lý yêu cầu tạo tài khoản.



Hình 4.2: Giao diện đăng ký tài khoản

4.3.2 Đăng nhập

Màn hình đăng nhập gồm các trường thông tin, nút gửi yêu cầu và trạng thái báo lỗi khi dữ liệu không hợp lệ.



Hình 4.3: Giao diện đăng nhập

4.4 Giao diện hỏi đáp pháp luật

4.4.1 Trạng thái chưa có câu hỏi

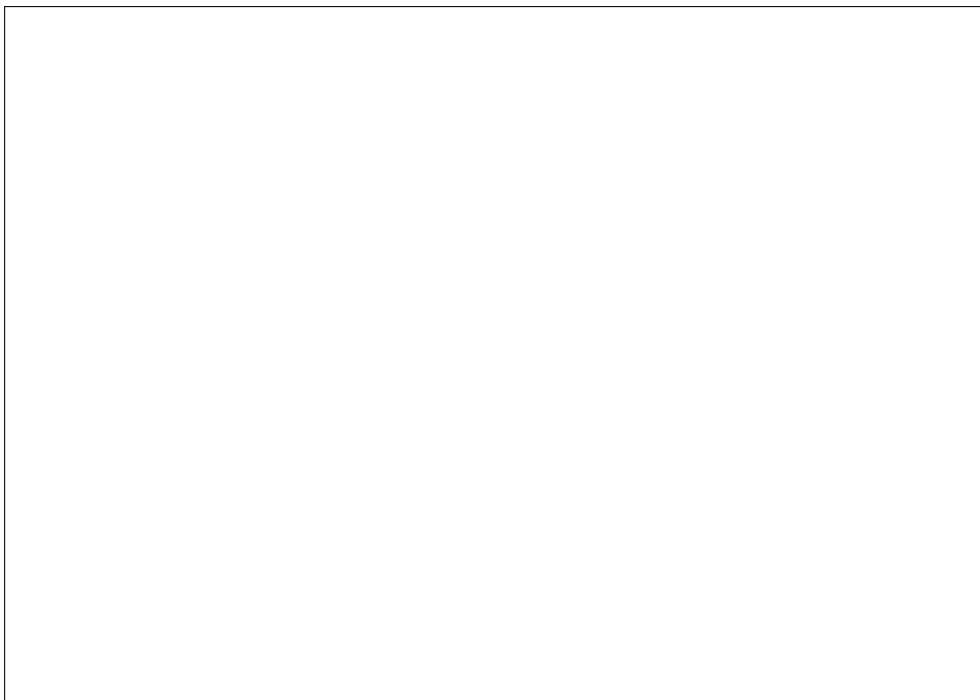
Trạng thái ban đầu của trang chat gồm vùng nhập câu hỏi và các thành phần điều hướng của phiên hội thoại.



Hình 4.4: Giao diện chat khi bắt đầu phiên

4.4.2 Câu trả lời có căn cứ

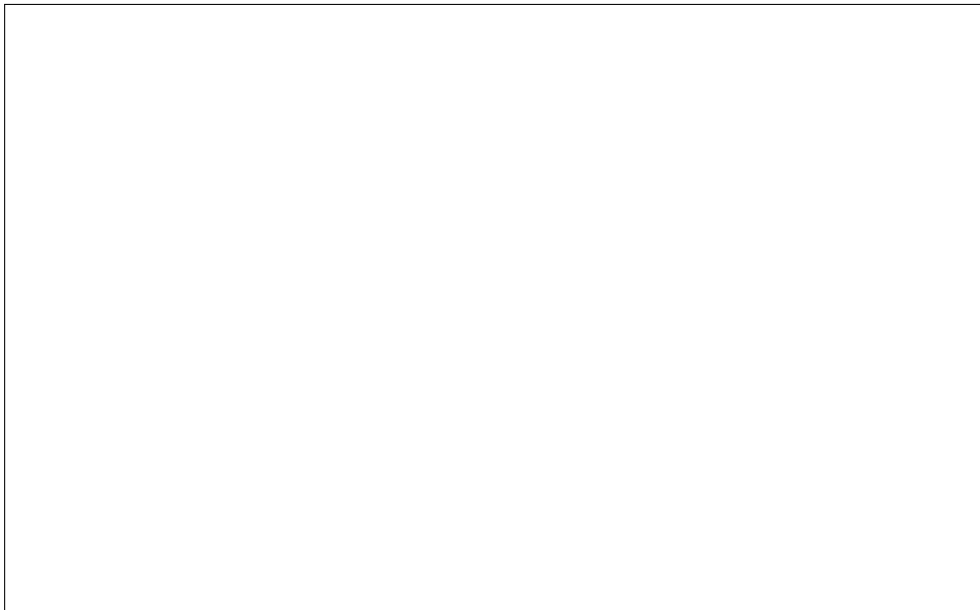
Sau khi xử lý câu hỏi, giao diện hiển thị nội dung trả lời, citation và thao tác mở nguồn tương ứng.



Hình 4.5: Giao diện câu trả lời pháp luật có trích dẫn

4.4.3 Trường hợp không đủ căn cứ

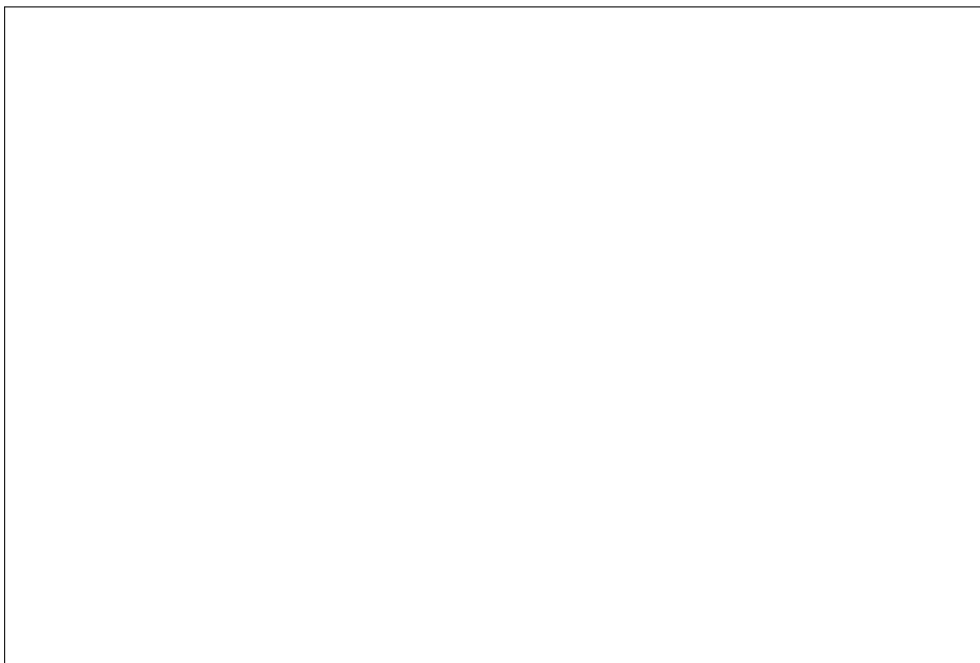
Khi không tìm thấy đủ căn cứ, giao diện hiển thị thông báo từ chối trả lời và không tạo nội dung pháp luật suy đoán.



Hình 4.6: Giao diện khi hệ thống từ chối trả lời do thiếu căn cứ

4.5 Hiển thị citation

Citation hiển thị thông tin văn bản, vị trí Điều, Khoản, Điểm và thao tác mở nguồn tương ứng.



Hình 4.7: Citation và thông tin văn bản nguồn

4.6 Trang nguồn pháp luật

4.6.1 Danh sách văn bản

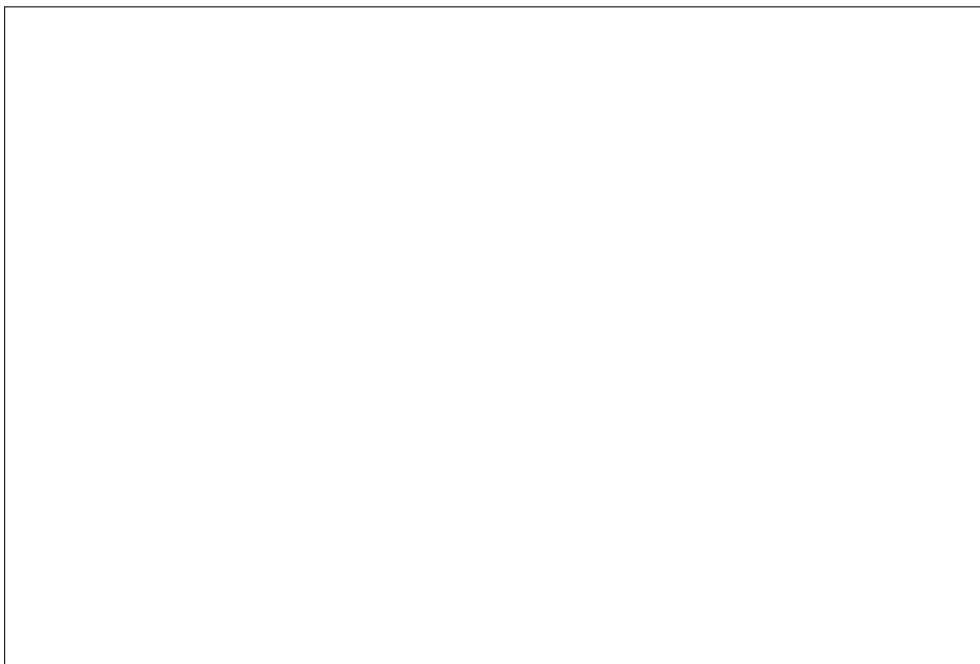
Trang nguồn pháp luật hiển thị danh sách văn bản từ legal API cùng các thông tin nhận biết từng nguồn.



Hình 4.8: Danh sách nguồn pháp luật

4.6.2 Tìm kiếm và lọc

Chức năng tìm kiếm nguồn gồm ô truy vấn, bộ lọc và danh sách kết quả theo văn bản hoặc điều khoản.



Hình 4.9: Tìm kiếm văn bản và điều khoản

4.6.3 Viewer Markdown

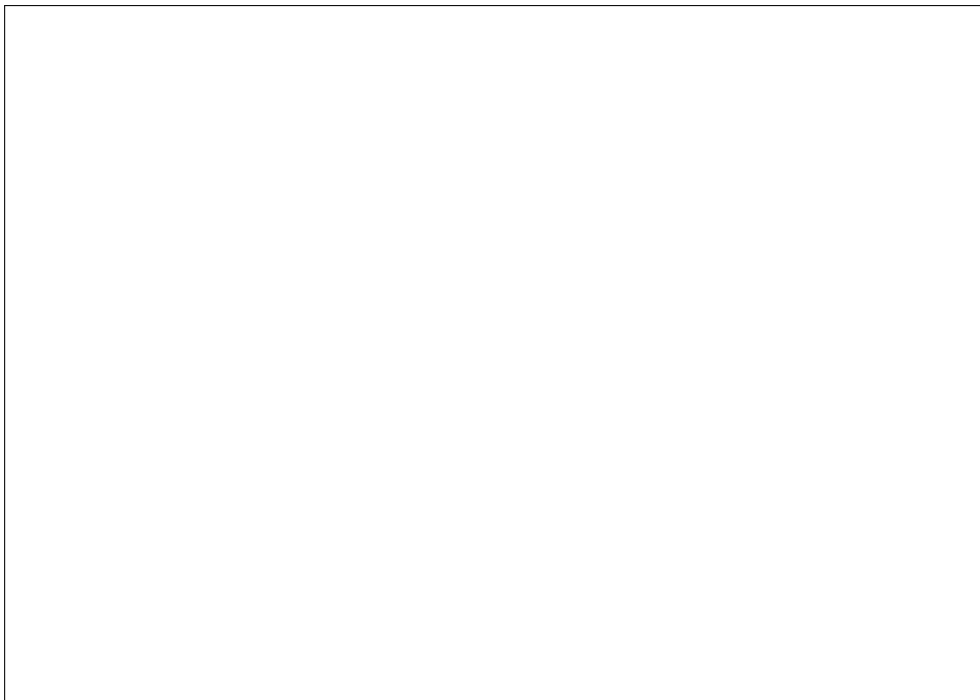
Viewer Markdown hiển thị nội dung pháp luật theo cấu trúc và vị trí được mở từ citation.



Hình 4.10: Hiển thị nguồn pháp luật dạng Markdown

4.6.4 Viewer PDF

Viewer PDF hiển thị trang tài liệu, điều khiển chuyển trang và vị trí được mở từ citation khi metadata cho phép.



Hình 4.11: Hiển thị PDF và điều hướng trang

4.7 Lịch sử hội thoại

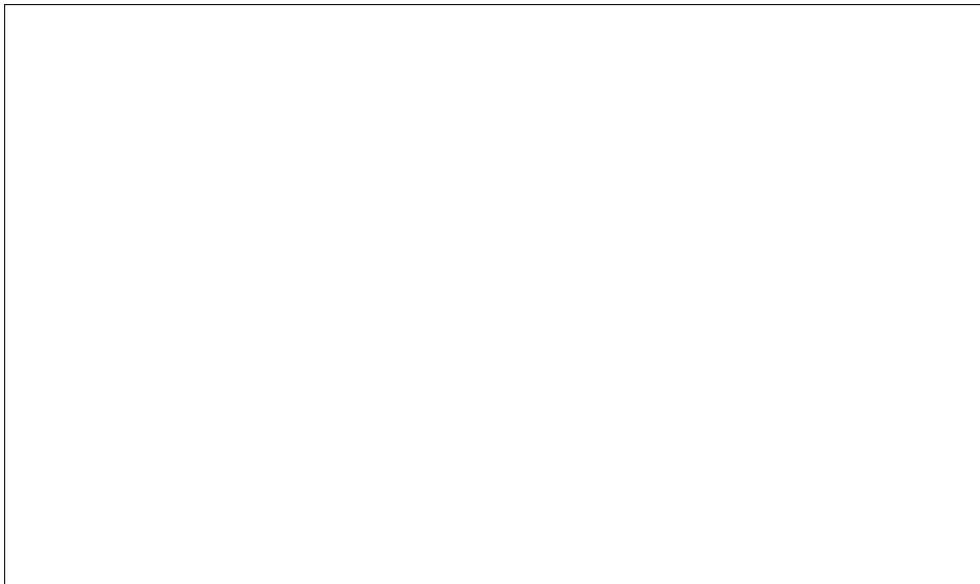
Trang lịch sử hiển thị danh sách phiên chat và các thao tác quản lý phiên của người dùng hiện tại.



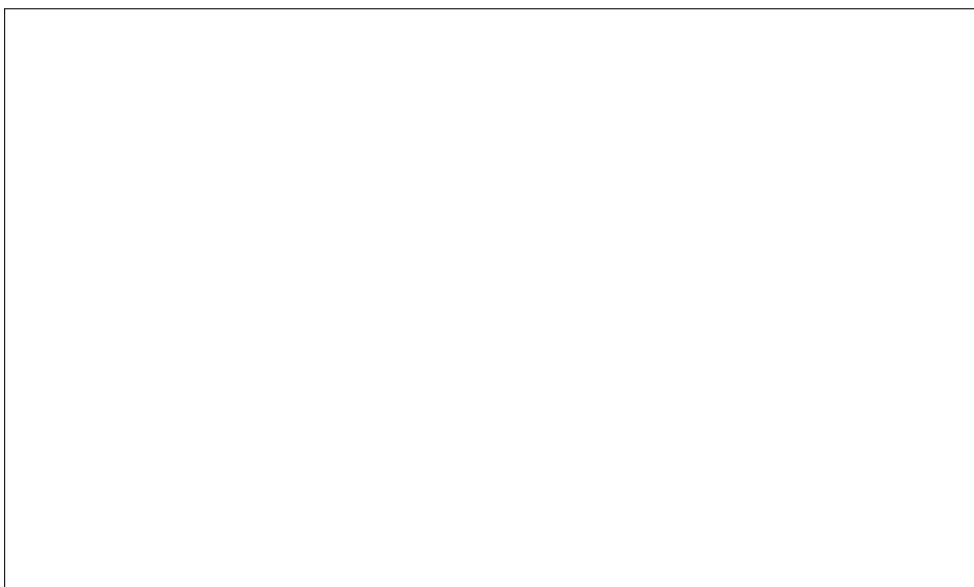
Hình 4.12: Giao diện quản lý phiên chat

4.8 Feedback và bookmark

Chức năng feedback cho phép người dùng gửi đánh giá đối với một câu trả lời đã lưu.



Hình 4.13: Giao diện gửi phản hồi cho câu trả lời

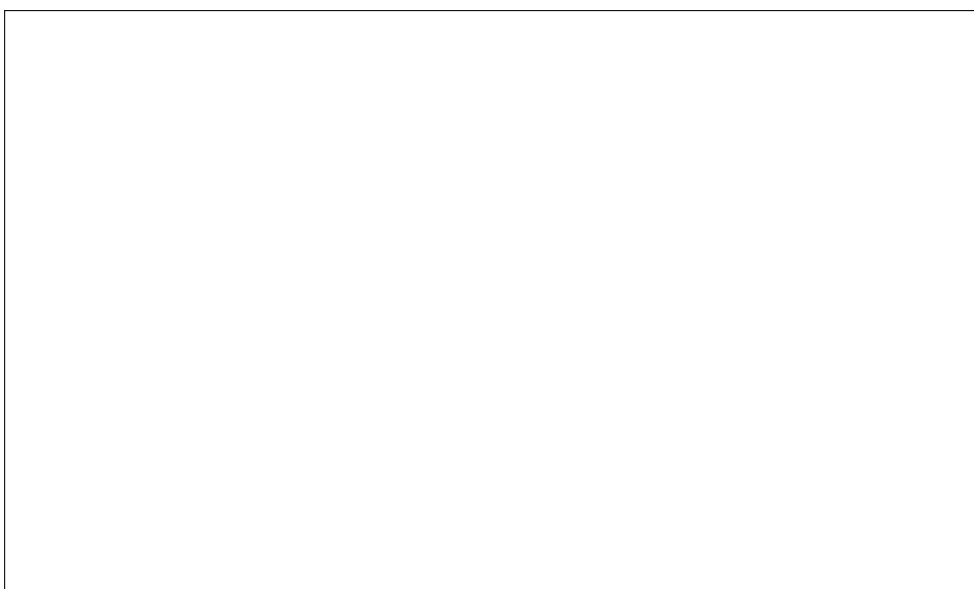


Hình 4.14: Giao diện đánh dấu câu trả lời

4.9 API và khả năng vận hành

4.9.1 Health check

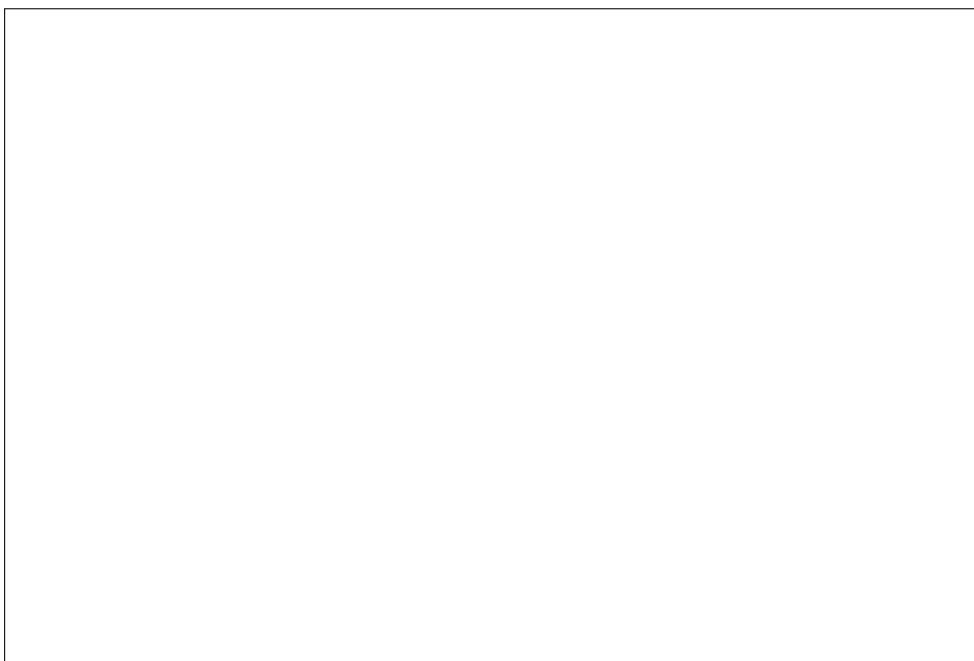
Health check trả trạng thái tiến trình API và trạng thái sẵn sàng của các dịch vụ phụ thuộc được cấu hình.



Hình 4.15: Kết quả health/readiness API

4.9.2 Tài liệu OpenAPI/Swagger

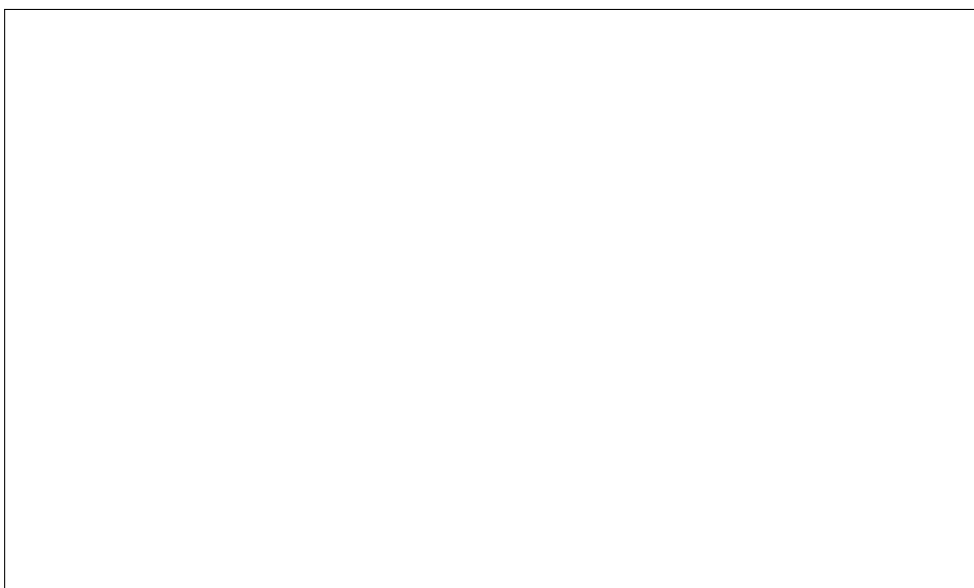
Tài liệu OpenAPI hiển thị các nhóm route chính cùng schema request và response do FastAPI sinh tự động.



Hình 4.16: Tài liệu API tự động của FastAPI

4.10 Kết quả kiểm thử

Kiểm thử được thực hiện theo các nhóm unit test, integration test và kiểm thử giao diện. Kết quả được ghi nhận theo số ca kiểm thử đạt, không đạt và điều kiện môi trường tương ứng.



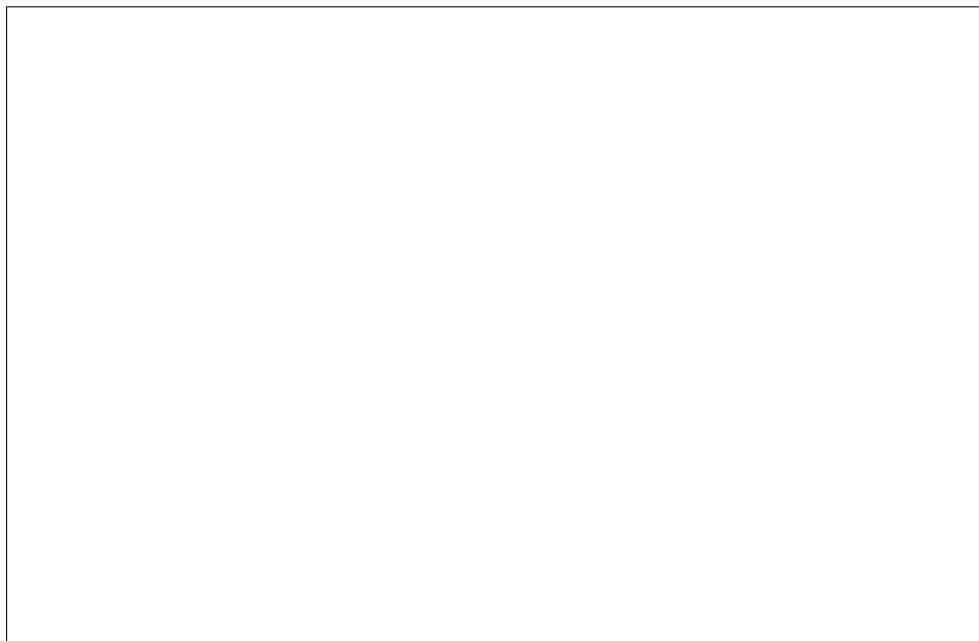
Hình 4.17: Kết quả kiểm thử của hệ thống

4.11 Kết quả đánh giá khóa luận

Bộ đánh giá 40 trường hợp sử dụng các chỉ số sau để đo chất lượng truy xuất, vị trí pháp lý, trích dẫn, hành vi từ chối trả lời và độ trễ.

Bảng 4.1: Các chỉ số dùng trong bộ đánh giá 40 trường hợp

Chỉ số	Ý nghĩa
Retrieval hit@k	Kiểm tra có truy xuất được ít nhất một vị trí pháp lý mong đợi trong nhóm kết quả đầu hay không
Document accuracy	Tỷ lệ xác định đúng văn bản pháp luật
Article accuracy	Tỷ lệ xác định đúng Điều
Clause accuracy	Tỷ lệ xác định đúng Khoản
Point accuracy	Tỷ lệ xác định đúng Điểm
Citation validity	Tỷ lệ trích dẫn hợp lệ theo dữ liệu nguồn
Answer correctness (manual)	Đánh giá thủ công mức độ đúng của nội dung câu trả lời
Abstention accuracy	Tỷ lệ hệ thống từ chối đúng ở các trường hợp thiếu căn cứ
Mean, P50, P95 latency	Thời gian phản hồi trung bình và các phân vị độ trễ



Hình 4.18: Tổng hợp kết quả đánh giá hệ thống

4.12 Phân tích lỗi

Lỗi được phân loại theo vị trí phát sinh. Các nhóm chính gồm không tìm được chunk đúng, tìm đúng văn bản nhưng sai Điều hoặc Khoản, truy xuất đủ căn cứ nhưng phần diễn giải sai và hành vi từ chối trả lời khác với kỳ vọng. Việc phân loại theo nhóm giúp xác định lỗi thuộc dữ liệu, truy xuất, kiểm tra căn cứ hay sinh câu trả lời.

4.13 Kết luận chương

Chương 4 ghi nhận các chức năng giao diện, API, kiểm thử và bộ chỉ số đánh giá. Các phần này cho phép xem kết quả từ thao tác người dùng đến retrieval và hành vi từ chối trả lời.

CHƯƠNG V. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được

Hệ thống đã hình thành các thành phần chính: xử lý dữ liệu pháp luật, tìm kiếm kết hợp, phân tích câu hỏi, kiểm tra căn cứ, sinh câu trả lời, tạo trích dẫn, xác thực người dùng, lưu lịch sử hội thoại, feedback, bookmark và tra cứu nguồn pháp luật. Bộ đánh giá 40 trường hợp hỗ trợ kiểm tra retrieval, vị trí pháp lý, trích dẫn, hành vi từ chối trả lời và độ trễ. Các kết luận định lượng chỉ có ý nghĩa khi đi kèm số liệu đo tương ứng.

5.2 Hạn chế

Phạm vi dữ liệu hiện là corpus cục bộ, chưa đại diện toàn bộ pháp luật giao thông. Các quy tắc analyzer và reference parser tập trung vào một số mẫu câu tiếng Việt thường gặp nên không bao phủ mọi cách diễn đạt. Retrieval còn phụ thuộc chất lượng embedding và metadata đầu vào. Việc review độ đúng nội dung vẫn cần con người. Hệ thống cũng phụ thuộc OpenRouter cho model provider khi generation trực tuyến.

5.3 Hướng phát triển

Các phần còn có thể phát triển sau khóa luận gồm mở rộng corpus theo quy trình kiểm chứng nguồn, quản lý phiên bản văn bản và ngày hiệu lực, tăng số trường hợp trong bộ đánh giá, thử thêm embedding hoặc reranker, xử lý cross-reference chi tiết hơn, kiểm tra claim sau generation, bổ sung tracing và xây quy trình cập nhật corpus có kiểm thử hồi quy.

5.4 Kết luận chung

Hệ thống gần căn cứ có thể mở lại với mỗi câu trả lời pháp luật. Khi dữ liệu chưa đủ, backend trả trạng thái từ chối thay vì suy đoán. Trong phạm vi khóa luận, chất lượng retrieval, tính đúng của vị trí pháp lý và khả năng kiểm tra nguồn là các tiêu chí chính để đánh giá hệ thống.

TÀI LIỆU THAM KHẢO

- [1] **Nguyễn Chí Cường.** *Lập trình căn bản.* Khoa Kỹ thuật - Công nghệ, Đại học Tây Đô, 2023.
- [2] **Nguyễn Chí Cường.** *Phân tích và thiết kế hệ thống thông tin.* Khoa Kỹ thuật - Công nghệ, Đại học Tây Đô, 2024.
- [3] **Ngô Thị Lan.** *Nhập môn Công nghệ Phần mềm.* Khoa Kỹ thuật - Công nghệ, Đại học Tây Đô, 2024.
- [4] **Lâm Tấn Phương.** *Giáo trình Lý thuyết Thiết kế và lập trình Web.* Khoa Kỹ thuật - Công nghệ, Đại học Tây Đô, 2024.
- [5] **Lâm Tấn Phương.** *Chuyên đề ngôn ngữ lập trình.* Khoa Kỹ thuật - Công nghệ, Đại học Tây Đô, 2024.
- [6] **Qdrant.** *Combining Semantic and Lexical Search using Hybrid Search.* Tài liệu chính thức. <https://qdrant.tech/documentation/search/text-search/hybrid-search/>.
- [7] **Qdrant.** *Full-Text Search and BM25.* Tài liệu chính thức. <https://qdrant.tech/documentation/search/text-search/full-text-search/>.
- [8] **Qdrant.** *Vectors: Dense and Sparse Vectors.* Tài liệu chính thức. <https://qdrant.tech/documentation/manage-data/vectors/>.
- [9] **FastAPI.** *Features - OpenAPI, Validation, Pydantic and Dependency Injection.* Tài liệu chính thức. <https://fastapi.tiangolo.com/features/>.
- [10] **Supabase.** *Row Level Security.* Tài liệu chính thức. <https://supabase.com/docs/guides/database/postgres/row-level-security>.
- [11] **Vercel.** *Next.js App Router Documentation.* Tài liệu chính thức. <https://nextjs.org/docs/app>.
- [12] **OpenRouter.** *Embeddings API Documentation.* Tài liệu chính thức. <https://openrouter.ai/docs/client-sdks/typescript/api-reference/embeddings>.
- [13] **Python Software Foundation.** *Python 3.11 Documentation.* Tài liệu chính thức. <https://docs.python.org/3.11/>.
- [14] **Supabase.** *Auth Documentation.* Tài liệu chính thức. <https://supabase.com/docs/guides/auth>.
- [15] **Meta Open Source.** *React Documentation - Quick Start.* Tài liệu chính thức. <https://react.dev/learn>.

- [16] **Meta Open Source.** *React Documentation - Managing State*. Tài liệu chính thức. <https://react.dev/learn/managing-state>.
- [17] **Microsoft.** *The TypeScript Handbook*. Tài liệu chính thức. <https://www.typescriptlang.org/docs/handbook/intro.html>.
- [18] **Mozilla.** *PDF.js Getting Started*. Tài liệu chính thức. https://mozilla.github.io/pdf.js/getting_started/.
- [19] **Docker.** *Docker Compose Documentation*. Tài liệu chính thức. <https://docs.docker.com/reference/cli/docker/compose/>.
- [20] **OpenRouter.** *Chat Completions API Documentation*. Tài liệu chính thức. <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>.